

Digital Electronics

ECE 2105

SYLLABUS

Introduction of Combinational logic design: Overview of Boolean Algebra and K-Map, Half and Full Adders, Subtractors, Serial and Parallel Adders, BCD Adder. MSI devices: Comparators, Multiplexers, Encoder, Decoder, Driver & Multiplexed Display, Barrel shifter and ALU.

Sequential logic design: latch, Flip-flop, S-R FF, D FF, JK FF, T FF, and Master-Slave JK FF, Edge triggered FF, Ripple and Synchronous counters, Shift registers, Timing Analysis of sequential circuits.

Designing of State Machines: Finite state machines, Design of synchronous FSM, State Reduction, Timing issues in synchronous circuits. Algorithmic State Machines, Designing synchronous circuits like Pulse train generator, Pseudorandom Binary Sequence generator, Clock generation. Design of asynchronous circuits.

Logic Families and Semiconductor Memories: TTL NAND gate, Specifications, Noise margin, Propagation delay, fan-in, fan-out, Tristate TTL, ECL, CMOS families and their interfacing, Memory elements, Concept of Programmable logic devices, Logic implementation using Programmable Devices.

TEXT BOOKS:

1. A. A. Kumar, FUNDAMENTALS OF DIGITAL CIRCUITS. Prentice Hall India Pvt., Limited, (2e), 2016.
2. R. P. Jain, Modern Digital Electronics. McGraw-Hill Education (India) Pvt Limited, (4e), 2003.
3. W.H. Gothmann, Digital Electronics- An introduction to theory and practice, PHI, (2e), 2006.
4. S. Brown and Z. Vranesic, Fundamentals of Digital logic with Verilog Design, McGraw Hill, (3e) 2013.
5. M. Mano and M. Ciletti, “Digital Design: With an introduction to Verilog HDL”, Pearson, 2013.

Overview

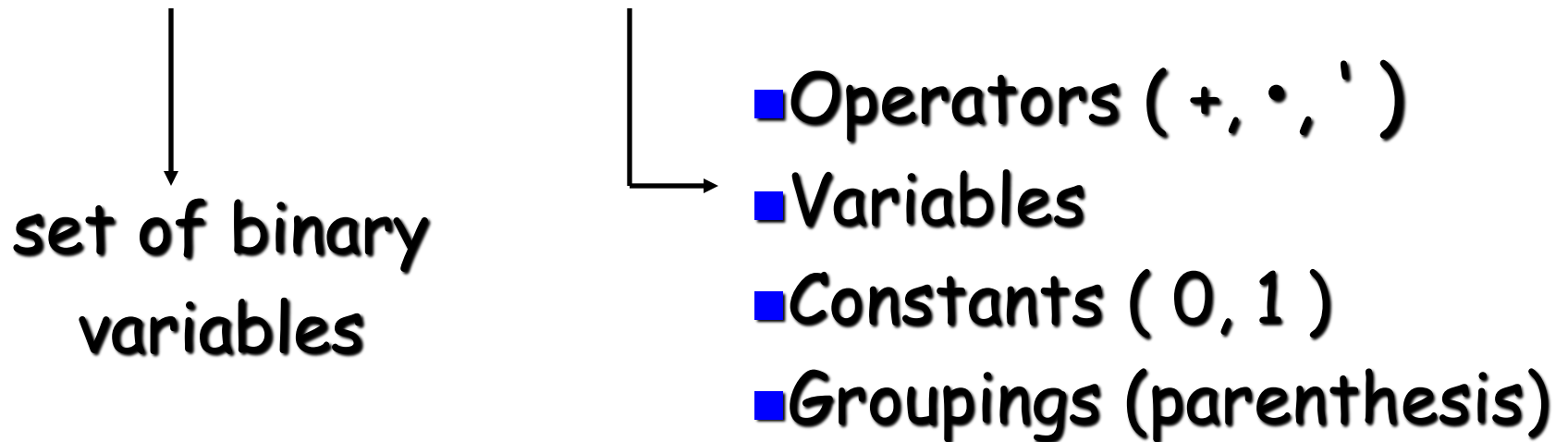
- Boolean Algebra
 - Basic Properties
 - Algebraic Manipulation
- Standard and Canonical Forms
 - Minterms and Maxterms (Canonical forms)
 - SOP and POS (Standard forms)
- Binary logic and Gates

Binary Logic

- Deals with binary variables that take 2 discrete values (0 and 1), and with logic operations
- Three basic logic operations:
 - AND, OR, NOT
- Binary/logic variables are typically represented as letters: A,B,C,...,X,Y,Z

Binary Logic Function

$F(\text{vars}) = \text{expression}$

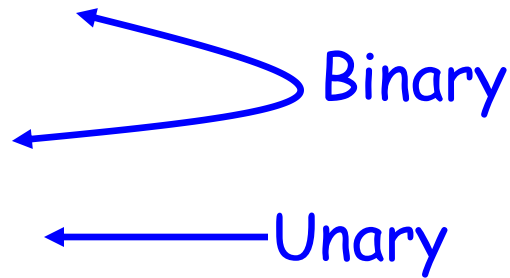


Example: $F(a,b) = a' \bullet b + b'$

$G(x,y,z) = x \bullet (y + z')$

Basic Logic Operators

- AND
- OR
- NOT



- $F(a,b) = a \bullet b$, F is 1 if and only if $a=b=1$
- $G(a,b) = a + b$, G is 1 if either $a=1$ or $b=1$
- $H(a) = a'$, H is 1 if $a=0$

Basic Logic Operators (cont.)

- 1-bit logic AND resembles binary multiplication:

$$0 \bullet 0 = 0, \quad 0 \bullet 1 = 0,$$

$$1 \bullet 0 = 0, \quad 1 \bullet 1 = 1$$

- 1-bit logic OR resembles binary addition, except for one operation:

$$0 + 0 = 0, \quad 0 + 1 = 1,$$

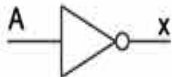
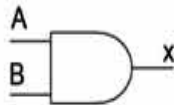
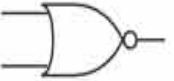
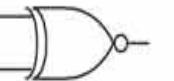
$$1 + 0 = 1, \quad 1 + 1 = 1 \ (\neq 10_2)$$

Que.: Let a function $F()$ depend on n variables. How many cases are there in the truth table of $F()$?

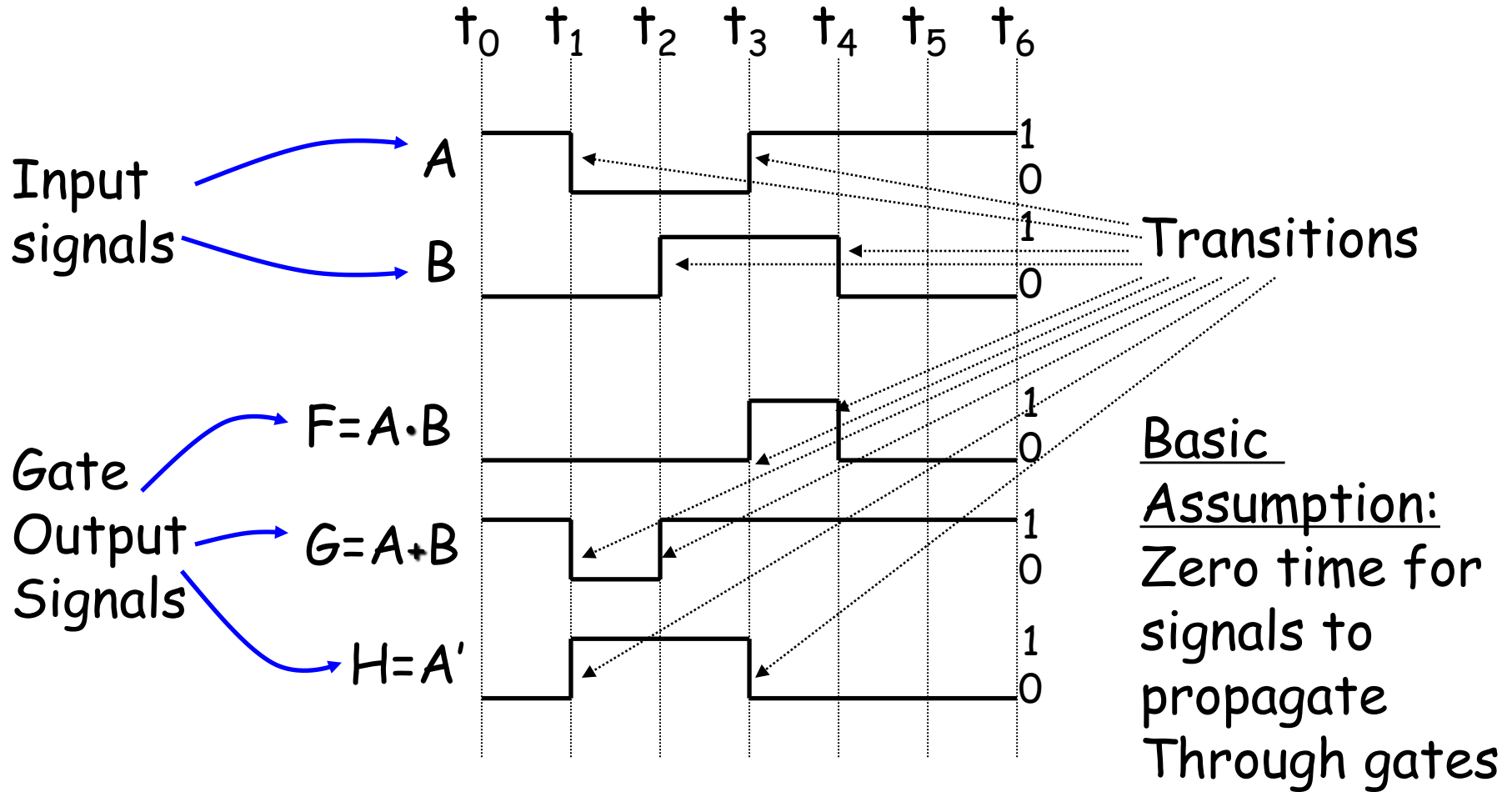
Ans.: 2^n rows, since there are 2^n possible binary patterns/combinations for the n variables

Logic Gates

- Logic gates are abstractions of electronic circuit components that operate on one or more input signals to produce an output signal.

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR																																																																																																
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A\oplus B$	$\overline{A\oplus B}$																																																																																																
Symbol																																																																																																							
Truth Table	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	0	1	0	0	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	1
A	X																																																																																																						
0	1																																																																																																						
1	0																																																																																																						
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					

Timing Diagram

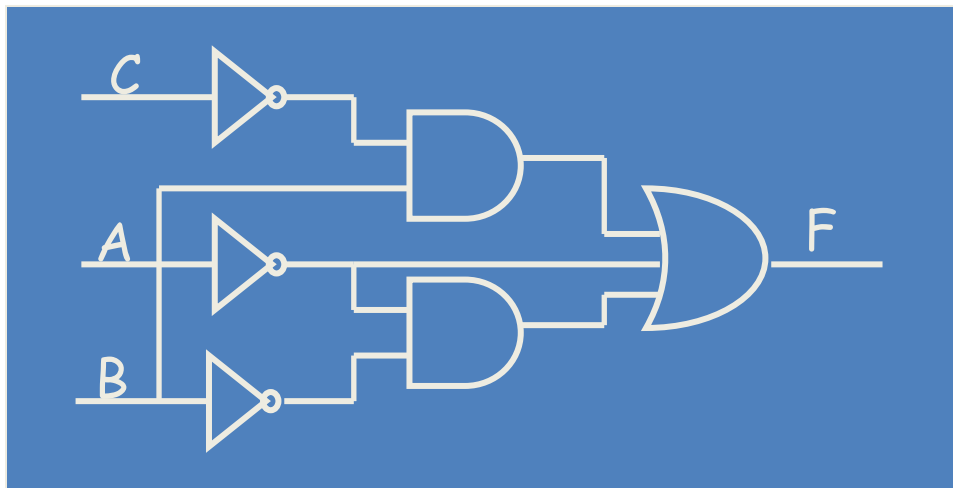


Logic Circuit from Logic Function

- Consider function $F = A' + B \cdot C' + A' \cdot B'$

A combinational logic circuit can be constructed to implement F , by appropriately connecting input signals and logic gates:

- Circuit input signals $\rightarrow$ from function variables (A, B, C)
- Circuit output signal $\rightarrow$ function output (F)
- Logic gates $\rightarrow$ from logic operations



Combinational Logic Circuit from Logic Function (cont.)

- In order to design a cost-effective and efficient circuit, we must minimize the circuit's size (area) and propagation delay (time required for an input signal change to be observed at the output line)

- Observe the truth table of

$$F = A' + B \cdot C' + A' \cdot B'$$

$$F = A' (1 + B') + B \cdot C'$$

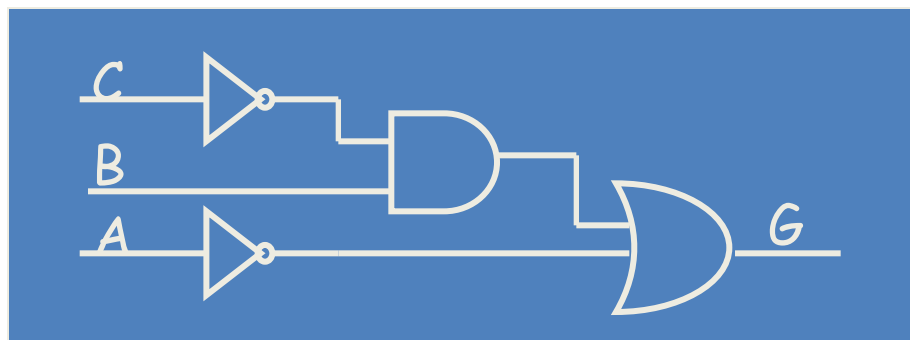
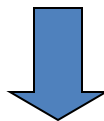
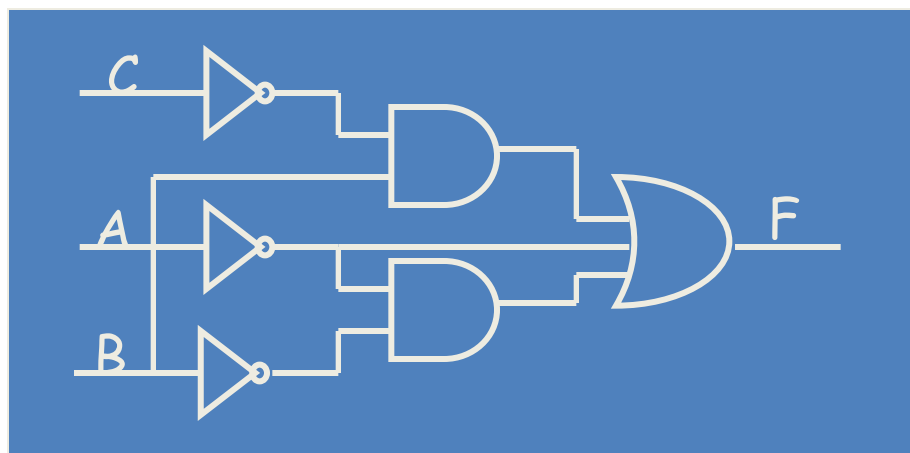
$$G = A' + BC'$$

Truth tables for F and G are identical →
same function

- Use G to implement the logic circuit (less components)

A	B	C	F	G
0	0	0	1	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0

Combinational Logic Circuit from Logic Function (cont.)



Boolean Algebra Properties

Let X: Boolean variable, 0,1: constants

1. $X + 0 = X$ -- Zero Axiom
2. $X \cdot 1 = X$ -- Unit Axiom
3. $X + 1 = 1$ -- Unit Property
4. $X \cdot 0 = 0$ -- Zero Property
5. $X + X = X$ -- Idepotence
6. $X \cdot X = X$ -- Idepotence
7. $X + X' = 1$ -- Complement
8. $X \cdot X' = 0$ -- Complement
9. $(X')' = X$ -- Involution
10. $X + Y = Y + X$
11. $X \cdot Y = Y \cdot X$ -- Commutative
12. $X + (Y + Z) = (X + Y) + Z$
13. $X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z$ -- Associative
14. $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$
15. $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$ -- Distributive
16. $(X + Y)' = X' \cdot Y'$
17. $(X \cdot Y)' = X' + Y'$ -- DeMorgan's

And many more...

Truth Tables (cont.)

- Truth table: a unique representation of a Boolean function
- Tabular form that represents the relationship between the input variables of a function and its output
- If two functions have identical truth tables, the functions are equivalent (and vice-versa).
- Truth tables can be used to prove equality theorems.
- However, the size of a truth table grows exponentially with the number of variables involved, hence unwieldy. This motivates the use of Boolean Algebra.

Boolean expressions-NOT unique

- Unlike truth tables, expressions representing a Boolean function are NOT unique.
- Example:
 - $F(x,y,z) = x' \cdot y' \cdot z' + x' \cdot y \cdot z' + x \cdot y \cdot z'$
 - $G(x,y,z) = x' \cdot y' \cdot z' + y \cdot z'$
- The corresponding truth tables for $F(x,y,z)$ and $G(x,y,z)$ are to the right. They are identical.
- Thus, $F(x,y,z) = G(x,y,z)$

x	y	z		F	G
0	0	0		1	1
0	0	1		0	0
0	1	0		1	1
0	1	1		0	0
1	0	0		0	0
1	0	1		0	0
1	1	0		1	1
1	1	1		0	0

Algebraic Manipulation

- Boolean algebra is a useful tool for simplifying digital circuits.
- Why do it? Simpler can mean cheaper, smaller, faster.
- Example: Simplify $F = x'yz + x'yz' + xz$.

$$\begin{aligned} F &= x'yz + x'yz' + xz \\ &= x'y(z+z') + xz \\ &= x'y \cdot 1 + xz \\ &= x'y + xz \end{aligned}$$

Algebraic Manipulation (cont.)

- Example: Prove

$$x'y'z' + x'yz' + xyz' = x'z' + yz'$$

- **Proof:**

$$\begin{aligned} & x'y'z' + x'yz' + xyz' \\ &= x'y'z' + x'yz' + x'yz' + xyz' \\ &= x'z'(y' + y) + yz'(x' + x) \\ &= x'z' \cdot 1 + yz' \cdot 1 \\ &= x'z' + yz' \end{aligned}$$

Canonical and Standard Forms

- We need to consider formal techniques for the simplification of Boolean functions.
 - Identical functions will have exactly the same canonical form.
 - Minterms and Maxterms
 - Sum-of-Minterms and Product-of- Maxterms
 - Product and Sum terms
 - Sum-of-Products (SOP) and Product-of-Sums (POS)

Definitions

- *Literal*: A variable or its complement
- *Product term*: literals connected by $\cdot$
- *Sum term*: literals connected by $+$
- *Minterm*: a product term in which all the variables appear exactly once, either complemented or actual literal
- *Maxterm*: a sum term in which all the variables appear exactly once, either complemented or actual literal.

Minterm

- Represents exactly one combination in the truth table.
- Denoted by m_j , where j is the decimal equivalent of the minterm's corresponding binary combination (b_j).
- A variable in m_j is complemented if its value in b_j is 0, otherwise its value in b_j is 1.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding minterm is denoted by

$$m_j = A'BC$$

Maxterm

- Represents exactly one combination in the truth table.
- Denoted by M_j , where j is the decimal equivalent of the maxterm's corresponding binary combination (b_j).
- A variable in M_j is complemented if its value in b_j is 1, otherwise its value in b_j is 0.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding maxterm is denoted by

$$M_j = A+B'+C'$$

Truth Table notation for Minterms and Maxterms

- Minterms and Maxterms are easy to denote using a truth table.
- Example:
Assume 3 variables x,y,z
(order is fixed)

x	y	z	Minterm	Maxterm
0	0	0	$x'y'z' = m_0$	$x+y+z = M_0$
0	0	1	$x'y'z = m_1$	$x+y+z' = M_1$
0	1	0	$x'yz' = m_2$	$x+y'+z = M_2$
0	1	1	$x'yz = m_3$	$x+y'+z' = M_3$
1	0	0	$xy'z' = m_4$	$x'+y+z = M_4$
1	0	1	$xy'z = m_5$	$x'+y+z' = M_5$
1	1	0	$xyz' = m_6$	$x'+y'+z = M_6$
1	1	1	$xyz = m_7$	$x'+y'+z' = M_7$

Canonical Forms (Unique)

- Any Boolean function $F()$ can be expressed as a *unique* **sum** of **minterms** and a unique **product** of **maxterms** (under a fixed variable ordering).
- In other words, every function $F()$ has two canonical forms:
 - Canonical Sum-Of-Products (sum of minterms)
 - Canonical Product-Of-Sums (product of maxterms)

Canonical Forms (cont.)

- Canonical Sum-Of-Products:

The minterms included are those m_j such that $F() = 1$ in row j of the truth table for $F()$.

$$F = ABC + ABC' + AB'C'$$

- Canonical Product-Of-Sums:

The maxterms included are those M_j such that $F() = 0$ in row j of the truth table for $F()$.

$$F = (A' + B' + C')(A' + B' + C)(A' + B + C)$$

Example

- Truth table for $f_1(a,b,c)$ at right
- The canonical sum-of-products form for f_1 is

$$\begin{aligned} f_1(a,b,c) &= m_1 + m_2 + m_4 + m_6 \\ &= a'b'c + a'bc' + ab'c' + abc' \end{aligned}$$

- The canonical product-of-sums form for f_1 is

$$\begin{aligned} f_1(a,b,c) &= M_0 \cdot M_3 \cdot M_5 \cdot M_7 \\ &= (a+b+c) \cdot (a+b'+c') \cdot \\ &\quad (a'+b+c') \cdot (a'+b'+c'). \end{aligned}$$

Observe that: $m_j = M_j'$

a	b	c	f_1
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Shorthand: $\sum$ and $\prod$

- $f_1(a,b,c) = \sum m(1,2,4,6)$, where $\sum$ indicates that this is a sum-of-products form, and $m(1,2,4,6)$ indicates that the minterms to be included are m_1 , m_2 , m_4 , and m_6 .
- $f_1(a,b,c) = \prod M(0,3,5,7)$, where $\prod$ indicates that this is a product-of-sums form, and $M(0,3,5,7)$ indicates that the maxterms to be included are M_0 , M_3 , M_5 , and M_7 .
- Since $m_j = M_j'$ for any j ,
$$F_1(a,b,c) = \sum m(1,2,4,6) = \prod M(0,3,5,7)$$

Conversion Between Canonical Forms

- Replace $\sum$ with $\prod$ (or *vice versa*) and replace those j 's that appeared in the original form with those that do not.

- Example:

$$\begin{aligned}f_1(a,b,c) &= a'b'c + a'bc' + ab'c' + abc' \\&= m_1 + m_2 + m_4 + m_6 \\&= \sum(1,2,4,6) \\&= \prod(0,3,5,7) \\&= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c')\end{aligned}$$

Standard Forms (NOT Unique)

- Standard forms are “*like*” canonical forms, except that not all variables need appear in the individual product (SOP) or sum (POS) terms.
- Example:
$$f_1(a,b,c) = a'b'c + bc' + ac'$$

is a *standard* sum-of-products form
- $$f_1(a,b,c) = (a+b+c) \cdot (b'+c') \cdot (a'+c')$$

is a *standard* product-of-sums form.

Conversion of SOP from standard to canonical form

- Expand *non-canonical* terms by inserting equivalent of 1 in each missing variable x:

$$(x + x') = 1$$

- Remove duplicate minterms

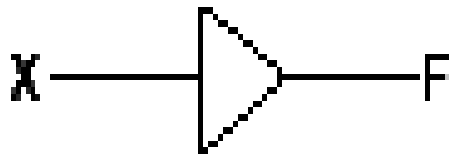
- $$\begin{aligned} f_1(a,b,c) &= a'b'c + bc' + ac' \\ &= a'b'c + (a+a')bc' + a(b+b')c' \\ &= a'b'c + abc' + a'bc' + abc' + ab'c' \\ &= a'b'c + abc' + a'bc + ab'c' \end{aligned}$$

Conversion of POS from standard to canonical form

- Expand noncanonical terms by adding 0 in terms of missing variables (*e.g.*, $xx' = 0$) and using the distributive law
- Remove duplicate maxterms

- $$\begin{aligned} f_1(a,b,c) &= (a+b+c) \cdot (b'+c') \cdot (a'+c') \\ &= (a+b+c) \cdot (\textcolor{red}{a}a' + b' + c') \cdot (a' + \textcolor{red}{b}b' + c') \\ &= (a+b+c) \cdot (a+b'+c') \cdot (\textcolor{blue}{a}' + \textcolor{blue}{b}' + \textcolor{blue}{c}') \cdot \\ &\quad (a'+b+c') \cdot (\textcolor{blue}{a}' + \textcolor{blue}{b}' + \textcolor{blue}{c}') \\ &= (a+b+c) \cdot (a+b'+c') \cdot (a'+b'+c') \cdot (a'+b+c') \end{aligned}$$

BUFFER



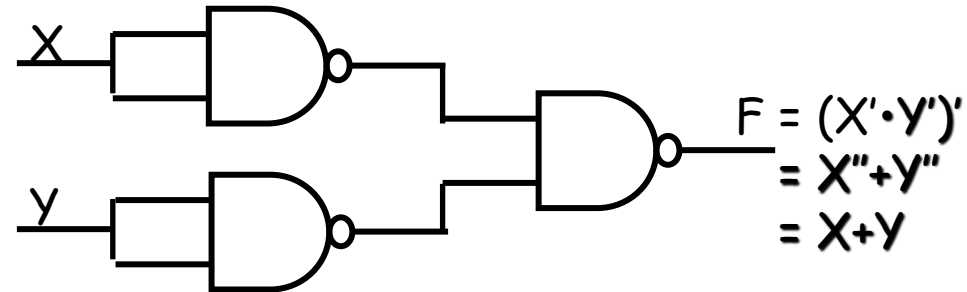
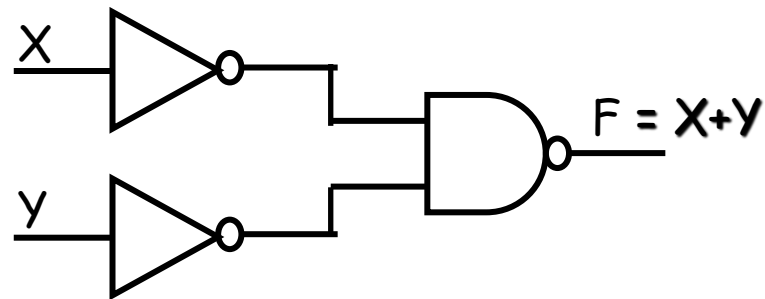
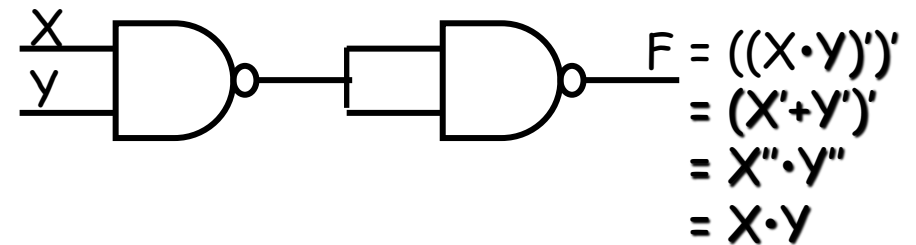
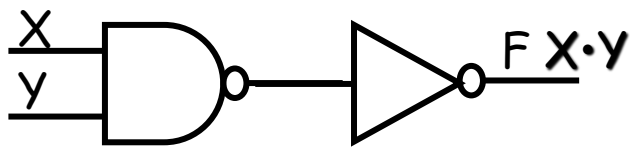
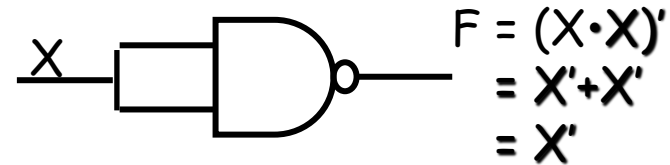
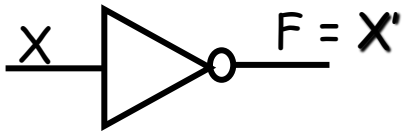
$$F = X$$

X	F
0	0
1	1

NAND Gate

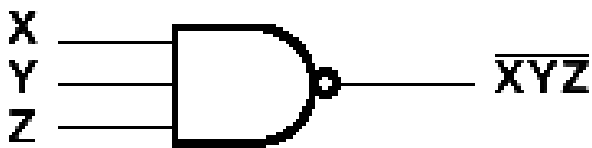
- Known as a “universal” gate because ANY digital circuit can be implemented with NAND gates alone.
- To prove the above, it suffices to show that AND, OR, and NOT can be implemented using NAND gates only.

NAND Gate Emulation

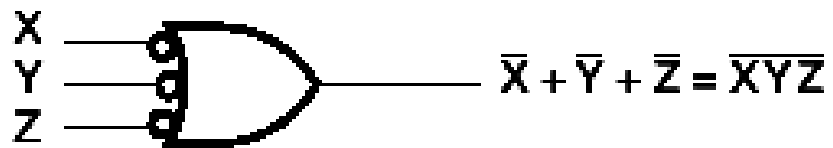


NAND Circuits

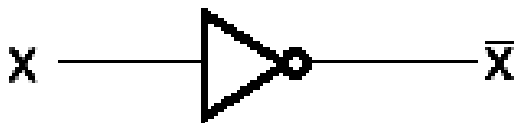
- To easily derive a NAND implementation of a Boolean function:
 - Find a simplified SOP
 - Design SOP with basic gates
 - Change AND-OR circuit to a NAND circuit using bubble
 - Use the alternative symbols below



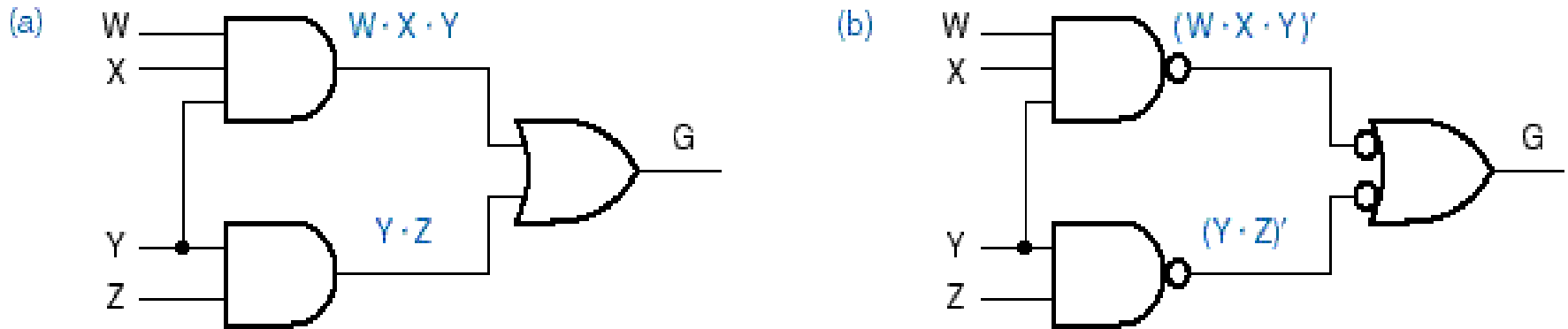
(a) AND - NOT



(b) NOT - OR



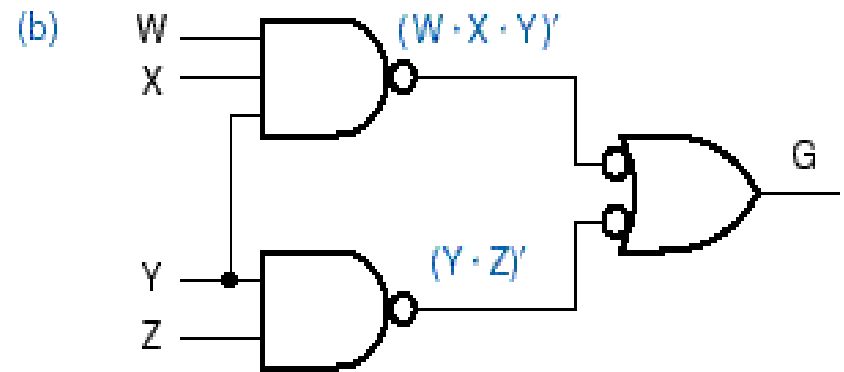
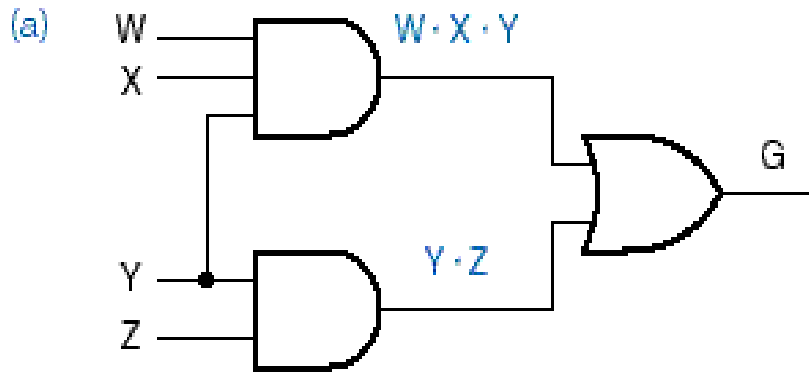
AND-OR (SOP) Emulation Using NANDs



Two-level implementations

- a) Original SOP
- b) Implementation with NANDs

AND-OR (SOP) Emulation Using NANDs (cont.)



Verify:

(a) $G = WXY + YZ$

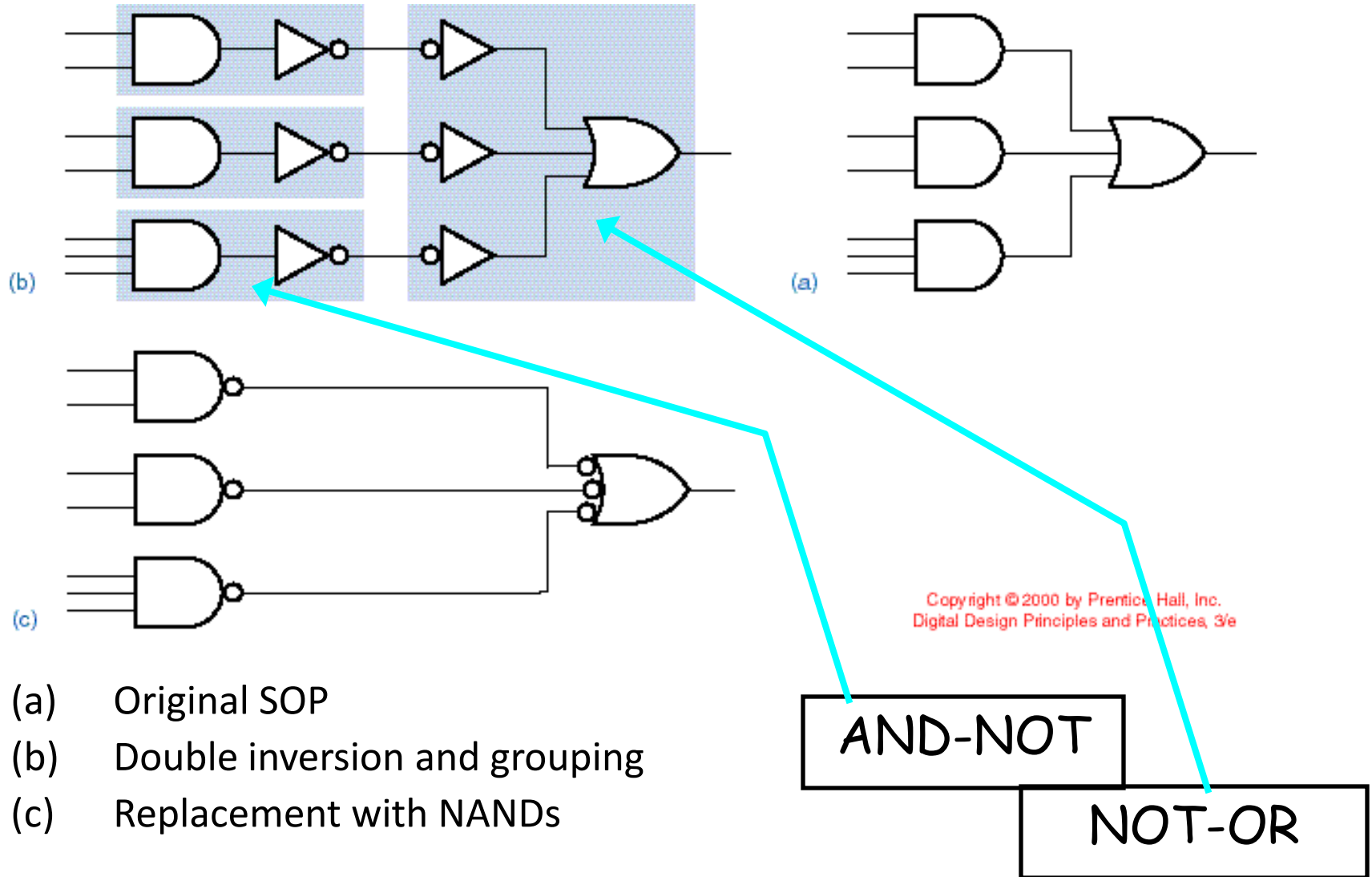
(b) $G = ((WXY)' \cdot (YZ)')'$
 $= (WXY)'' + (YZ)'' = WXY + YZ$

Multilevel NAND Circuits

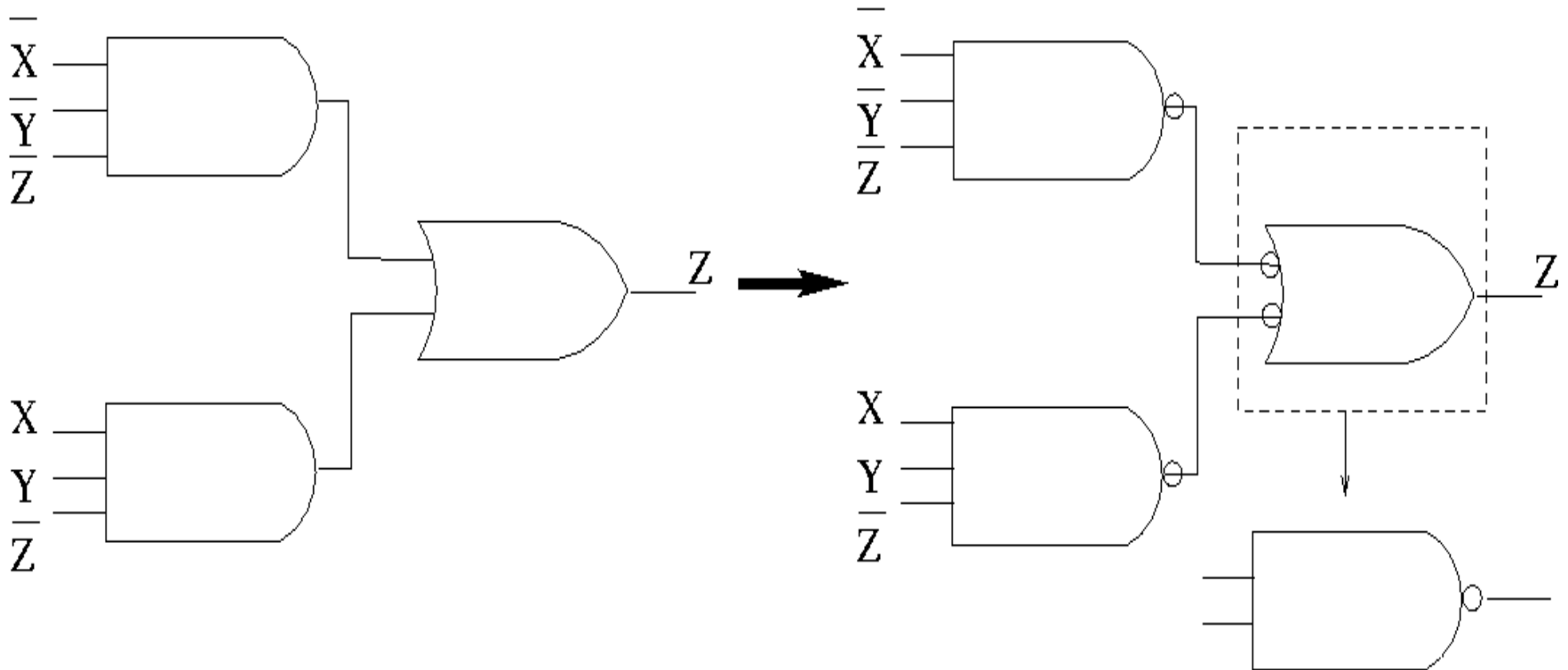
Starting from a multilevel circuit:

1. Convert all AND gates to NAND gates with AND-NOT graphic symbols.
2. Convert all OR gates to NAND gates with NOT-OR graphic symbols.
3. Check all the bubbles in the diagram. For every bubble that is not counteracted by another bubble along the same line, insert a NOT gate or complement the input literal from its original appearance.

SOP with NAND



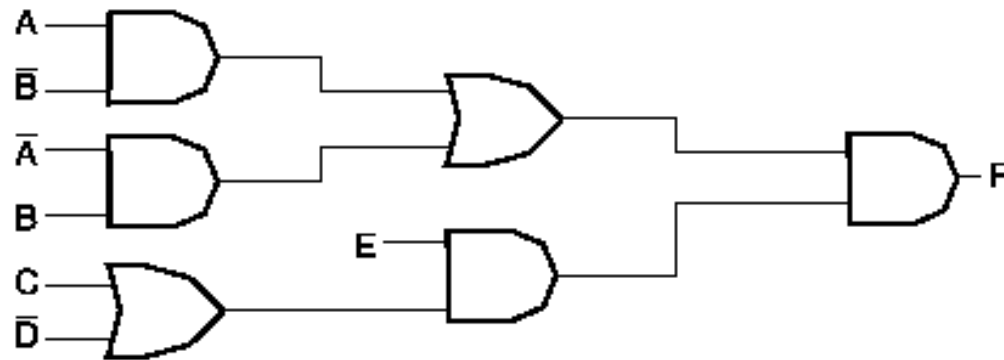
Example (cont.)



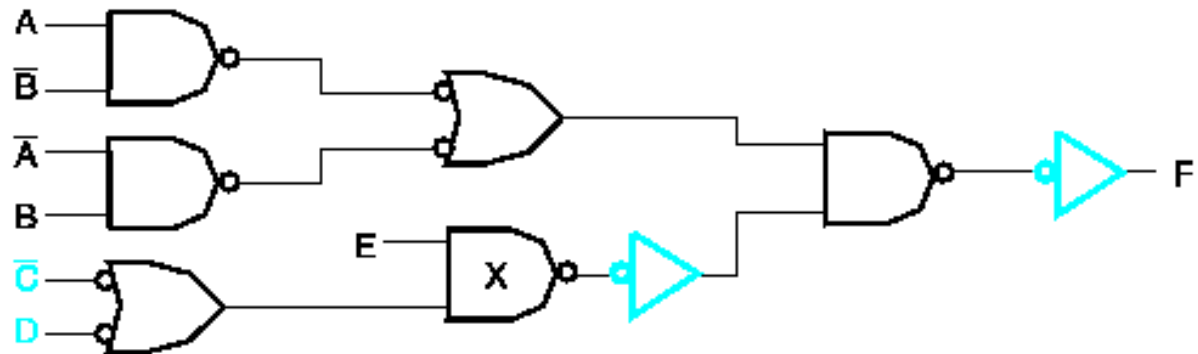
Two-level implementation with NANDs

$$F = X'Y'Z' + XYZ'$$

Another Example!



(a) AND – OR gates



(b) NAND gates

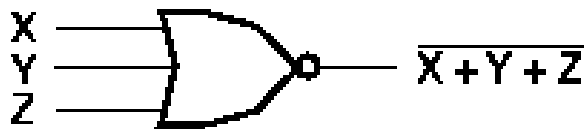
Fig. 2-32 Implementing $F = (A\bar{B} + \bar{A}B)E(C + \bar{D})$

NOR Gate

- Also a “universal” gate because ANY digital circuit can be implemented with NOR gates alone.
- This can be similarly proven as with the NAND gate.

NOR Circuits

- To easily derive a NOR implementation of a Boolean function:
 - Find a simplified POS
 - Design POS using basic gates
 - Change OR-AND circuit to a NOR circuit
 - Use the alternative symbols below

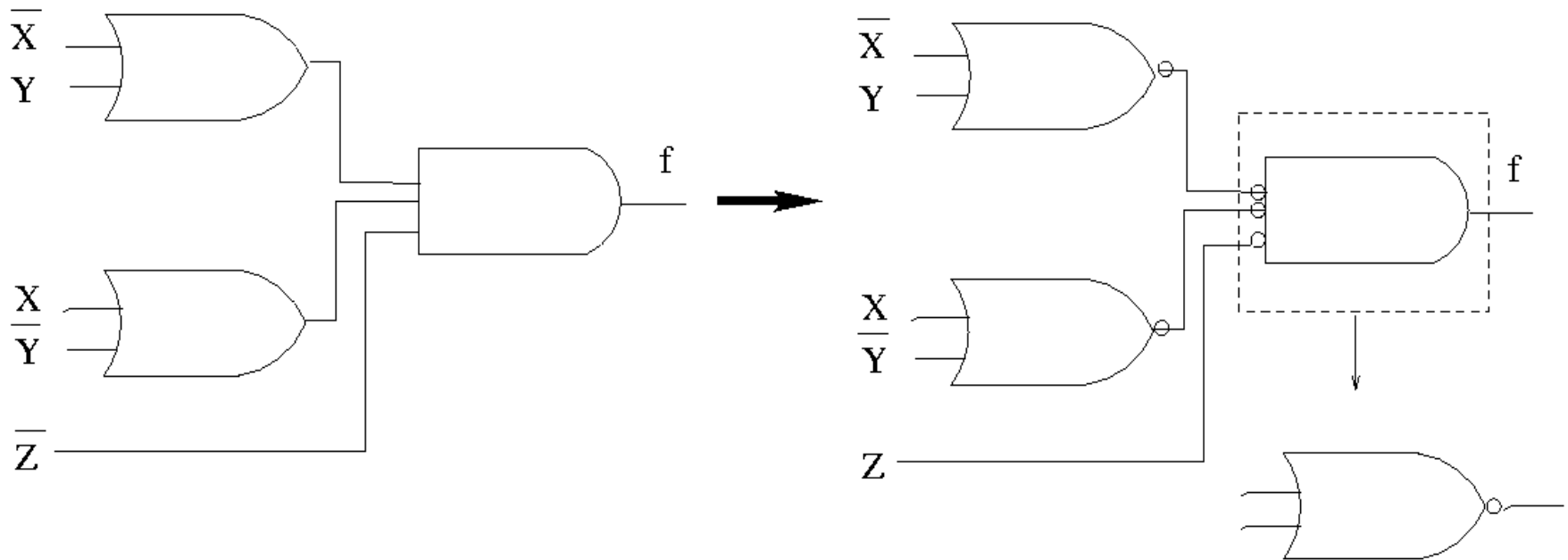


(a) OR – NOT



(b) NOT – AND

Example (cont.)



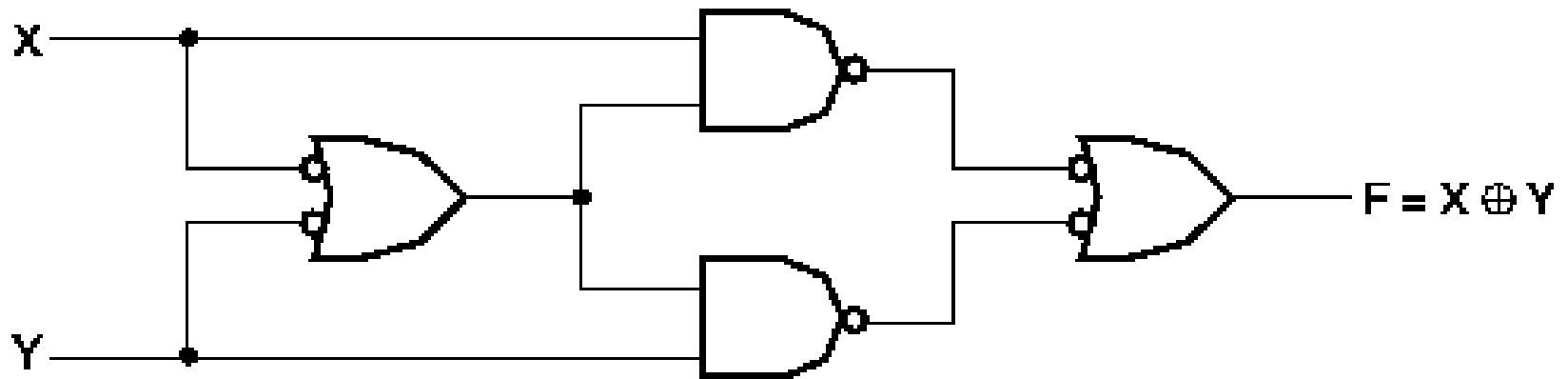
Two-level implementation with NORs

$$F = (F')' = (X' + Y)(X + Y')Z'$$

XOR function implementation

- $\text{XOR}(a,b) = ab' + a'b$
- Straightforward: 5 gates
 - 2 inverters, two 2-input ANDs, one 2-input OR
 - 2 inverters & 3 2-input NANDs
- Nonstraightforward:
 - 4 NAND gates

XOR circuit with 4 NANDs



Karnaugh Map

Karnaugh Map

The Karnaugh map, also known as the K-map, is a method to simplify Boolean algebra expressions.

The Karnaugh map reduces the need for extensive calculations by taking advantage of humans' pattern-recognition capability.

Karnaugh Map: A graphical technique for simplifying a Boolean expression into either form:

- minimal sum of products (MSP)
- minimal product of sums (MPS)

Goal of the simplification.

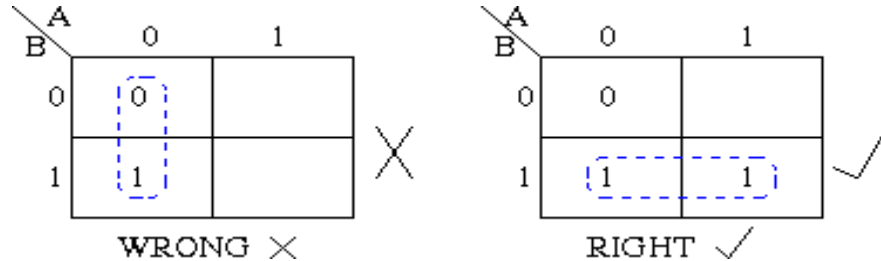
- There are a minimal number of product/sum terms
- Each term has a minimal number of literals

These terms can be used to write a minimal Boolean expression representing the required logic.

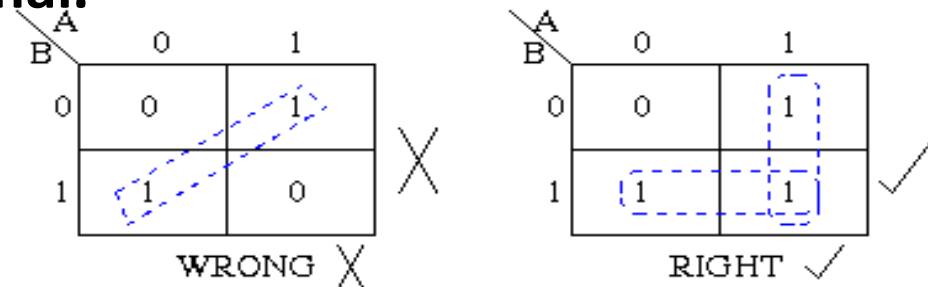
Karnaugh Maps - Rules of Simplification

The Karnaugh map uses the following rules for the simplification of expressions by grouping together adjacent cells containing *ones*.

1.Groups may not include any cell containing a zero.



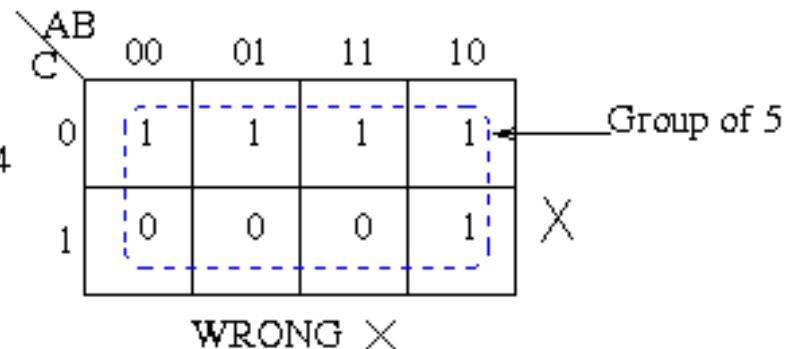
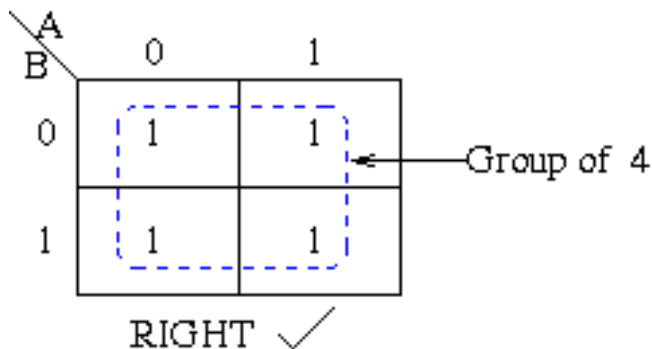
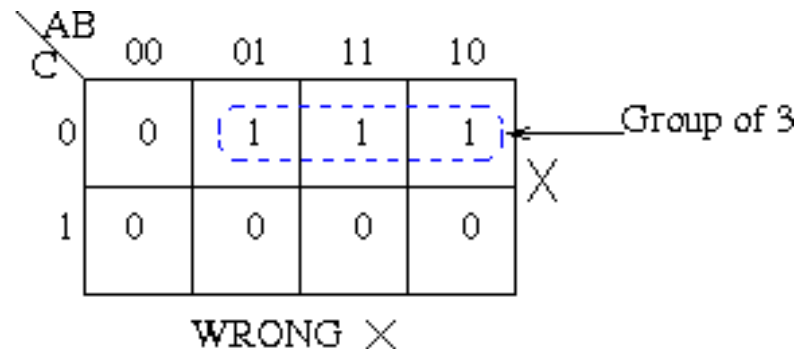
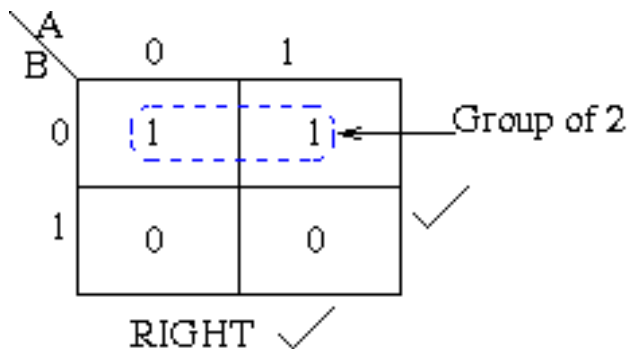
2.Groups may be horizontal or vertical, but not diagonal.



3. Groups must contain 1, 2, 4, 8 cells or in general 2^n cells.

That is if $n = 1$, a group will contain two 1's since $2^1 = 2$.

If $n = 2$, a group will contain four 1's since $2^2 = 4$.



4. Each group should be as large as possible.

$\overline{C}$ \ AB	00	01	11	10
0	1	1	1	1
1	0	0	1	1

RIGHT ✓

$\overline{C}$ \ AB	00	01	11	10
0	1	1	1	1
1	0	0	1	1

WRONG ✗

(Note that no Boolean laws broken,
but not sufficiently minimal)

5. Each cell containing a one must be in at least one group.

$\overline{C}$ \ AB	00	01	11	10
0	0	0	1	1
1	0	0	0	1

Group I
Group II

1 present in at least one group.

6.Groups may overlap.

AB		00	01	11	10
C					
0		1	1	1	1
1		0	0	1	1

Groups overlapping. ✓

RIGHT ✓

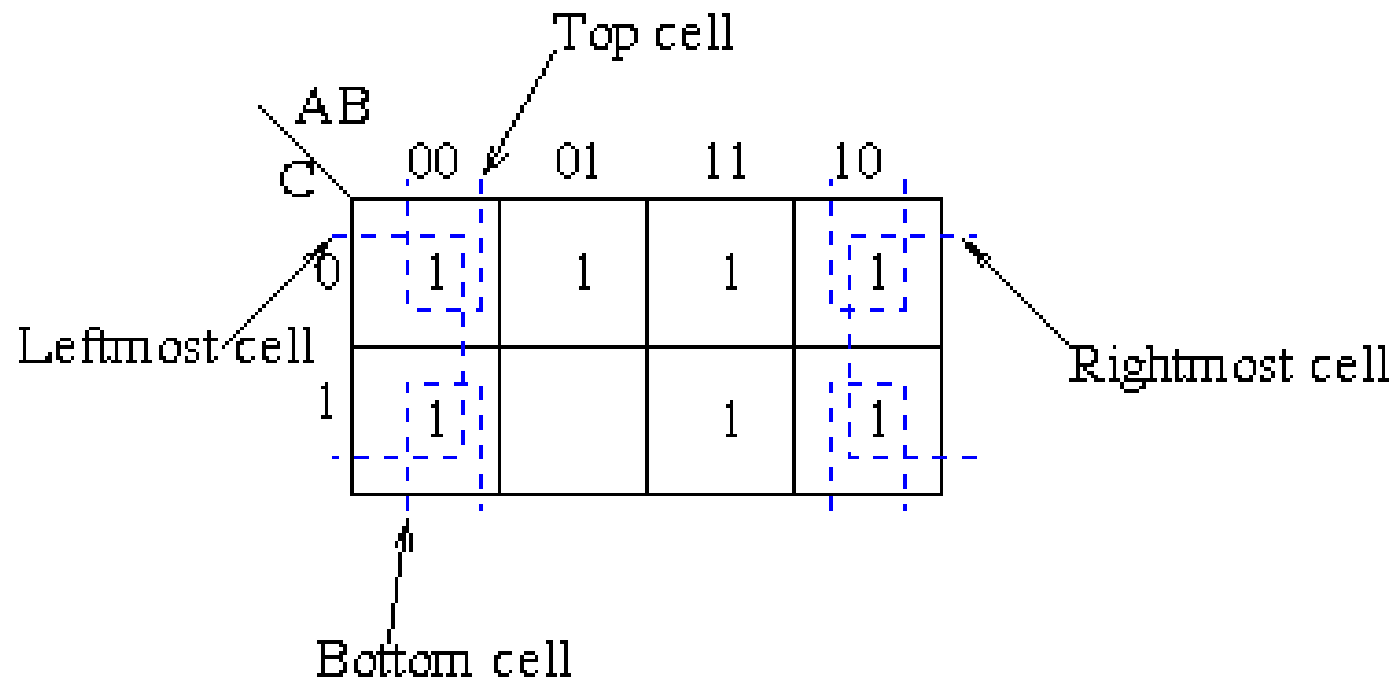
AB		00	01	11	10
C					
0		1	1	1	1
1		0	0	1	1

Groups not overlapping. ✗

WRONG ✗

7.Groups may wrap around the table.

The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.



8. There should be as few groups as possible, as long as this does not contradict any of the previous rules.

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

RIGHT ✓

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

WRONG ✗

Summmmary:

- No zeros allowed.
- No diagonals.
- Only power of 2 number of cells in each group.
- Groups should be as large as possible.
- Every one must be in at least one group.
- Overlapping allowed.
- Wrap around allowed.
- Fewest number of groups possible.

Examples

Two-Variable K-Map

		a	
		0	1
b	0		
	1		

Karnaugh maps

- Karnaugh maps, or K-maps, are often used to simplify logic problems with 2, 3 or 4 variables.

Cell = 2^n , where n is a number of variables

For the case of 2 variables, we form a map consisting of $2^2=4$ cells as shown in Figure

		A	
		0	1
B	0	$A + B$	$\bar{A} + B$
	1	$A + \bar{B}$	$\bar{A} + \bar{B}$

Maxterm

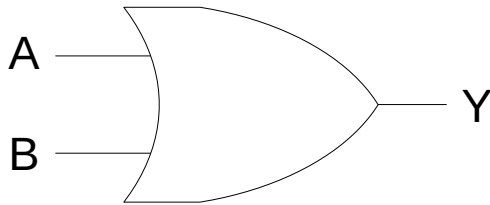
		A	
		0	1
B	0	00 0	10 2
	1	01 1	11 3

		A	
		0	1
B	0	$\bar{A}\bar{B}$	$A\bar{B}$
	1	$\bar{A}B$	AB

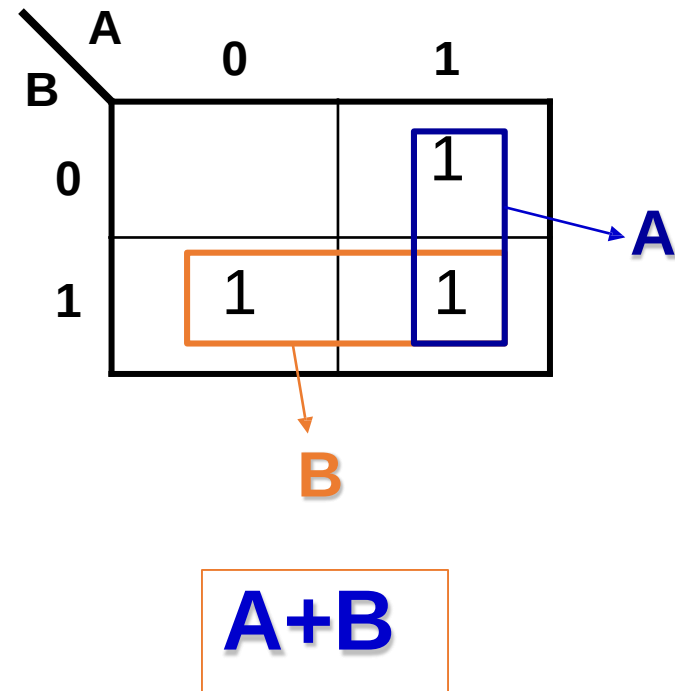
Minterm

Example

2-variable Karnaugh maps are trivial but can be used to introduce the methods you need to learn. The map for a 2-input OR gate looks like this:



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



Example 1:

Consider the following map. The function plotted is: $Z = f(A,B) = AB' + AB$

B \ A	0	1
0		1
1		1

Note that values of the input variables form the rows and columns.

That is the logic values of the variables A and B (with one denoting true form and zero denoting false form) form the head of the rows and columns respectively.

The map displayed is a one dimensional type which can be used to simplify an expression in two variables.

There is a two-dimensional map that can be used for up to four variables, and a three-dimensional map for up to six variables.

Referring to the map above, the two adjacent 1's are grouped together.

Through inspection it can be seen that variable B has its true and false form within the group.

This eliminates variable B leaving only variable A which only has its true form.

The minimized answer therefore is $Z = A$.

Karnaugh maps

- 3 variables Karnaugh map

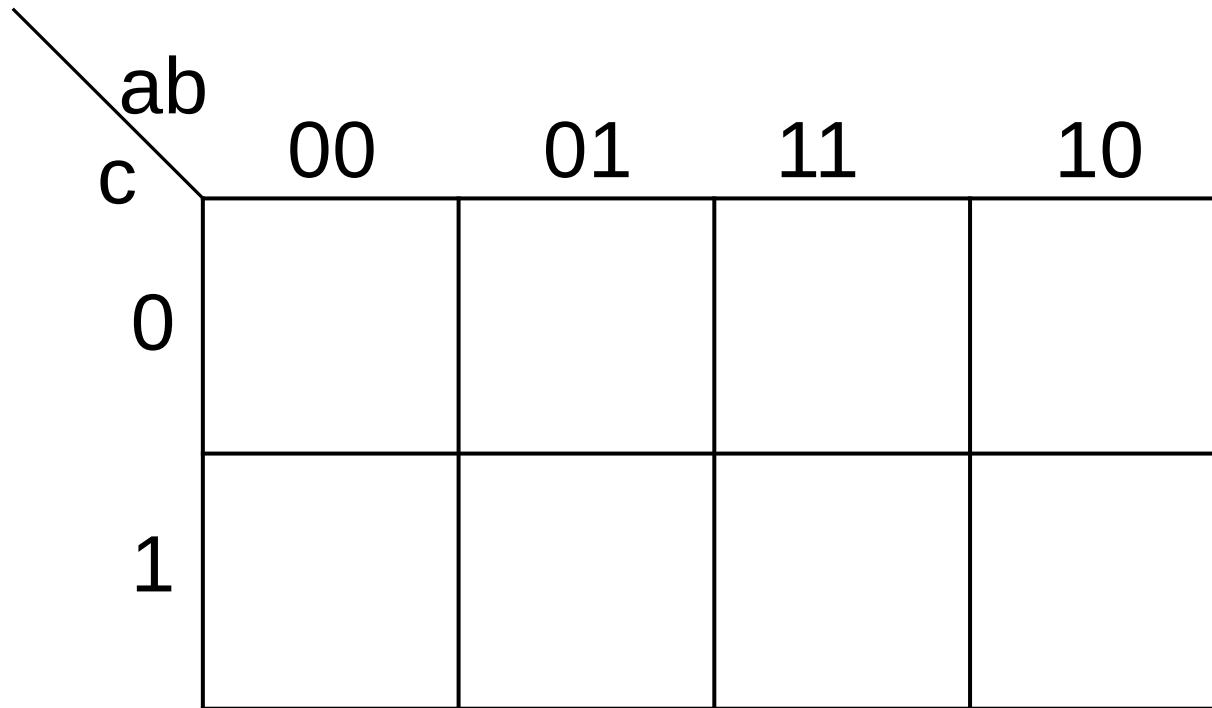
$$\text{Cell} = 2^3 = 8$$

AB C		AB			
		00	01	11	10
C	0	0 $\overline{A}\overline{B}\overline{C}$	2 $\overline{A}B\overline{C}$	6 $AB\overline{C}$	4 $A\overline{B}\overline{C}$
	1	1 $\overline{A}\overline{B}C$	3 $\overline{A}BC$	7 ABC	5 $A\overline{B}C$

Three-Variable K-Map

		ab			
		00	01	11	10
c	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

Three-Variable K-Map



ab \ c	00	01	11	10
0				
1				

Edges are adjacent



Terminology

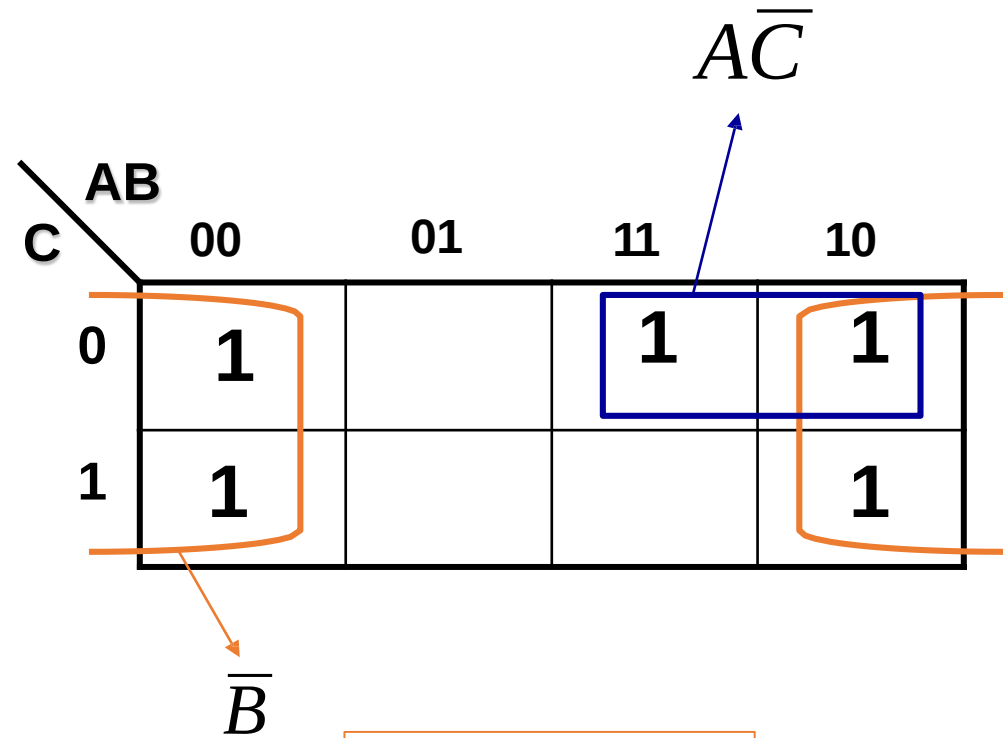
- Implicant
 - Product term that could be used to cover minterms of a function
- Prime Implicant
 - An implicant that is not part of another implicant
- Essential Prime Implicant
 - An implicant that covers at least one minterm that is not contained in another prime implicant

Guidelines for Simplifying Functions

- Group as many squares as possible. This eliminates the most variables.
- Make as few groups as possible. Each group represents a separate product term.
- You must **cover** each minterm at least once. However, it may be covered more than once.

Example

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



$$\overline{B} + A\overline{C}$$

Three-Variable K-Map Example

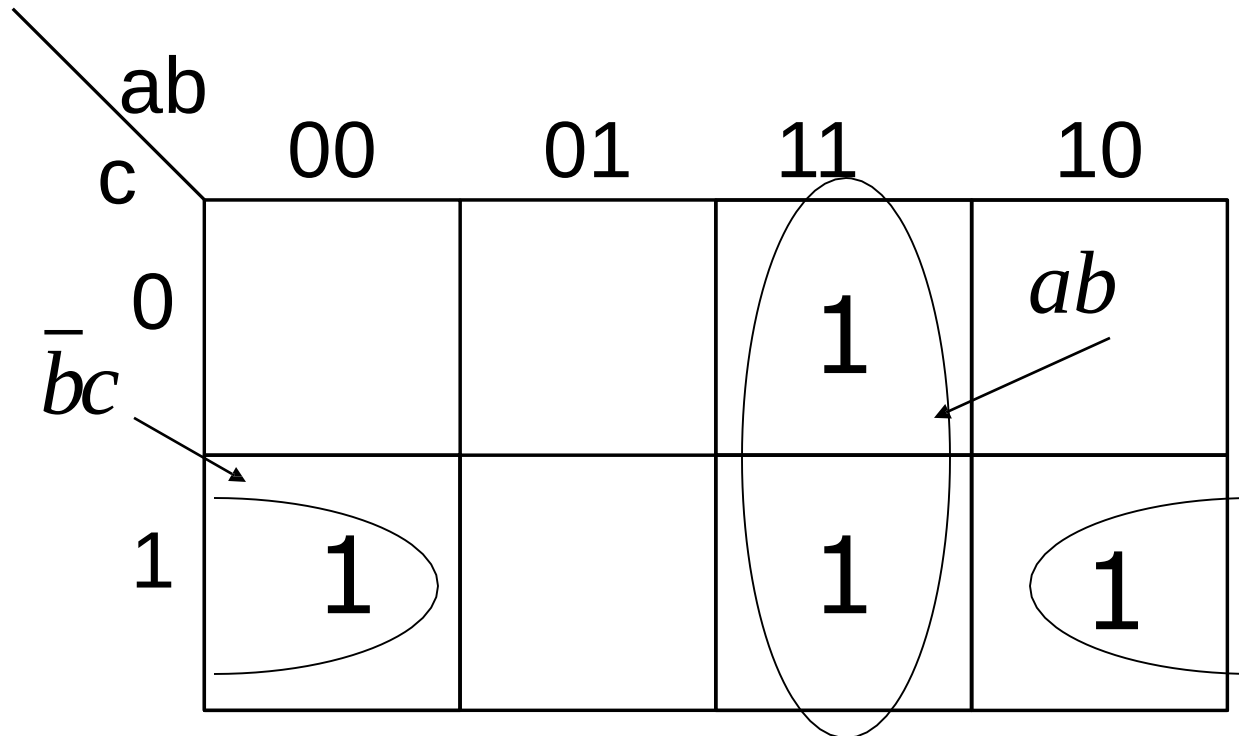
Plot 1's (minterms) of switching function

ab \ c		ab			
		00	01	11	10
c	0			1	
	1	1	1		1

$$F(a,b,c) = \sum m(1,3,5,6)$$

Three-Variable K-Map Example

Plot 1's (minterms) of switching function



$$F(a, b, c) = ab + \bar{b}c$$

Three-Variable K-Maps

$$f = \sum (0,4) = \overline{B}\overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	0
	1	1	0	0	0

$$f = \sum (4,5) = A\overline{B}$$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1	1	1	0	0

$$f = \sum (0,1,4,5) = \overline{B}$$

		BC			
		00	01	11	10
A	0	1	1	0	0
	1	1	1	0	0

$$f = \sum (0,1,2,3) = \overline{A}$$

		BC			
		00	01	11	10
A	0	1	1	1	1
	1	0	0	0	0

$$f = \sum (0,4) = \overline{A}C$$

		BC			
		00	01	11	10
A	0	0	1	1	0
	1	0	0	0	0

$$f = \sum (4,6) = A\overline{C}$$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1	1	0	0	1

$$f = \sum (0,2) = \overline{A}\overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	1
	1	0	0	0	0

$$f = \sum (0,2,4,6) = \overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	1
	1	1	0	0	1

Three-Variable K-Map Examples

A \ BC	BC			
	00	01	11	10
0		1		
1	1		1	1

A \ BC	BC			
	00	01	11	10
0	1		1	1
1	1			1

A \ BC	BC			
	00	01	11	10
0			1	1
1	1	1		

A \ BC	BC			
	00	01	11	10
0			1	
1	1		1	1

A \ BC	BC			
	00	01	11	10
0		1	1	1
1		1	1	

A \ BC	BC			
	00	01	11	10
0	1		1	
1		1	1	1

Example

- Use a K-Map to simplify the following Boolean expression

$$F(a, b, c) = \sum m(2, 3, 6, 7)$$

Three-Variable K-Map Example

Step 1: Plot the K-map

ab \ c		ab			
		00	01	11	10
c	0		1	1	
	1		1	1	

$$F(a,b,c) = \sum m(2,4,5,7)$$

Three-Variable K-Map Example

Step 2: Circle Prime Implicants

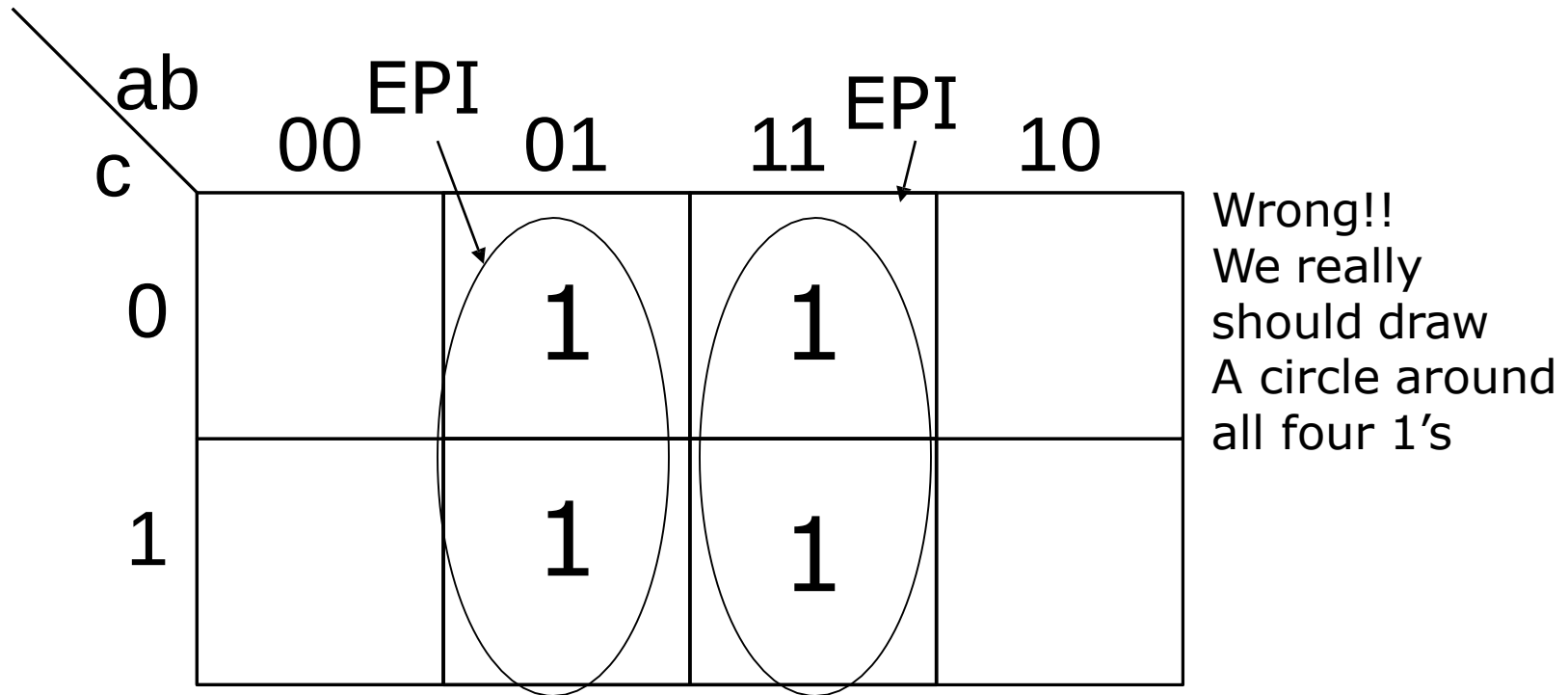
ab \ c		ab			
		00	01	11	10
c	0		1	1	
	1		1	1	

Wrong!!
We really
should draw
A circle around
all four 1's

$$F(a,b,c) = \sum m(2,3,6,7)$$

Three-Variable K-Map Example

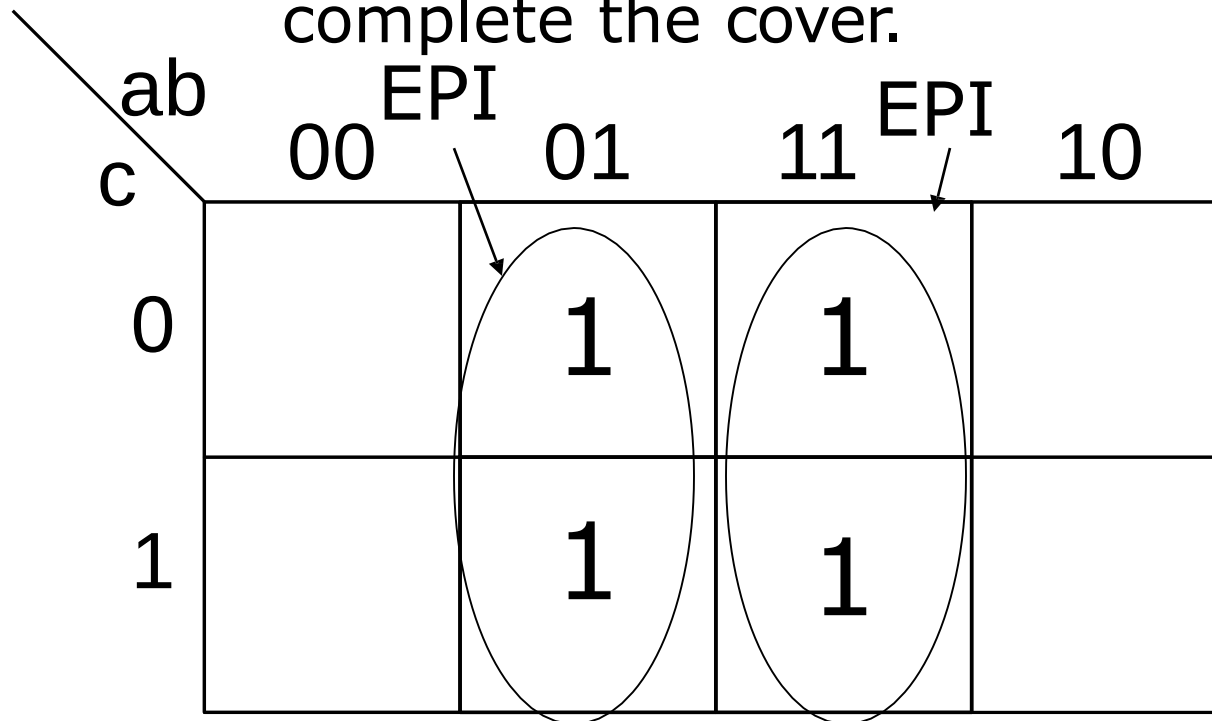
Step 3: Identify Essential Prime Implicants



$$F(a,b,c) = \sum m(2,3,6,7)$$

Three-Variable K-Map Example

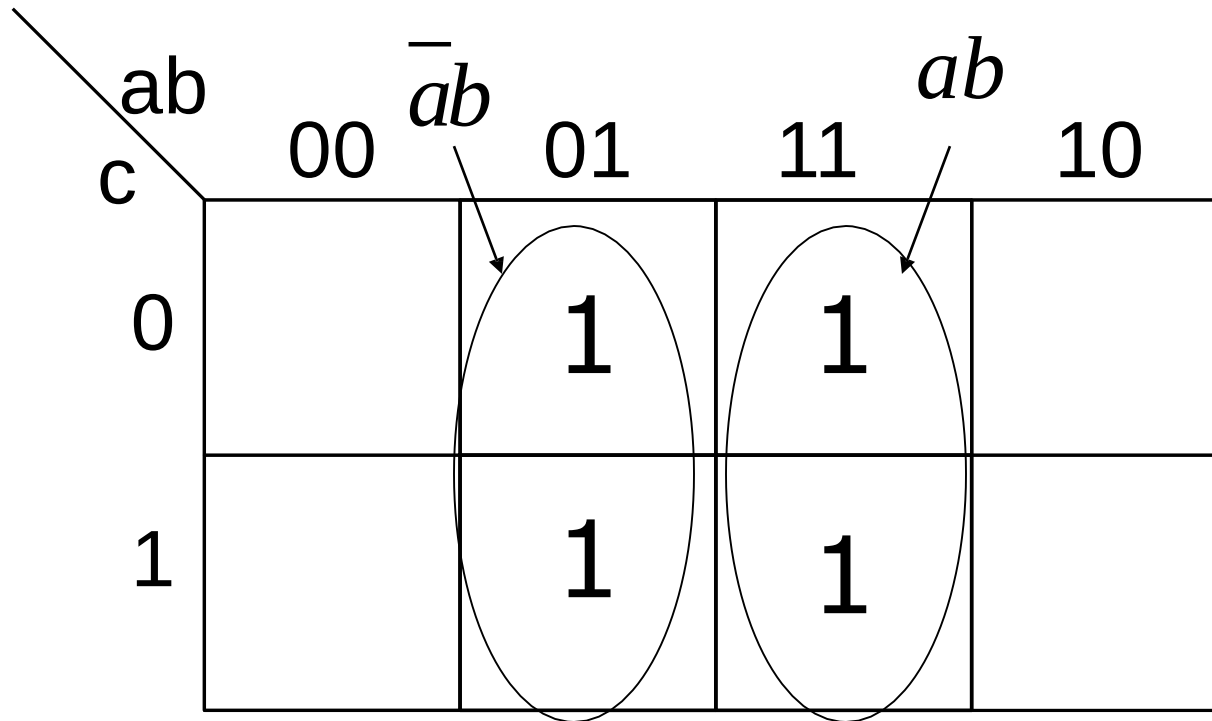
Step 4: Select Remaining Prime Implicants to complete the cover.



$$F(a,b,c) = \sum m(2,3,6,7)$$

Three-Variable K-Map Example

Step 5: Read the map.

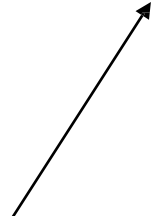


$$F(a,b,c) = \sum m(2,3,6,7)$$

Solution

$$F(a, b, c) = \bar{a}b + ab = b$$

Since we can still simplify the function
this means we did not use the largest
possible groupings.



$$F(a, b, c) = b$$

	ab	00	01	11	10
c	0		1	1	
	1		1	1	

$$F(a, b, c) = \sum m(2, 3, 6, 7)$$

Special Cases

Three-Variable K-Map Example

		ab			
		00	01	11	10
c	0	1	1	1	1
	1	1	1	1	1

$$F(a,b,c)=1$$

Three-Variable K-Map Example

		ab			
		00	01	11	10
c	0	0	0	0	0
	1	0	0	0	0

$$F(a, b, c) = 0$$

Three-Variable K-Map Example

		ab			
		00	01	11	10
c	0		1		1
	1	1		1	

$$F(a,b,c) = a \oplus b \oplus c$$

Example

- Use a K-Map to simplify the following Boolean expression

$$F(a, b, c) = \sum m(1, 2, 3, 5, 6)$$

Three-Variable K-Map Example

Step 1: Plot the K-map

ab \ c		ab			
		00	01	11	10
c	0		1	1	
	1	1	1		1

$$F(a,b,c) = \sum m(1,2,3,5,6)$$

Three-Variable K-Map Example

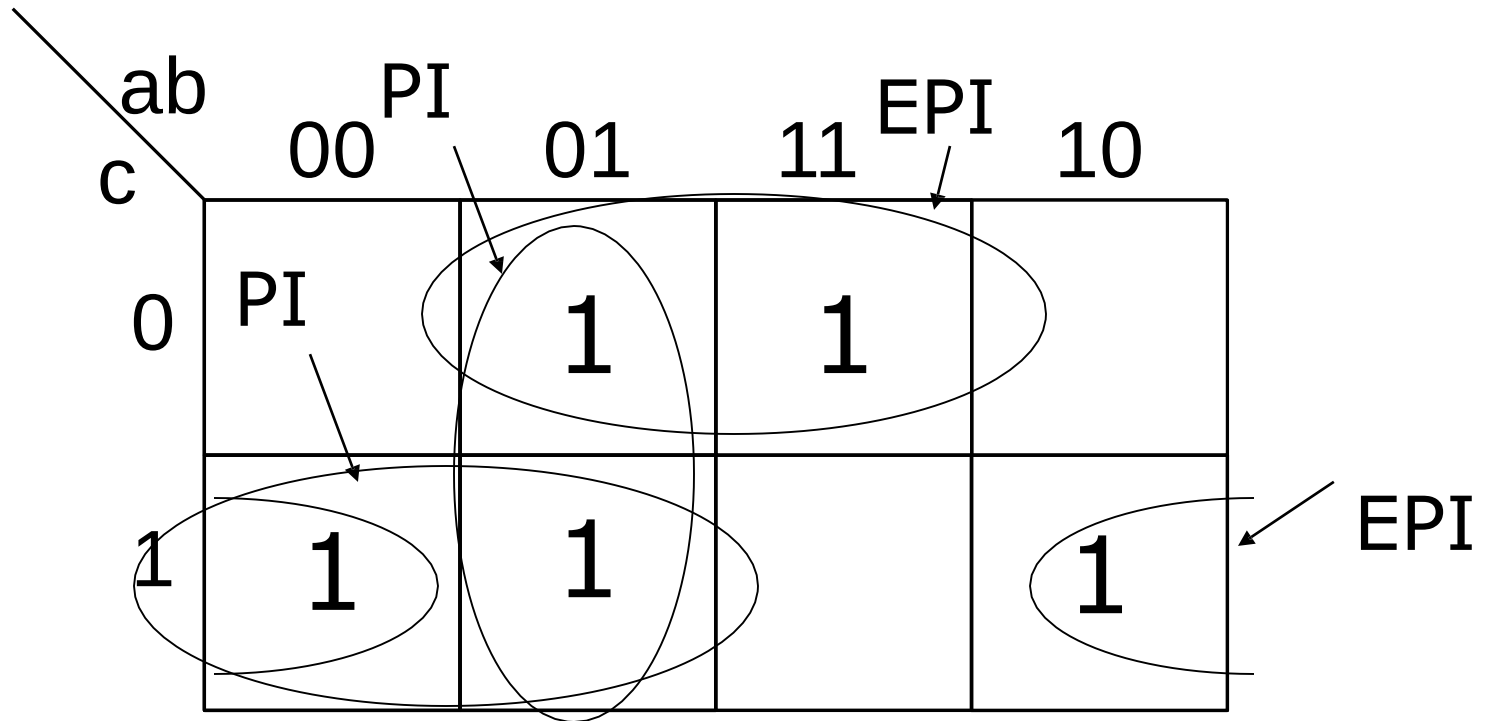
Step 2: Circle **ALL** Prime Implicants

ab		00	01	11	10
c					
0			1	1	
1		1	1		1

$$F(a,b,c) = \sum m(1,2,3,5,6)$$

Three-Variable K-Map Example

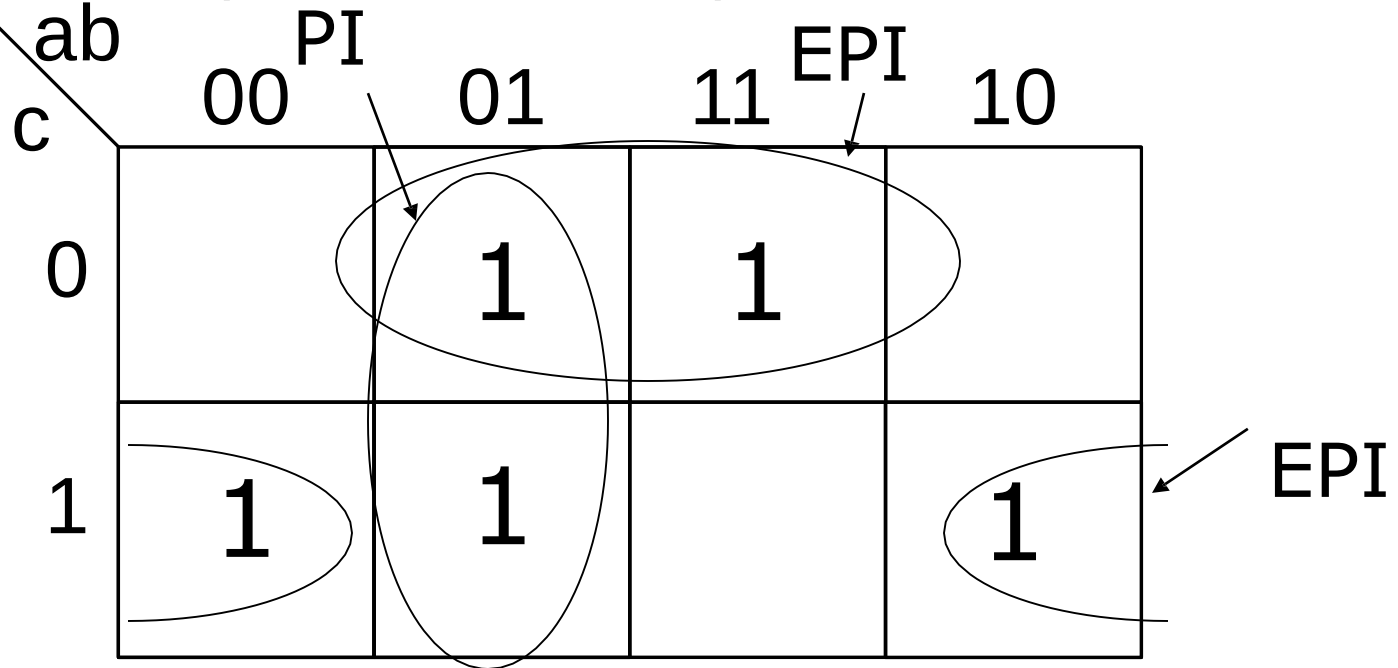
Step 3: Identify Essential Prime Implicants



$$F(a,b,c) = \sum m(1,2,3,5,6)$$

Three-Variable K-Map Example

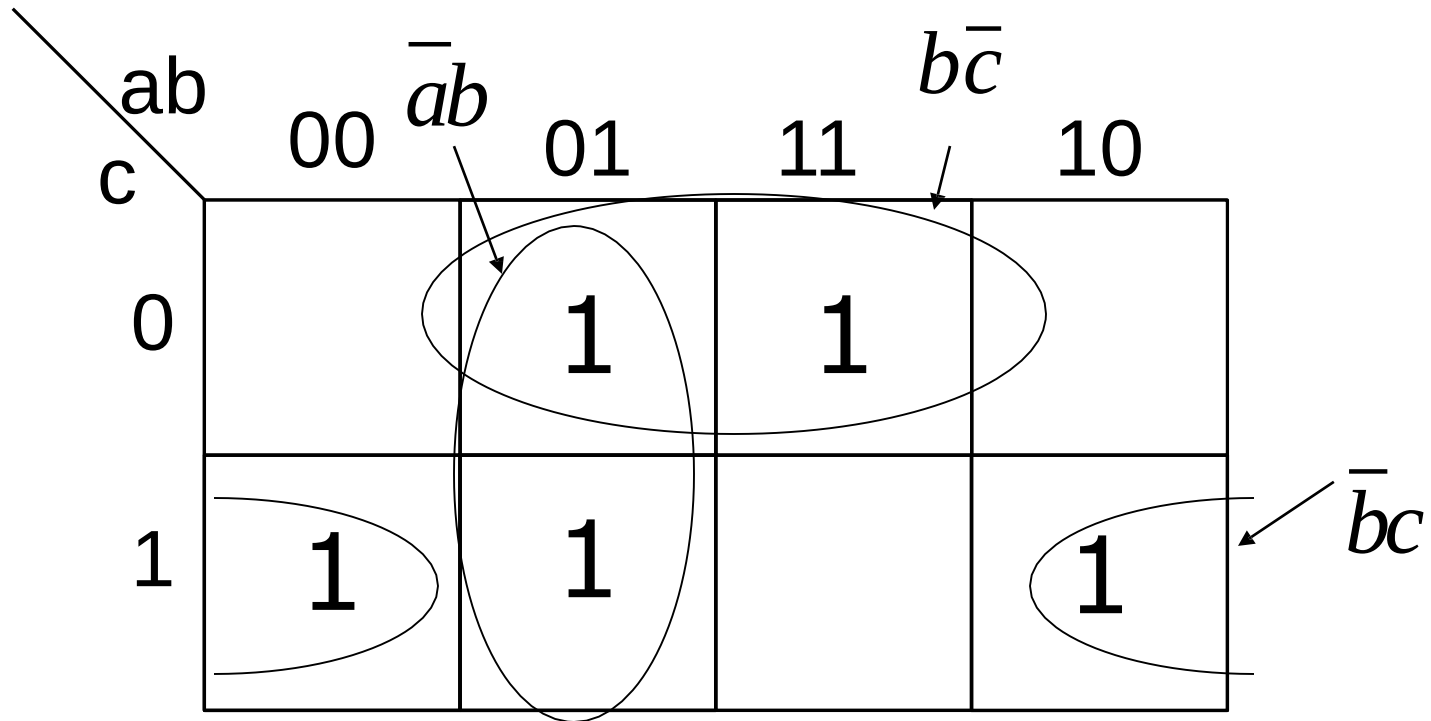
Step 4: Select minimum subset of remaining Prime Implicants to complete the cover.



$$F(a,b,c) = \sum m(1,2,3,5,6)$$

Three-Variable K-Map Example

Step 5: Read the map.



$$F(a, b, c) = \sum m(1, 2, 3, 5, 6)$$

Solution

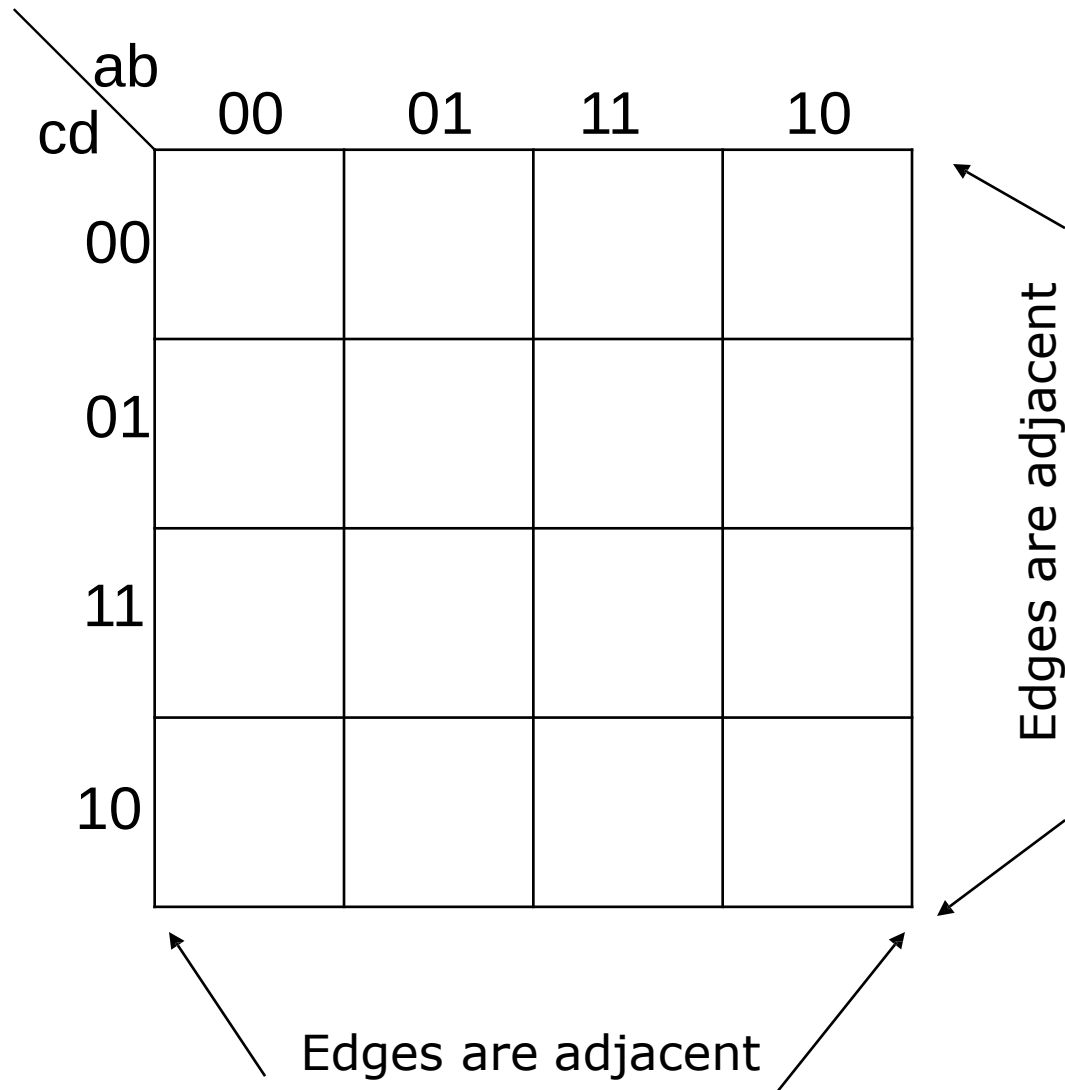
$$F(a, b, c) = \bar{a}b + b\bar{c} + \bar{b}c = \bar{a}b + b \oplus c$$

Four Variable Examples

Four-variable K-Map

		ab			
		00	01	11	10
cd	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

Four-variable K-Map



Four-Variable K-Maps

		CD			
		00	01	11	10
AB	00	1	0	0	0
	01	0	0	0	0
	11	0	0	0	0
	10	1	0	0	0

$$f = \sum(0,8) = \bar{B} \cdot \bar{C} \cdot \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	0	1	0	0
	11	0	1	0	0
	10	0	0	0	0

$$f = \sum(5,13) = B \cdot \bar{C} \cdot D$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	0	0	0	0
	11	0	1	1	0
	10	0	0	0	0

$$f = \sum(13,15) = A \cdot B \cdot D$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	1	0	0	1
	11	0	0	0	0
	10	0	0	0	0

$$f = \sum(4,6) = \bar{A} \cdot B \cdot \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	1	1
	01	0	0	1	1
	11	0	0	0	0
	10	0	0	0	0

$$f = \sum(2,3,6,7) = \bar{A} \cdot C$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	1	0	0	1
	11	1	0	0	1
	10	0	0	0	0

$$f = \sum(4,6,12,14) = B \cdot \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	1	1
	01	0	0	0	0
	11	0	0	0	0
	10	0	0	1	1

$$f = \sum(2,3,10,11) = \bar{B} \cdot C$$

		CD			
		00	01	11	10
AB	00	1	0	0	1
	01	0	0	0	0
	11	0	0	0	0
	10	1	0	0	1

$$f = \sum(0,2,8,10) = \bar{B} \cdot \bar{D}$$

Four-Variable K-Maps

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	1	1	1	1
	11	0	0	0	0
	10	0	0	0	0

$$f = \sum(4,5,6,7) = \bar{A} \bullet B$$

		CD			
		00	01	11	10
AB	00	0	0	1	0
	01	0	0	1	0
	11	0	0	1	0
	10	0	0	1	0

$$f = \sum(3,7,11,15) = C \bullet D$$

		CD			
		00	01	11	10
AB	00	1	0	1	0
	01	0	1	0	1
	11	1	0	1	0
	10	0	1	0	1

$$f = \sum(0,3,5,6,9,10,12,15)$$

$$f = A \otimes B \otimes C \otimes D$$

		CD			
		00	01	11	10
AB	00	0	1	0	1
	01	1	0	1	0
	11	0	1	0	1
	10	1	0	1	0

$$f = \sum(1,2,4,7,8,11,13,14)$$

$$f = A \oplus B \oplus C \oplus D$$

		CD			
		00	01	11	10
AB	00	0	1	1	0
	01	0	1	1	0
	11	0	1	1	0
	10	0	1	1	0

$$f = \sum(1,3,5,7,9,11,13,15)$$

$$f = D$$

		CD			
		00	01	11	10
AB	00	1	0	0	1
	01	1	0	0	1
	11	1	0	0	1
	10	1	0	0	1

$$f = \sum(0,2,4,6,8,10,12,14)$$

$$f = \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	1	1	1	1
	11	1	1	1	1
	10	0	0	0	0

$$f = \sum(4,5,6,7,12,13,14,15)$$

$$f = B$$

		CD			
		00	01	11	10
AB	00	1	1	1	1
	01	0	0	0	0
	11	0	0	0	0
	10	1	1	1	1

$$f = \sum(0,1,2,3,8,9,10,11)$$

$$f = \bar{B}$$

Example

- Use a K-Map to simplify the following Boolean expression

$$F(a, b, c, d) = \sum m(0, 2, 3, 6, 8, 12, 13, 15)$$

Four-variable K-Map

		ab			
		00	01	11	10
cd	00	1		1	1
	01			1	
	11	1		1	
	10	1	1		

$$F = \sum m(0,2,3,6,8,12,13,15)$$

Four-variable K-Map

		ab			
		00	01	11	10
cd	00	1		1	1
	01			1	
	11	1		1	
	10	1	1		

$$F = \overline{a}\overline{b}\overline{c}\overline{d} + \overline{a}\overline{b}c\overline{d} + \overline{a}c\overline{d}\overline{d} + abd + a\overline{c}\overline{d}$$

Four-Variable K-Maps Examples

		CD			
		00	01	11	10
AB	00	1	1		1
	01	1	1		1
	11	1	1		1
	10	1	1		

		CD			
		00	01	11	10
AB	00	1	1		1
	01				1
	11				
	10	1	1		1

		CD			
		00	01	11	10
AB	00				
	01	1	1	1	
	11	1	1		1
	10	1			

		CD			
		00	01	11	10
AB	00		1	1	
	01	1	1	1	1
	11	1		1	1
	10			1	

		CD			
		00	01	11	10
AB	00				
	01				
	11				
	10				

		CD			
		00	01	11	10
AB	00				
	01				
	11				
	10				

Five Variable K-Maps

$$F(a,b,c,d,e)$$

Five variable K-map

Use two four variable K-maps

_____ $A=1$

_____ $A=0$

Use Two Four-variable K-Maps

A=0 map

		bc			
		00	01	11	10
de	00				
	01				
	11				
	10				

A=1 map

		bc			
		00	01	11	10
de	00				
	01				
	11				
	10				

Five variable example

$$F(a, b, c, d, e) = \sum m(5, 7, 13, 15, 21, 23, 29, 31)$$

Use Two Four-variable K-Maps

A=0 map

de \ bc	bc			
	00	01	11	10
00				
01		1	1	
11		1	1	
10				

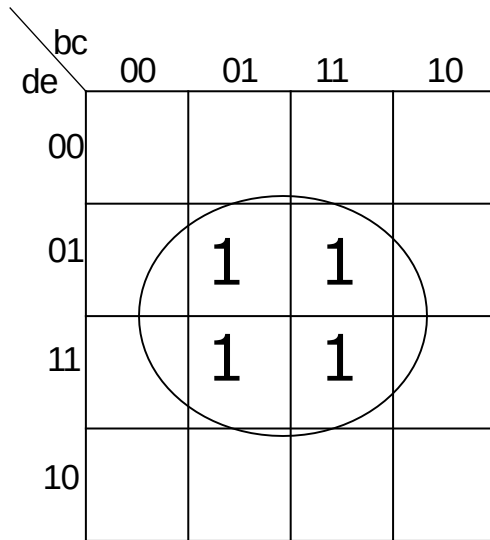
A=1 map

de \ bc	bc			
	00	01	11	10
00				
01		1	1	
11		1	1	
10				

$$F(a, b, c, d, e) = \sum m(5, 7, 13, 15, 21, 23, 29, 31)$$

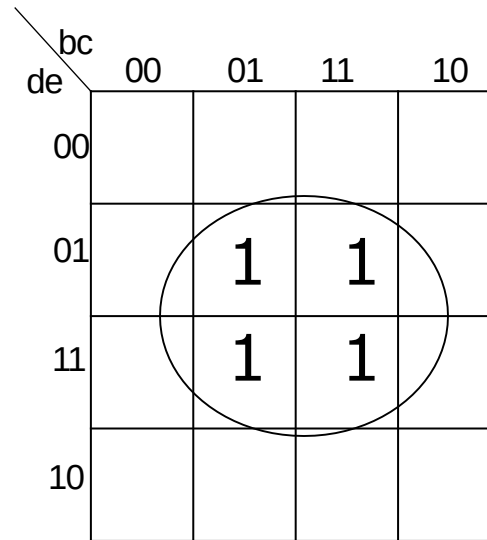
Use Two Four-variable K-Maps

A=0 map



$$F_1 = \bar{a}(\varnothing)$$

A=1 map

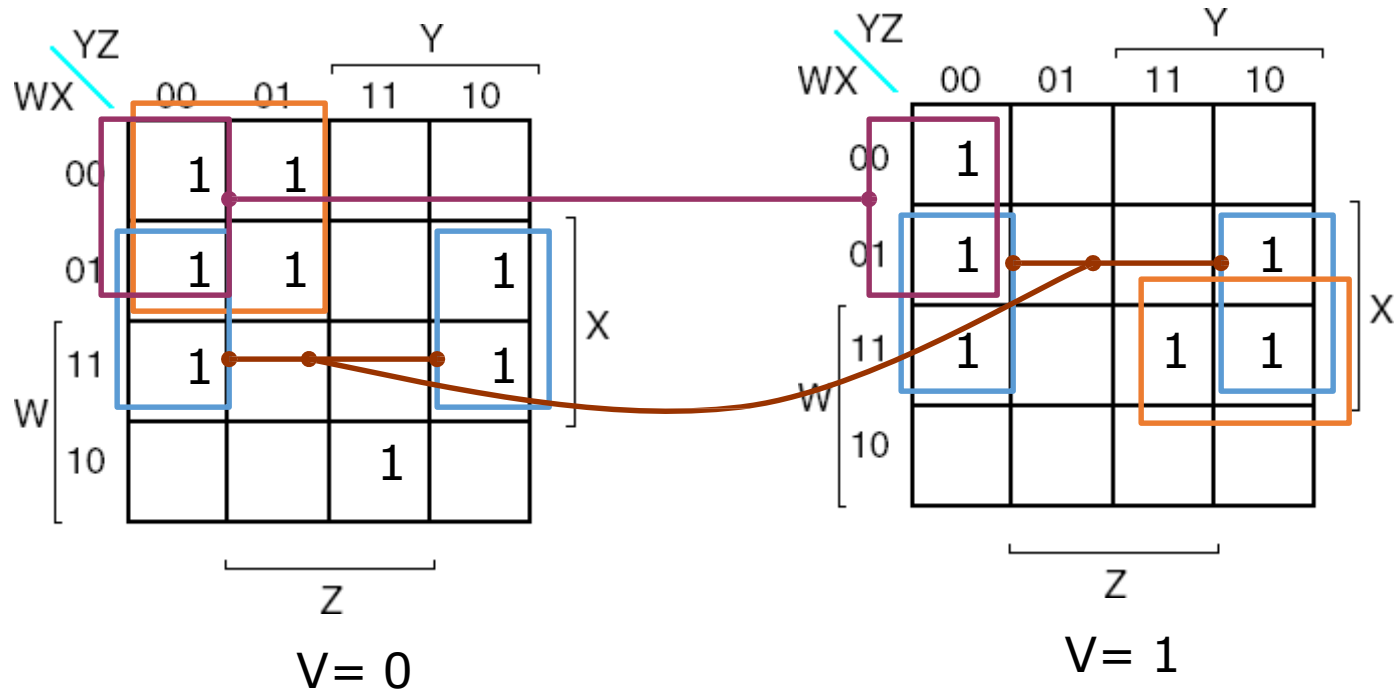


$$F_2 = a(\varnothing)$$

Five variable example

$$F = F_1 + F_2 = \bar{a}(ce) + a(ce) = ce$$

Simplify $f(V,W,X,Y,Z)=\Sigma m(0,1,4,5,6,11,12,14,16,20,22,28,30,31)$



$$f = XZ'$$

$$\Sigma m(4,6,12,14,20,22,28,30)$$

$$+ V'W'Y' \quad \Sigma m(0,1,4,5)$$

$$+ W'Y'Z' \quad \Sigma m(0,4,16,20)$$

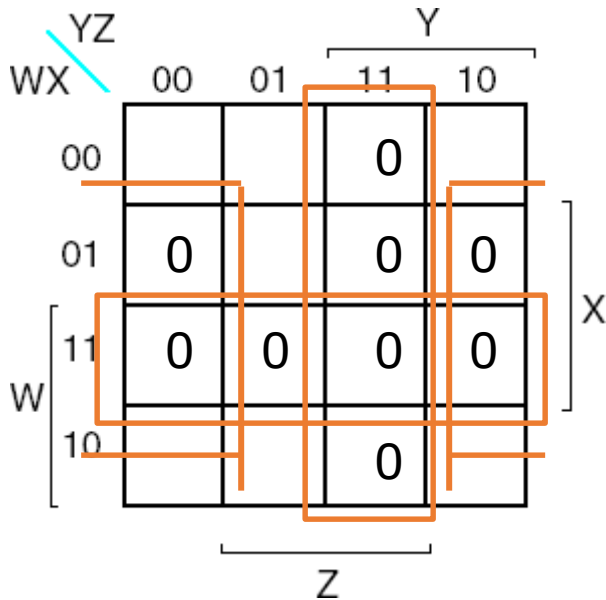
$$+ VWXY \quad \Sigma m(30,31)$$

$$+ V'WX'YZ \quad m11$$

PoS Optimization from SoP

$$F(W,X,Y,Z) = \sum m(0,1,2,5,8,9,10)$$

$$= \prod M(3,4,6,7,11,12,13,14,15)$$



$$F(W,X,Y,Z) = (W' + X')(Y' + Z')(X' + Z)$$

Or,

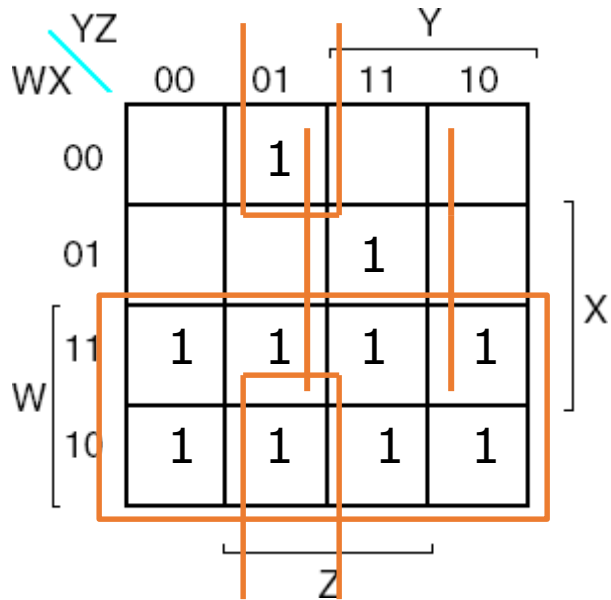
$$F(W,X,Y,Z) = X'Y' + X'Z' + W'Y'Z$$

Which one is the minimal one?

SoP Optimization from PoS

$$F(W,X,Y,Z) = \prod M(0,2,3,4,5,6)$$

$$= \sum m(1,7,8,9,10,11,12,13,14,15)$$



$$F(W,X,Y,Z) = W + XYZ + X'Y'Z$$

Plotting POS Functions

Don't-care Conditions

- In certain problems, some outputs are not specified.
- These outputs can be either '1' or '0'.
- They are called **don't-care conditions**, denoted by X (or sometimes, d).
- Example: An odd parity generator for BCD code which has 6 unused combinations.

- Don't-care conditions can be used to help simplify Boolean expression further in K-maps.
- They could be chosen to be either '1' or '0', depending on which gives the simpler expression.
- We usually use the notation Σd to denote the set of don't-care minterms. For example, the function P in the odd-parity generator for BCD can be written as:

$$P = \Sigma m(0, 3, 5, 6, 9) + \Sigma d(10, 11, 12, 13, 14, 15)$$

Don't-care Conditions

■ For comparison:

❖ WITHOUT don't-cares:

$$P = A'.B'.C'.D' + A'.B'.C.D + A'.B.C'.D \\ + A'.B.C.D' + A.B'.C'.D$$

❖ WITH don't-cares:

$$P = A'.B'.C'.D' + B'.C.D + B.C'.D \\ + B.C.D' + A.D$$

		C			
		D			
AB	CD	00	01	11	10
A	00	1		1	
	01		1		1
	11				
	10		1		

		C			
		D			
AB	CD	00	01	11	10
A	00	1		1	
	01		1		1
	11	X	X	X	X
	10		1	X	X

Four-variable K-Map

		ab			
		00	01	11	10
cd	00	1	d	1	1
	01			1	d
	11	d		1	
	10	1	1		

$$F(a, b, c, d) = \sum m(0, 2, 6, 8, 12, 13, 15) + d(3, 4, 9)$$

Four-variable K-Map

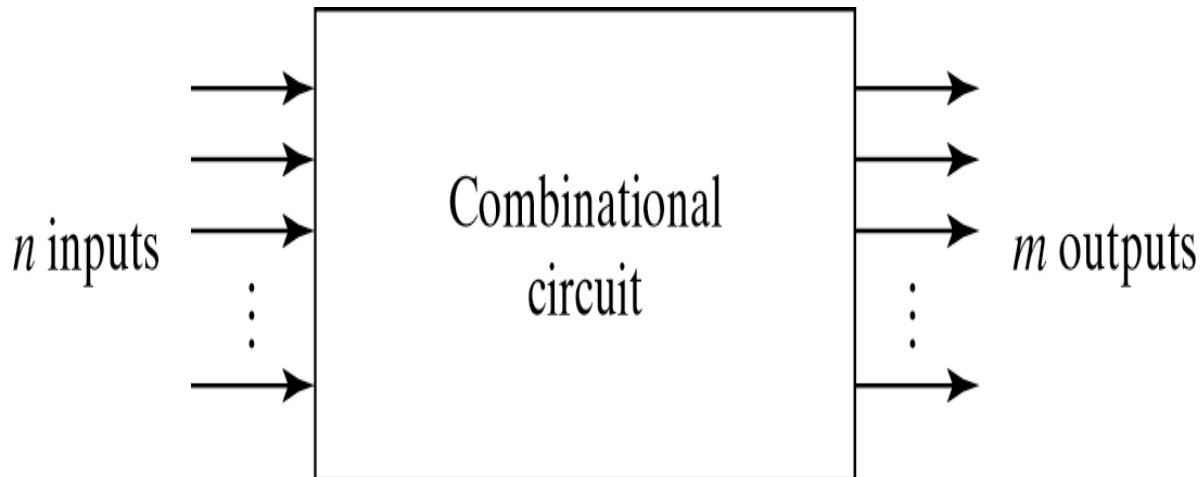
		ab			
		00	01	11	10
cd	00	1	d	1	1
	01			1	d
	11	d		1	
	10	1	1		

$$F = a\bar{c} + \bar{a}\bar{d} + abd$$

COMBINATIONAL CIRCUITS

A logic circuit whose output depends directly on only inputs is known as combinational circuit.

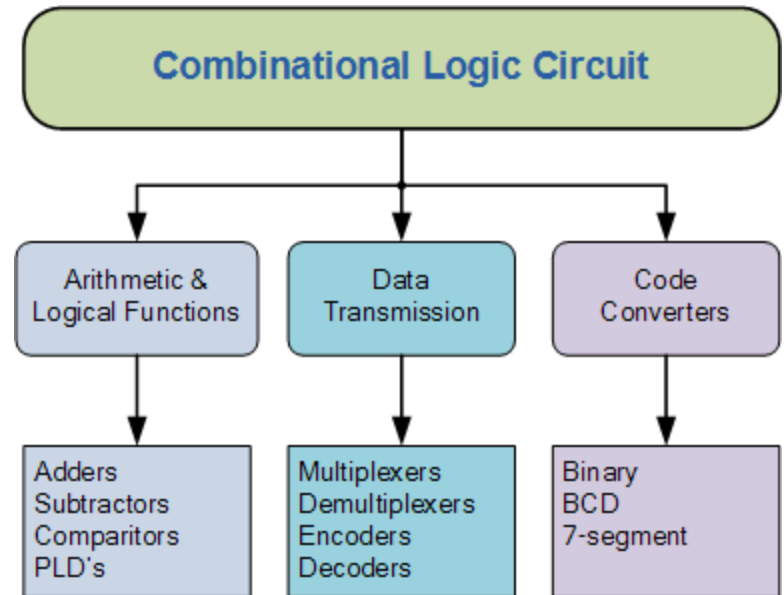
Combinational Logic Circuits are made up from basic logic NAND, NOR or NOT gates that are “combined” or connected together to produce more complicated switching circuits.



Classification of Combinational Logic

The function of a combinational logic circuit are:

1. Boolean Algebra – This forms the algebraic expression showing the operation of the logic circuit for each input variable
2. Truth Table – A truth table defines the function of a logic gate by providing a concise list that shows all the output states in tabular form for each possible combination of input variable.
3. Logic Diagram – This is a graphical representation of a logic circuit that shows the wiring and connections of each individual logic gate, represented by a specific graphical symbol, that implements the logic circuit.



Arithmetic Circuits

Arithmetic Circuits perform mathematical functions like Addition, Subtraction and Multiplication.

Problem Definition

- We will start out simply, adding two single digit binary numbers
- Let's make certain we can do this by hand before we construct the circuit
- There are four possible combinations of single digit, two number addition:
 - $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 10$

Notice that one of the combinations of two single digit additions is a double digit number, created by a carry:

$$1 + 1 = 10$$

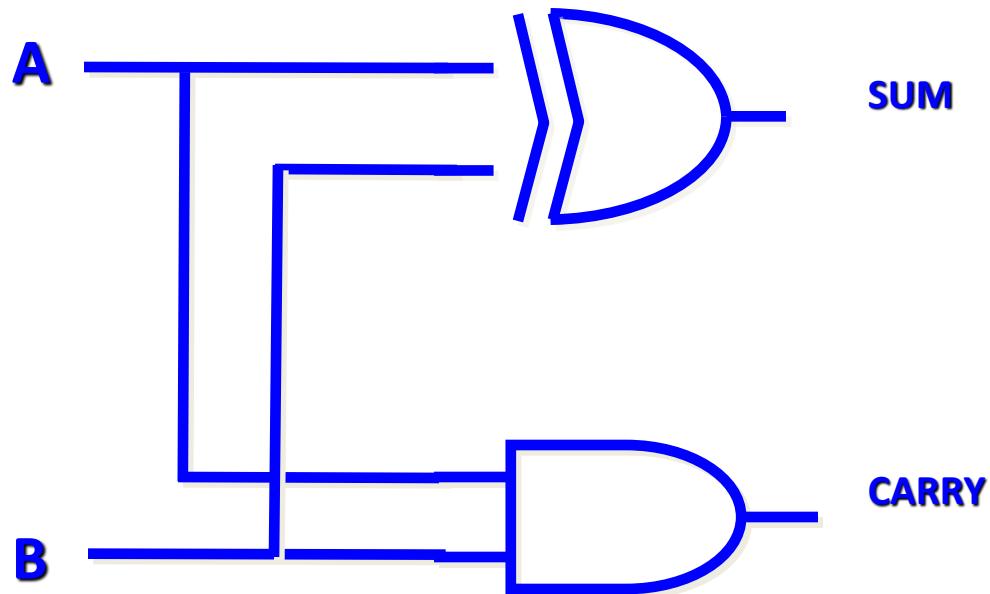
Truth Table for Adding Two 1-Bit Numbers

INPUTS		OUTPUTS	
A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

INPUTS		OUTPUTS	
A	B	Carry	Sum
		$A \cdot B$	$A \oplus B$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- The XOR circuit delivers the *Sum* of adding two single digits.
- The AND circuit produces the *Carry*.
- We've just created a fundamental circuit called a *Half Adder*, which provides the capability of adding two single bit numbers.

Circuit Diagram for the Half Adder

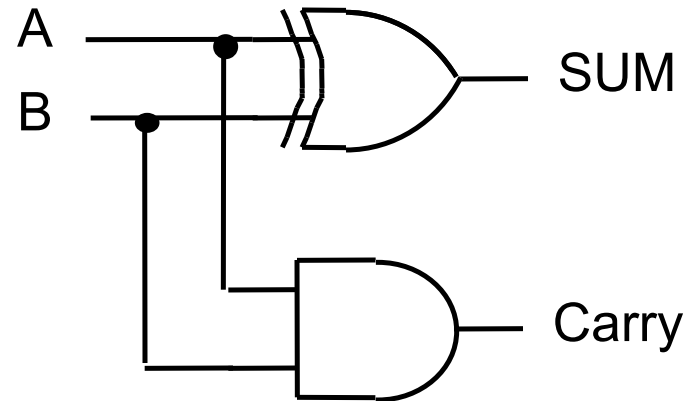


Half Adder

Add two binary numbers

- A, B -> single bit inputs
- Sum -> single bit sum
- Carry -> carry out

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Dec	Binary
1	1
+1	+1
2	10

Full Adder

Full Adder is a combinational circuit that performs the addition of three bits (two significant bits and previous carry).

The full adder is usually a component in a cascade of adders, which add 8, 16, etc, binary numbers.

C_i	A_i	B_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$C_i \backslash A_i B_i$				
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$S_i = A_i \text{ XOR } B_i \text{ XOR } C_i$$

Full Adder

Now consider implementation of carry out

Minimize circuit for carry out - C_{i+1}

C_i	A_i	B_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

		$A_i B_i$			
		00	01	11	10
C_i	0			1	
	1		1	1	1

C_{i+1}

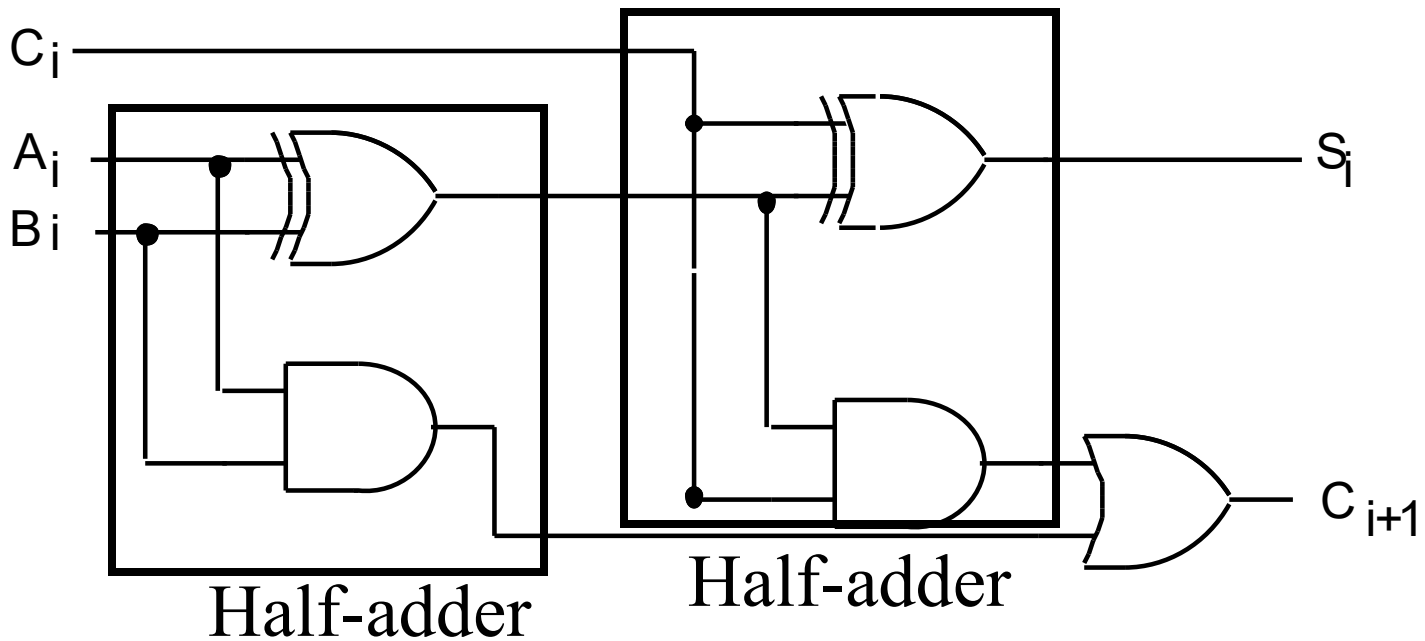
$$C_{i+1} = A_i B_i + C_i B_i + C_i A_i$$

Full Adder

Full adder made of several half adders

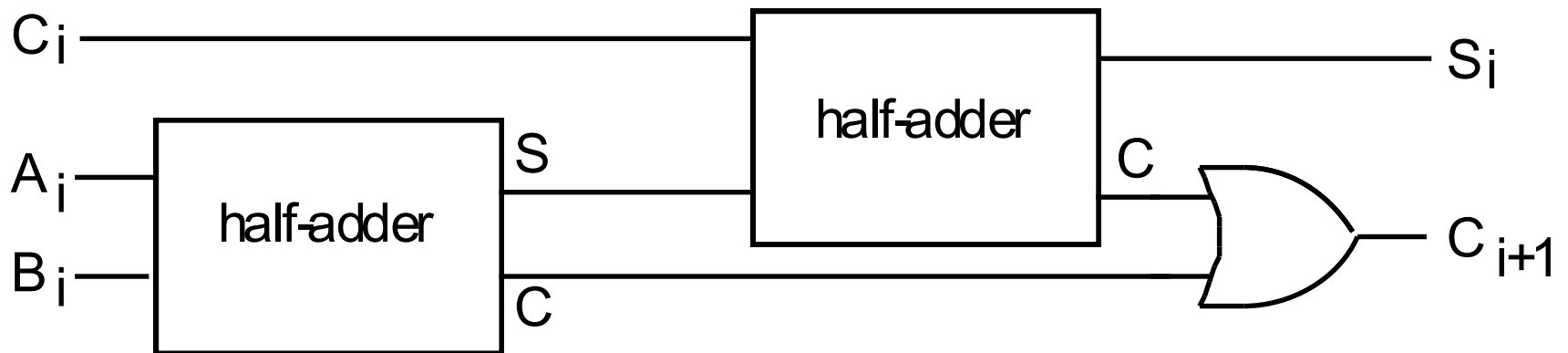
$$S_i = A_i \text{ XOR } B_i \text{ XOR } C_i$$

$$C_{i+1} = A_i B_i + C_i B_i + C_i A_i$$



Full Adder

Hardware repetition simplifies hardware design



A full adder can be made from two half adders (plus an OR gate).

Full adder equations

- Using Boolean algebra, we can simplify S and C_{out} as shown here.

X	Y	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\begin{aligned}S &= \Sigma m(1,2,4,7) \\&= X'Y'C_{in} + X'YC_{in}' + XY'C_{in}' + XYC_{in} \\&= X'(Y'C_{in} + YC_{in}') + X(Y'C_{in}' + YC_{in}) \\&= X'(Y \oplus C_{in}) + X(Y \oplus C_{in})' \\&= X \oplus Y \oplus C_{in}\end{aligned}$$

$$\begin{aligned}C_{out} &= \Sigma m(3,5,6,7) \\&= X'YC_{in} + XY'C_{in} + XYC_{in}' + XYC_{in} \\&= (X'Y + XY')C_{in} + XY(C_{in}' + C_{in}) \\&= (X \oplus Y)C_{in} + XY\end{aligned}$$

- Notice that XOR operations simplify things a bit, but we had to resort to using algebra since it's hard to find XOR-based expressions with K-maps.

The N -Bit Adder

- The next leap of faith is to move from adding 3 (or more) single digit numbers to adding multi-bit numbers
- First, note that the sum of adding two N -bit numbers can be $N + 1$ bits
- This comes as a result of possibly obtaining a carry into the next column
- Adding N -bit numbers isn't an abstract consideration – it's a **real** computing operation - at a minimum, a modern computer would add 16 bit numbers, done with a 16-bit adder

Multiple-bit Addition

Consider single-bit adder for each bit position.

$$\begin{array}{rcccc} & A_3 & A_2 & A_1 & A_0 \\ \mathbf{A} & 0 & 1 & 0 & 1 \end{array}$$

$$\begin{array}{rcccc} & B_3 & B_2 & B_1 & B_0 \\ \mathbf{B} & 0 & 1 & 1 & 1 \end{array}$$

$$\begin{array}{rcccc} & 1 & 1 & 1 & \\ \mathbf{A} & 0 & 1 & 0 & 1 \\ \mathbf{B} & 0 & 1 & 1 & 1 \\ \hline & 1 & 1 & 0 & 0 \end{array}$$

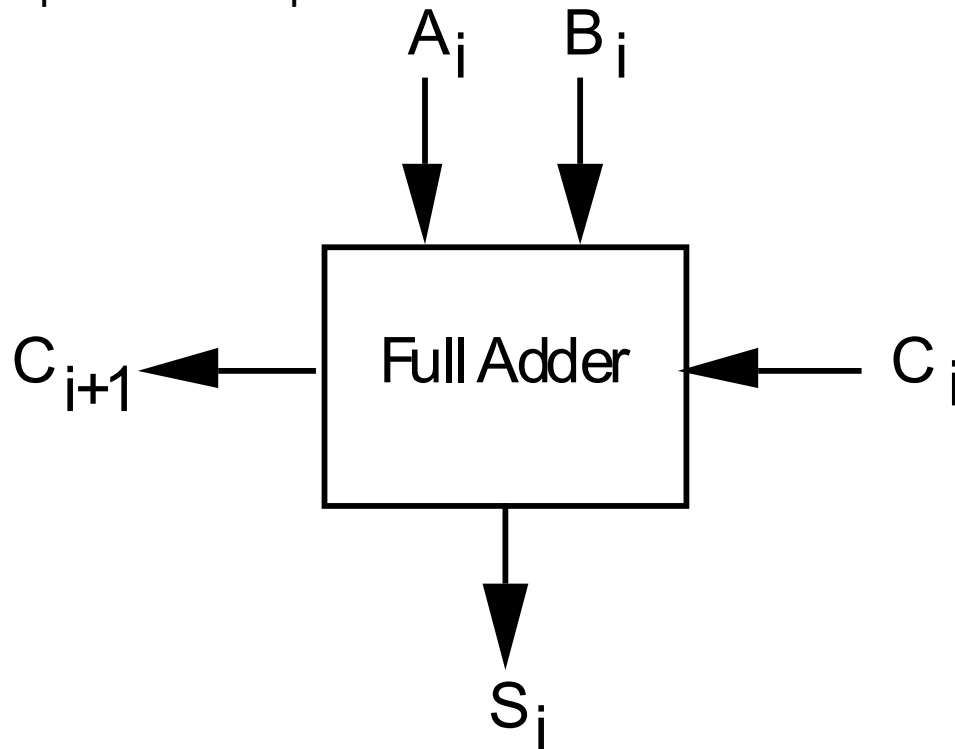
$$\begin{array}{r} C_{i+1} \leftarrow \\ C_i \\ A_i \\ + B_i \\ \hline S_i \end{array}$$

Each bit position creates a sum and carry

Full Adder

Putting it all together

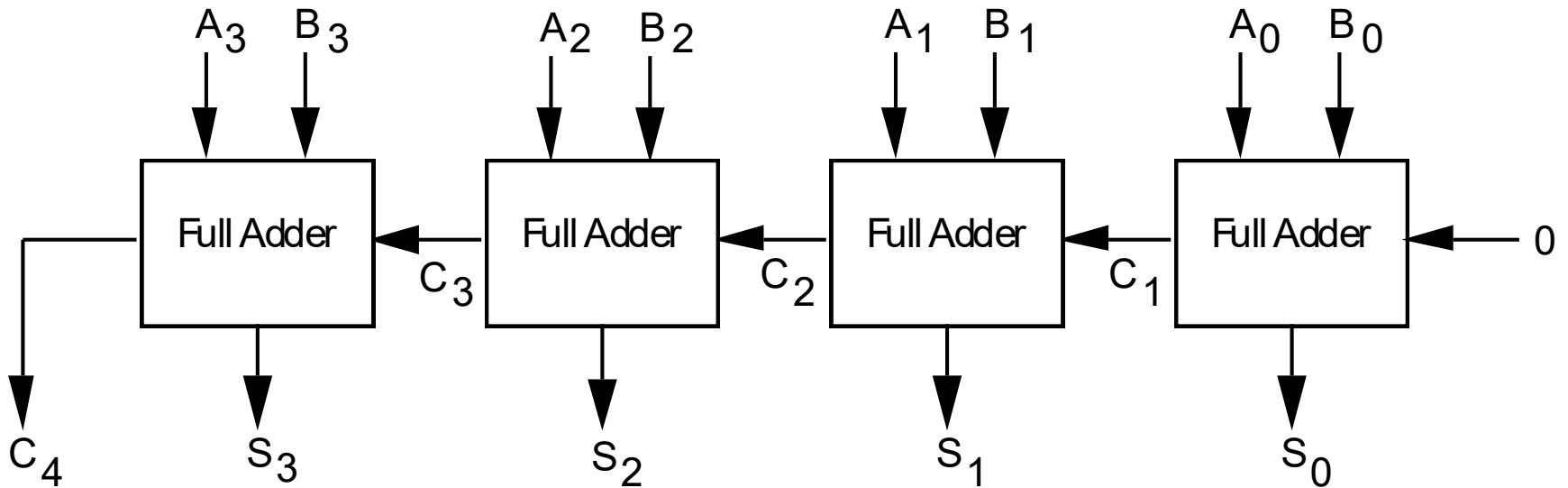
- Single-bit full adder
- Common piece of computer hardware



Block Diagram

4-Bit Adder

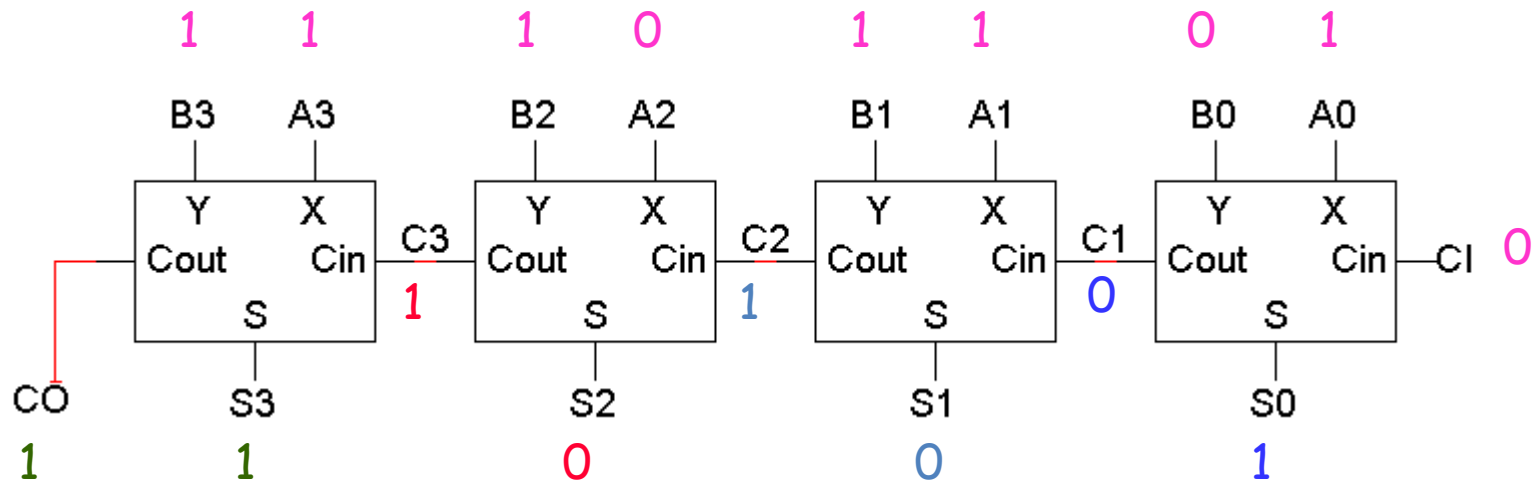
Chain single-bit adders together.
What does this do to delay?



C	1	1	1	0
A	0	1	0	1
B	0	1	1	1
S	1	1	0	0

An example of 4-bit addition

Let's try our initial example: $A=1011$ (eleven), $B=1110$ (fourteen).

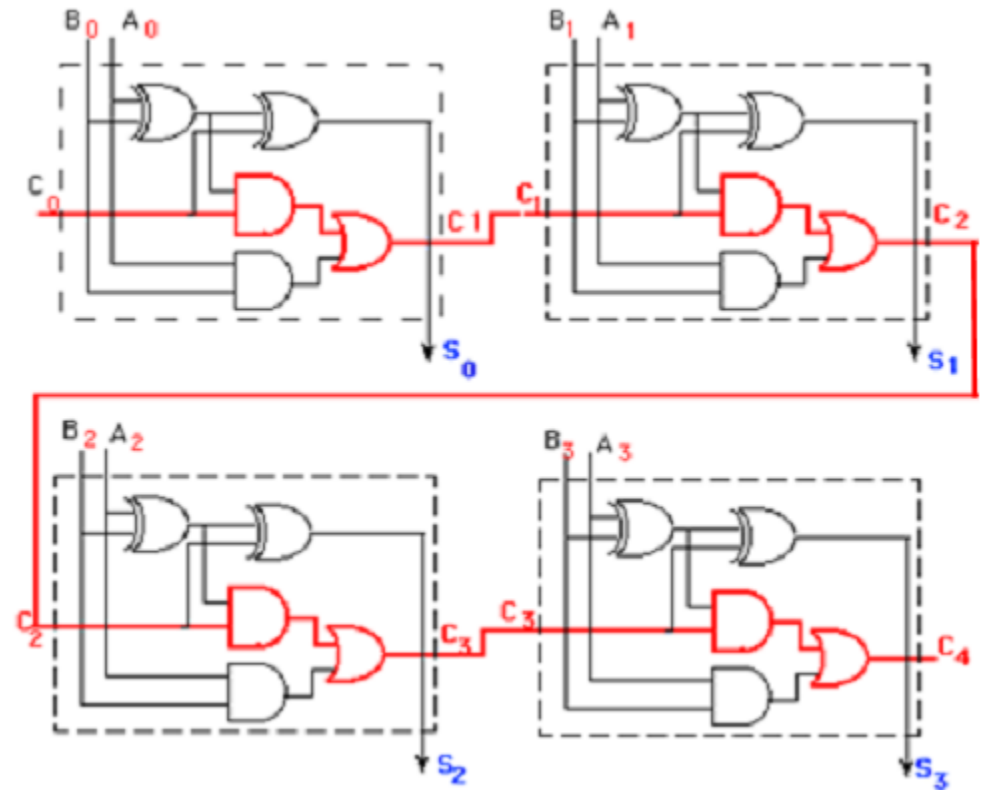


1. Fill in all the inputs, including $CI=0$
2. The circuit produces $C1$ and $S0$ ($1 + 0 + 0 = 01$)
3. Use $C1$ to find $C2$ and $S1$ ($1 + 1 + 0 = 10$)
4. Use $C2$ to compute $C3$ and $S2$ ($0 + 1 + 1 = 10$)
5. Use $C3$ to compute CO and $S3$ ($1 + 1 + 1 = 11$)

Ripple Carry Adder (RCA)

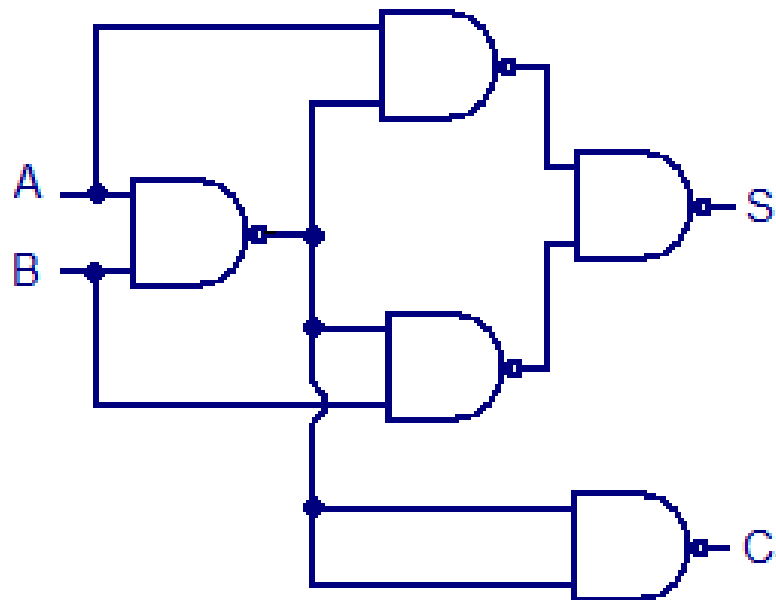
- Assume gate delay = T
- $8T$ to compute the last carry
- Total delay = $8 + 1 = 9T$
- 1 delay from first half adder
- Delay = $(2n+1)T$
- Where $n = \#$ of bit

How to improve?

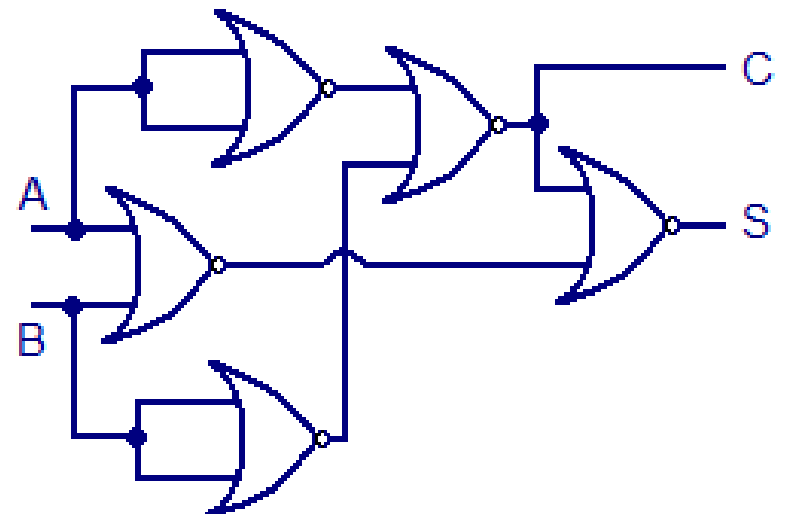


Carry Look Ahead Adder

- How to reduce propagation delay of ripple carry adders?
- **Carry look ahead adder:** All carries are computed as a function of C_0 (independent of n !)
- It works on the following standard principles:
 - A carry bit is generated when both input bits A_i and B_i are 1, or
 - When one of input bits is 1, and a carry in bit exists

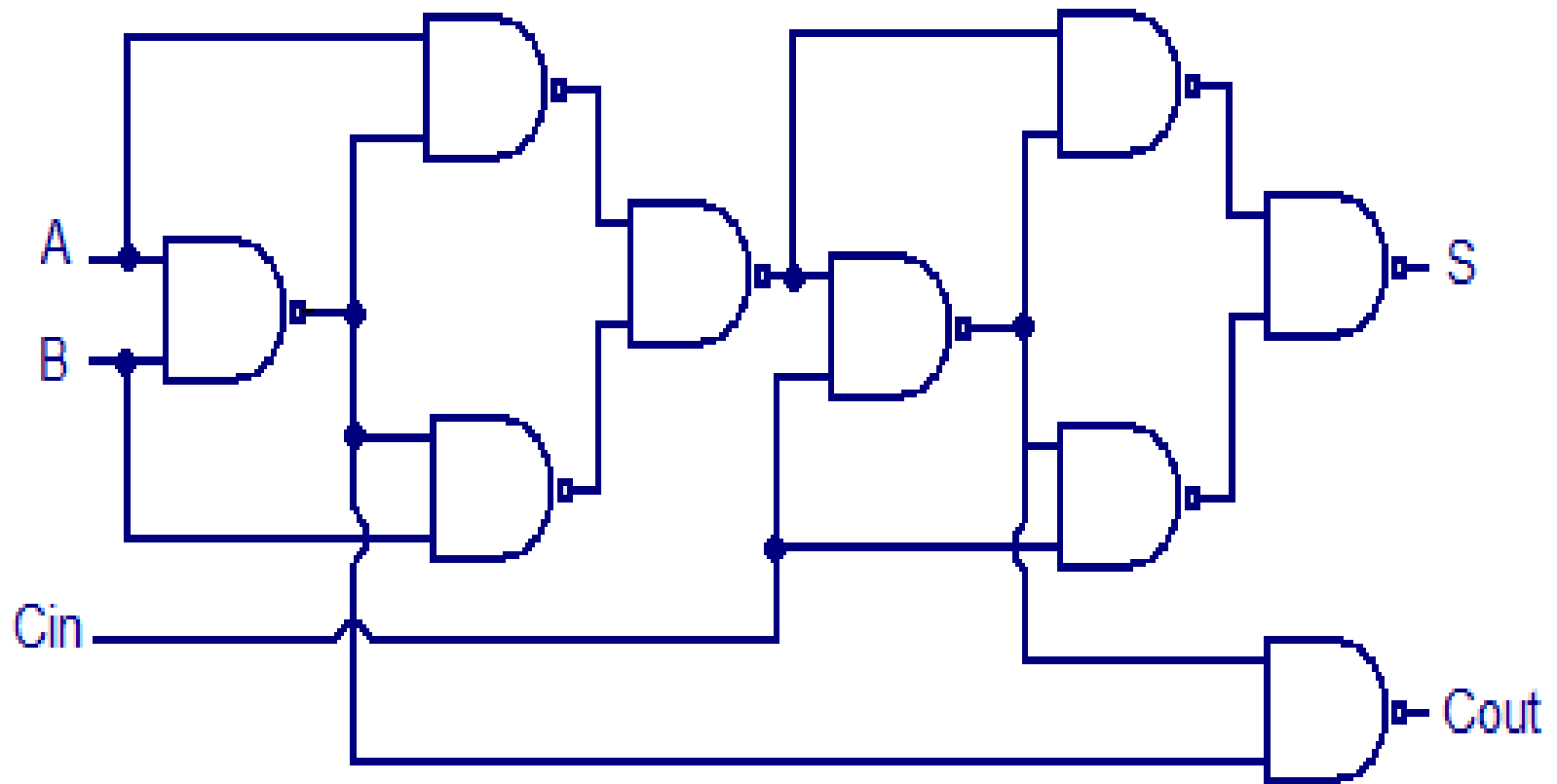


Half adder using NAND logic



Half adder using NOR logic

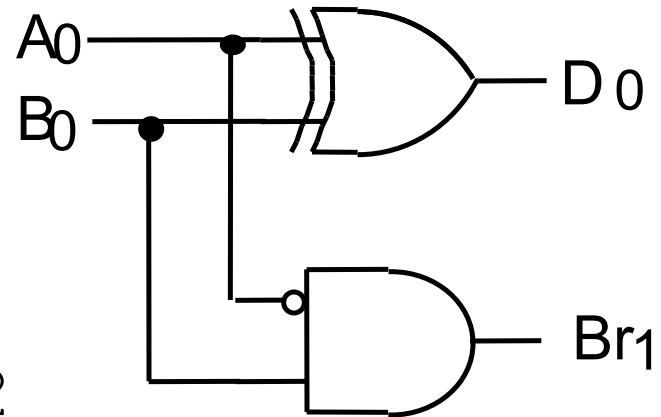
Full adder using NAND logic



Half Subtractor

A_0	B_0	D_0	Br_1
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$\begin{array}{r} 12 \\ 0 \\ -1 \\ \hline 1 \end{array}$$



Full Subtractor

A	B	C	D	BOR
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

		AB			
		00	01	11	10
C	0	0	1	0	1
	1	1	0	1	0

D

$$D = A \text{ XOR } B \text{ XOR } C$$

Same as Sum in full adder

Full Subtractor

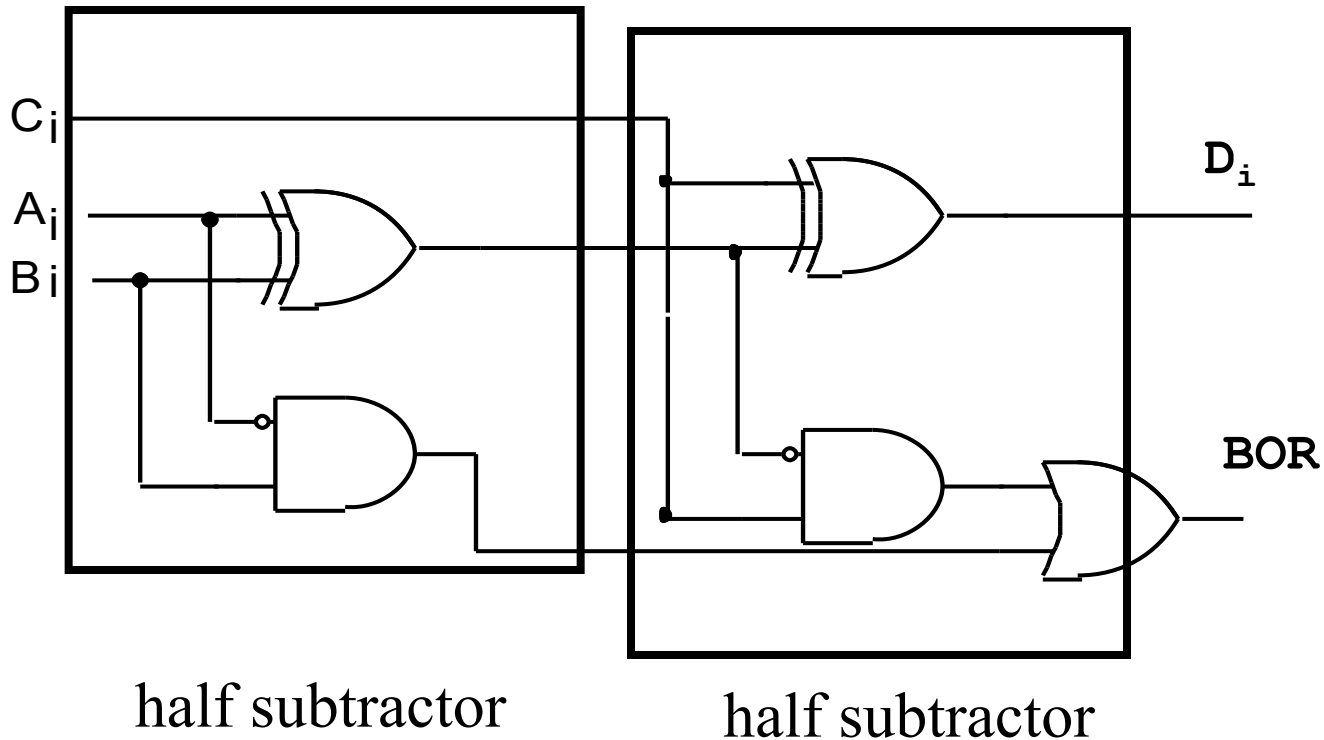
A	B	C	D	BOR
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$\text{BOR} = A' B + C B + C A'$$

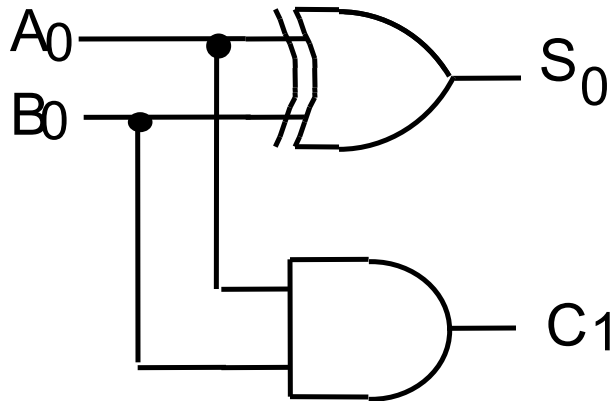
Full Subtractor

$$D_i = A_i \text{ XOR } B_i \text{ XOR } C_i$$

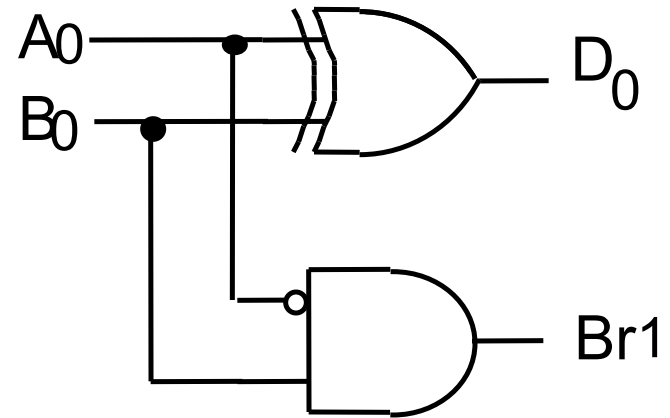
$$\text{BOR} = A_i' B_i + C_i B_i + C_i A_i'$$



Adder/Subtractor - 1



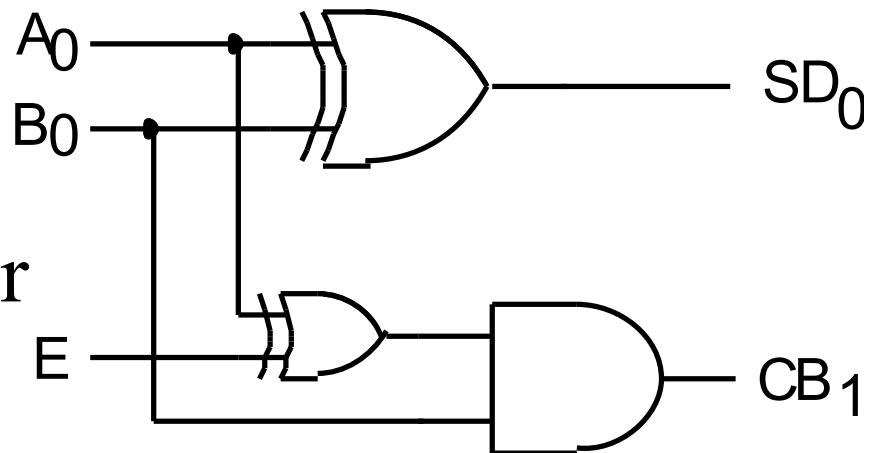
Half adder



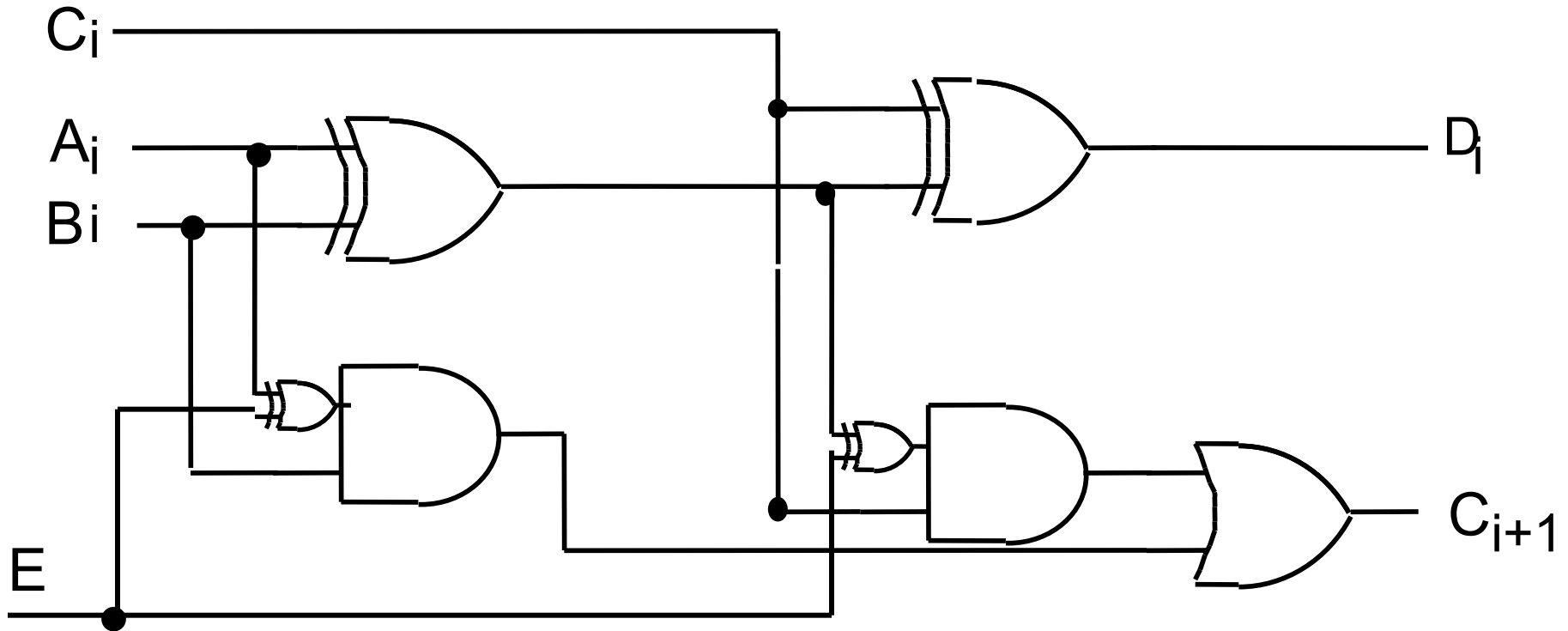
Half subtractor

$E = 0$: Half adder

$E = 1$: Half subtractor



Adder/Subtractor-1

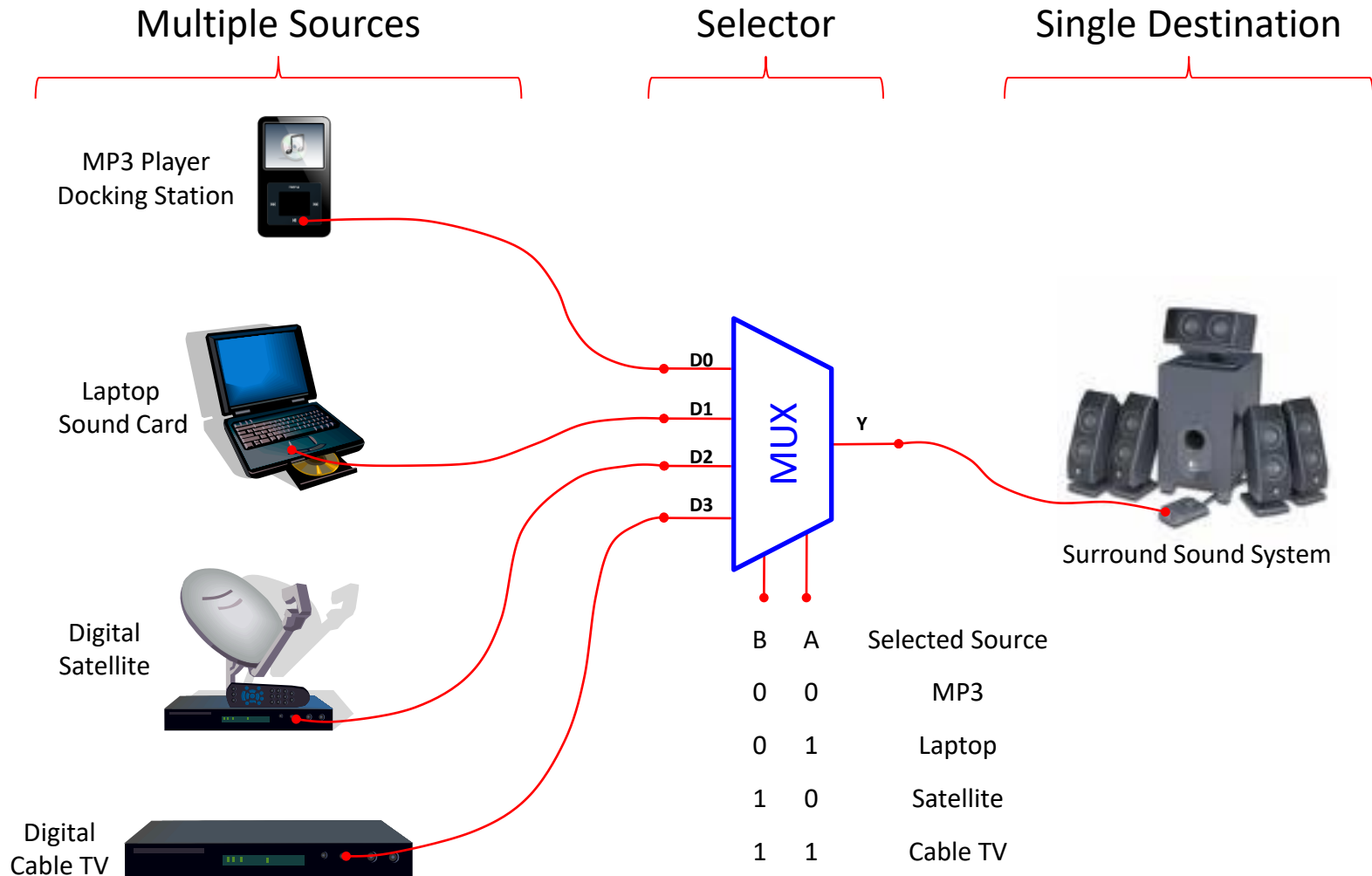


$E = 0$: Full adder

$E = 1$: Full subtractor

Multiplexer / Demultiplexer

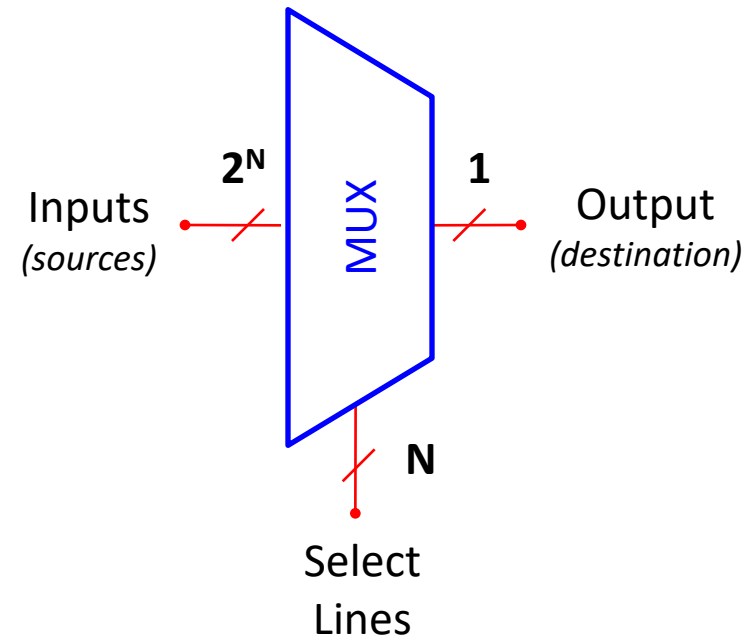
Typical Application of a MUX



What is a Multiplexer (MUX)?

- A MUX is a digital switch that has multiple inputs (sources) and a single output (destination). (Data selector)
- The select lines determine which input is connected to the output.
- MUX Types
 - 2-to-1 (1 select line)
 - 4-to-1 (2 select lines)
 - 8-to-1 (3 select lines)
 - 16-to-1 (4 select lines)

Multiplexer
Block Diagram

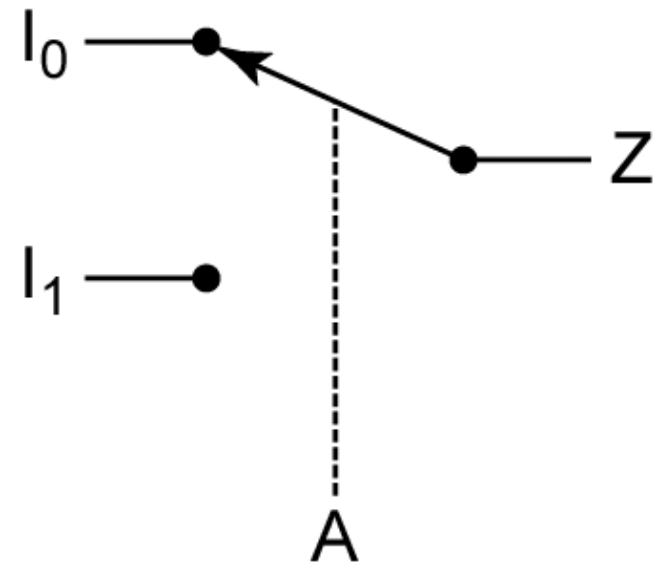
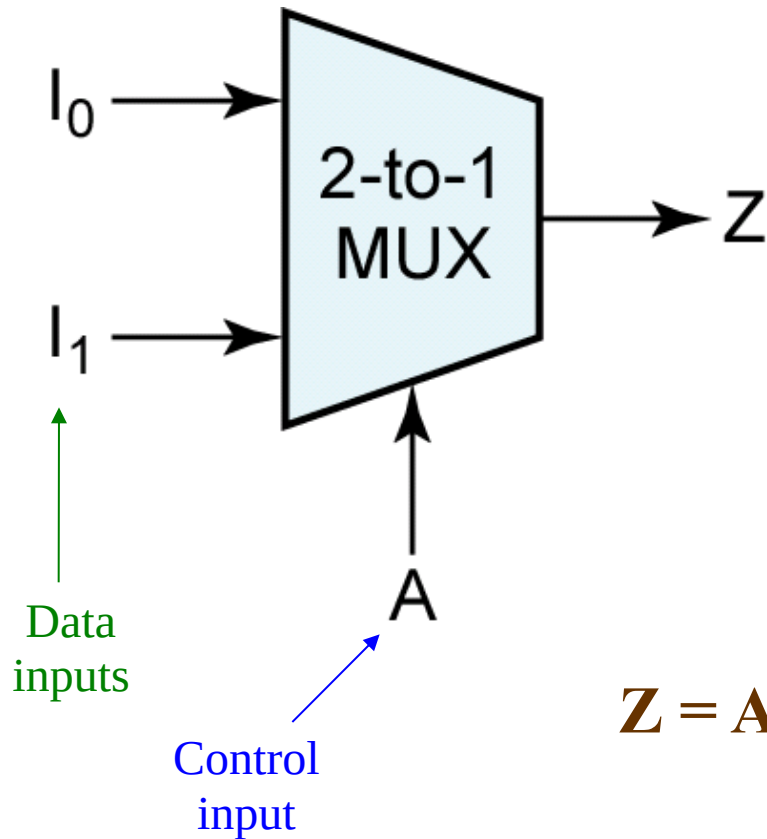


MULTIPLEXERS (Data Selectors)

A multiplexers (MUX) is a device that allows digital information from several sources to be routed onto a single line for transmission over that line to a common destination.

The basic multiplexers has several data input lines and a single output line. It also has data- select inputs, which permit digital data on any one of the inputs to be switched to the output line.

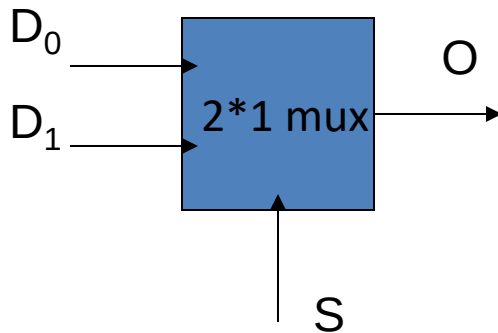
Multiplexers



$$Z = A'.I_0 + A.I_1$$

Design of a 2*1 Mux

- 2*1 mux Block Diagram
- Truth Table



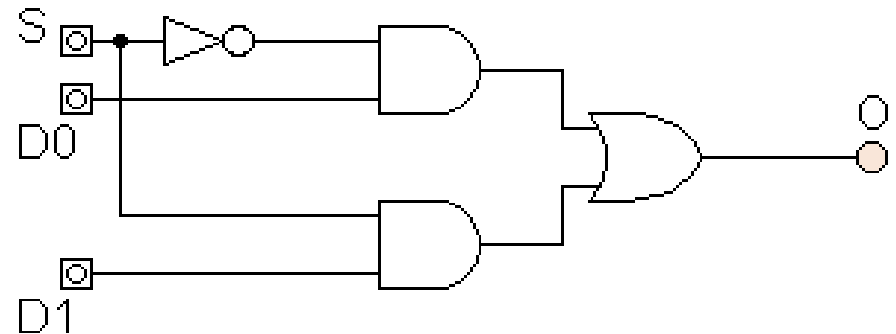
S	D ₁	D ₀	O
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Design of 2*1 Mux Continued

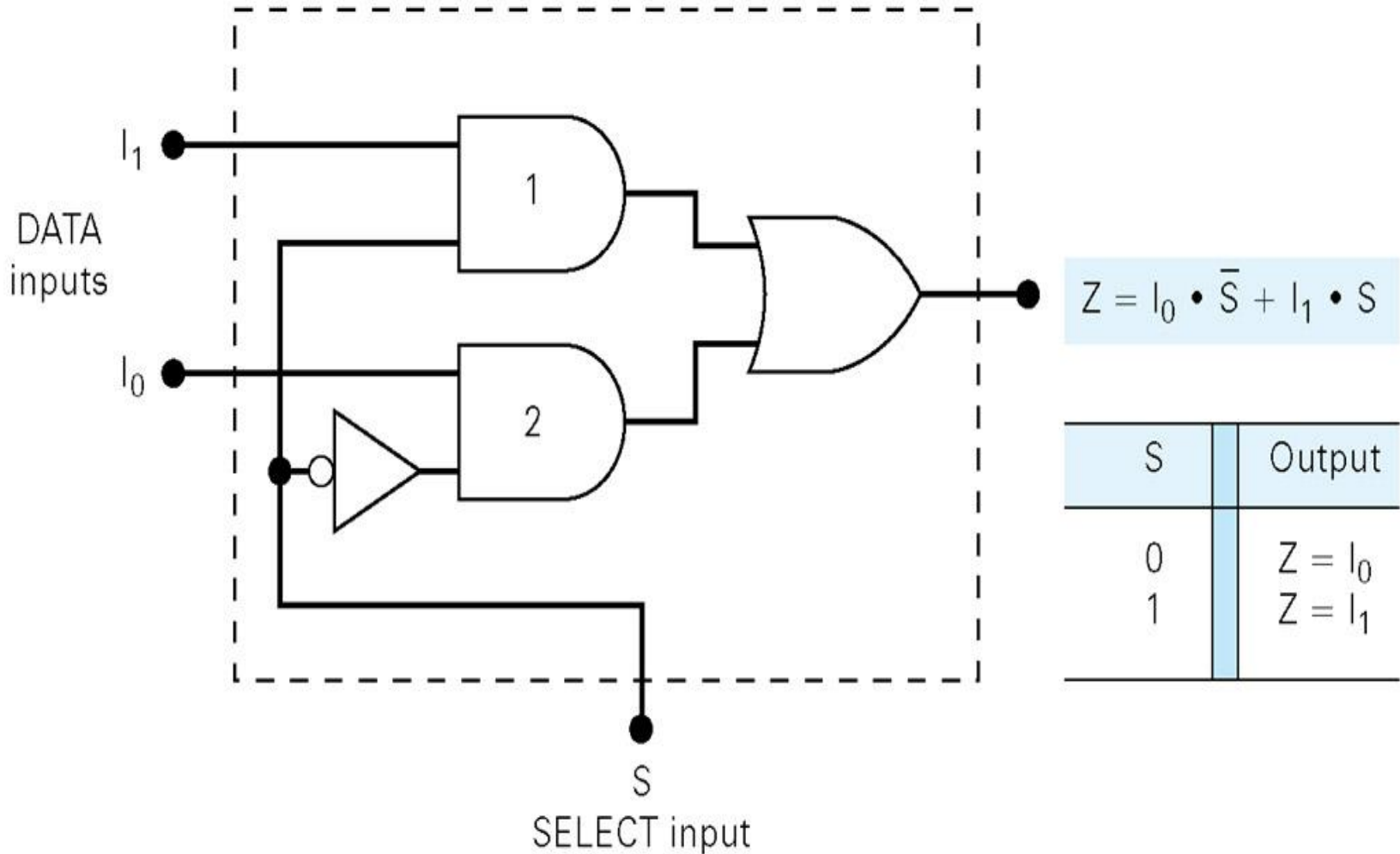
- K-Map

	$S'D_0$	01	11	10
D0				
0		1		
1		1	1	1

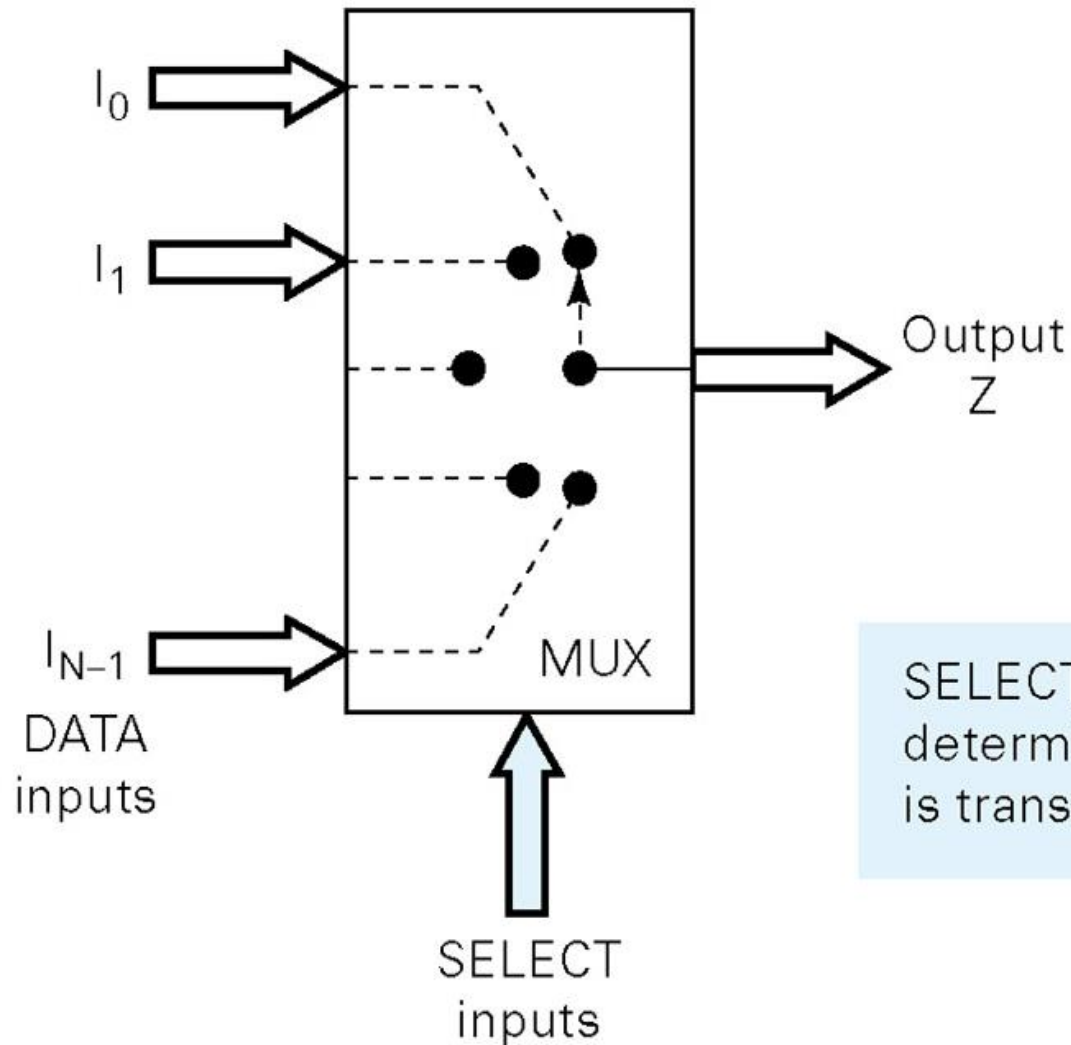
$$O = S'D_0 + SD_1$$



Two-input multiplexer

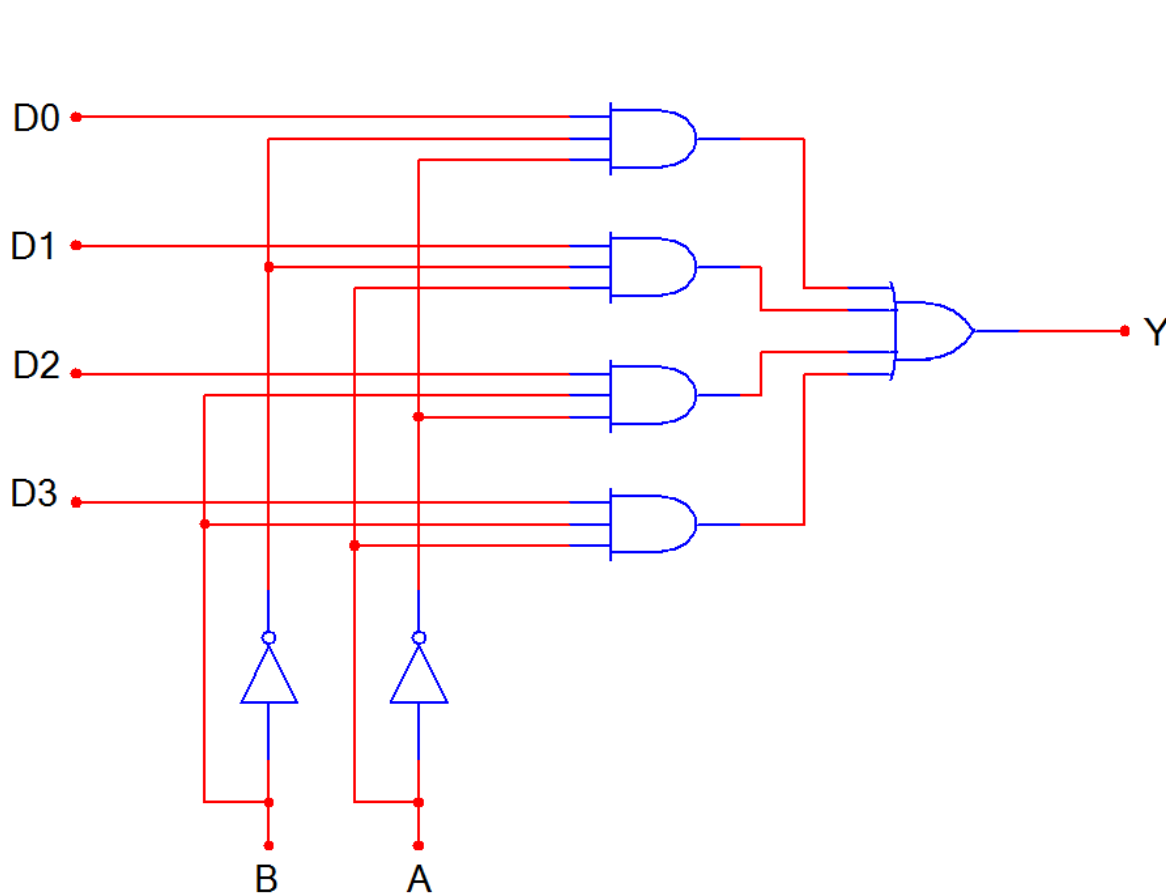


Functional diagram of MUX

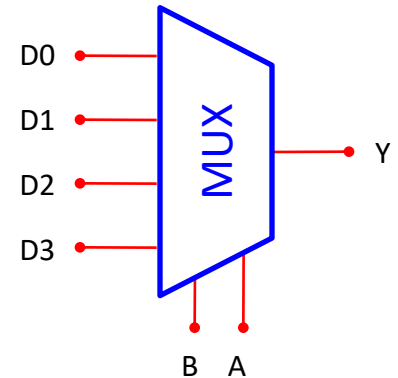


SELECT input code determines which input is transmitted to output Z.

4-to-1 Multiplexer (MUX)



$$Y = A'B'D0 + A'BD1 + AB'D2 + ABD3$$



A	B	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

Uses of Multiplexers

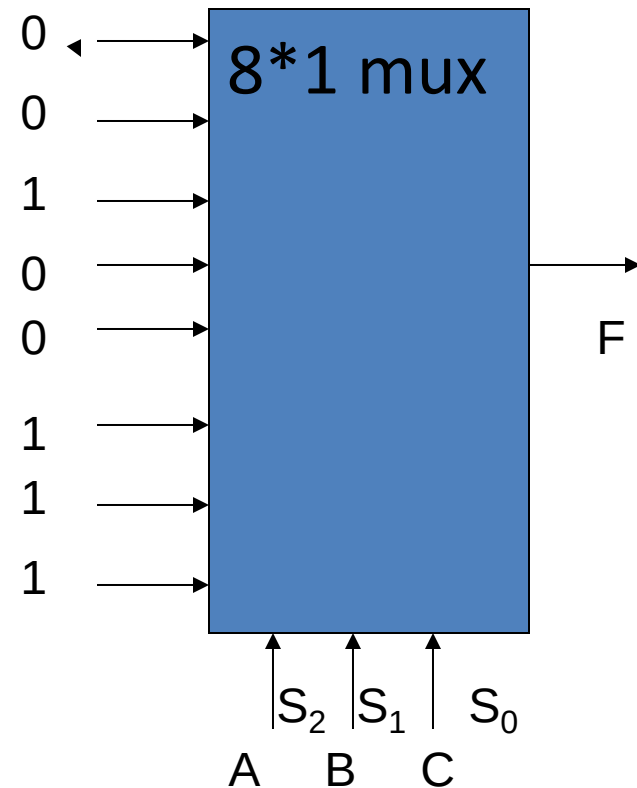
- Used in data communications for several computers to communicate over 1 line
- Used in radio to select one channel from many
- Used to route data within a computer
- Used for Logic function generation

Logic gate designing using MUX

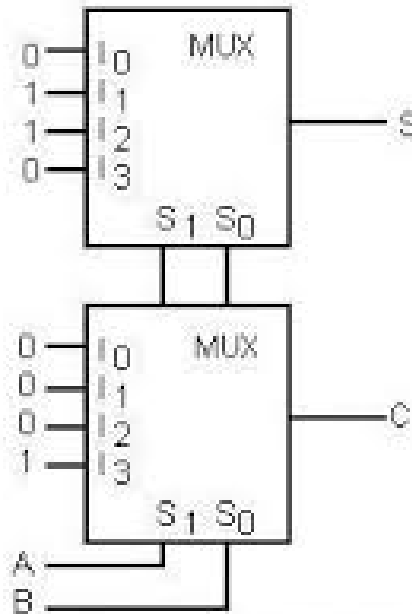
Logic function generation

Implement the Given truth table using a suitable MUX

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

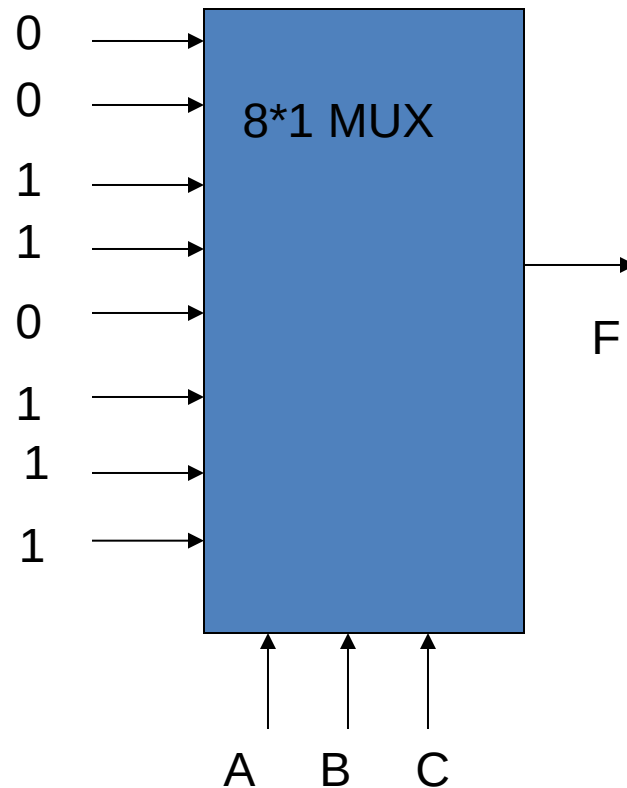


Implement the Half-adder circuit by using suitable multiplexer.



Using an 8-to-1 multiplexer, design a logic circuit to realize the following Boolean function

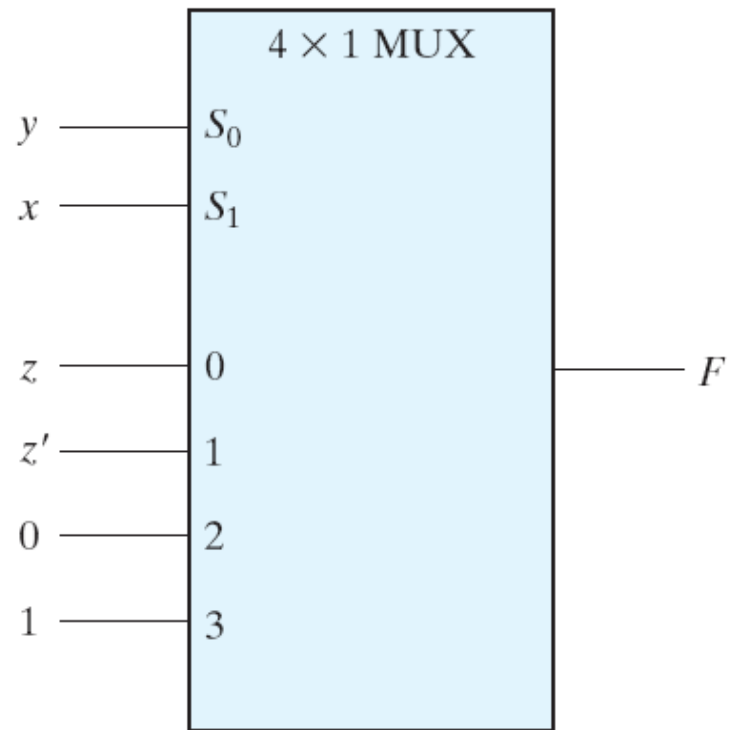
$$F(A,B,C) = \sum m(2, 3, 5, 6, 7)$$



$F(x,y,z) = \Sigma m(1, 2, 6, 7)$ using 4×1 mux

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table

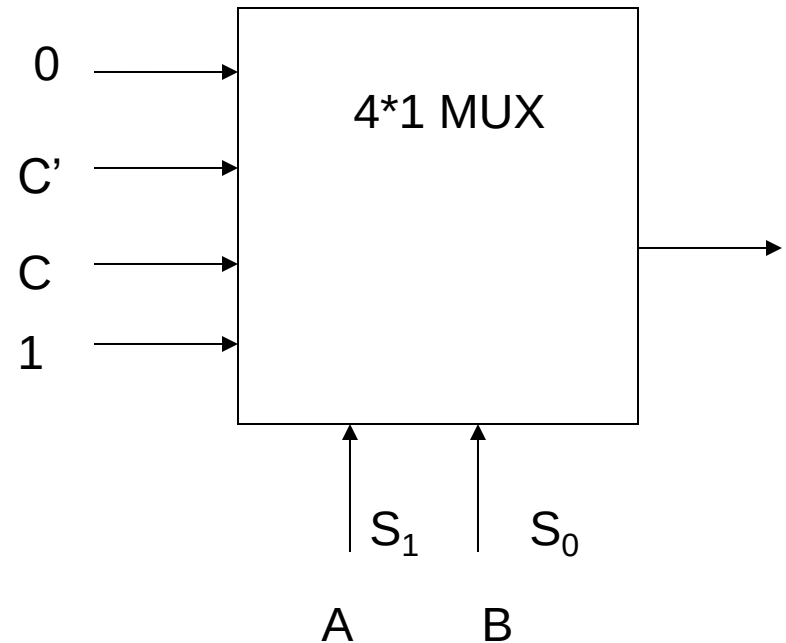


(b) Multiplexer implementation

Function Generation using Multiplexers

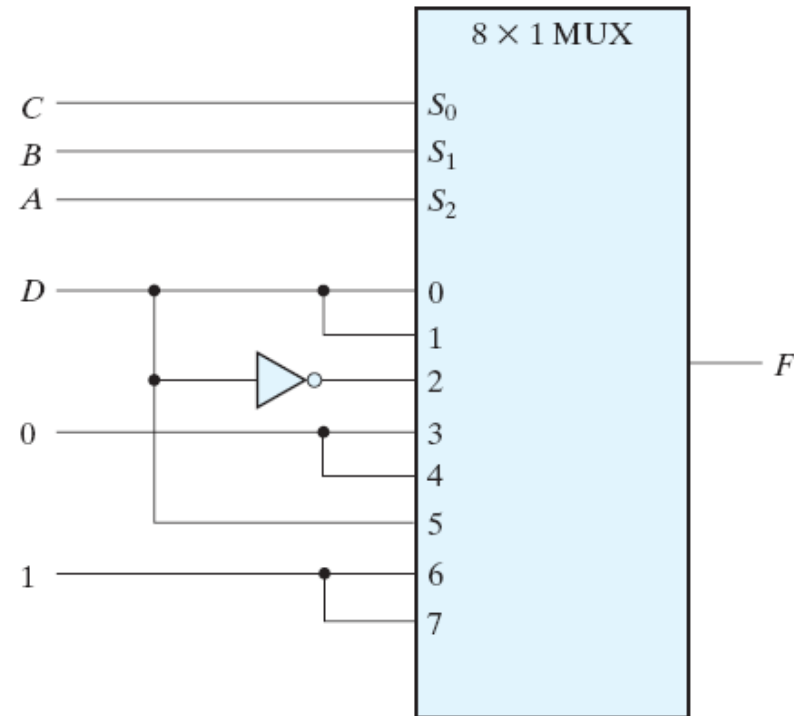
- Given the following truth table use a 4*1 Mux to generate F

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Example: $F(A,B,C,D) = \sum m(1,3,4,11,12-15)$ using 8×1 mux

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>F</i>	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



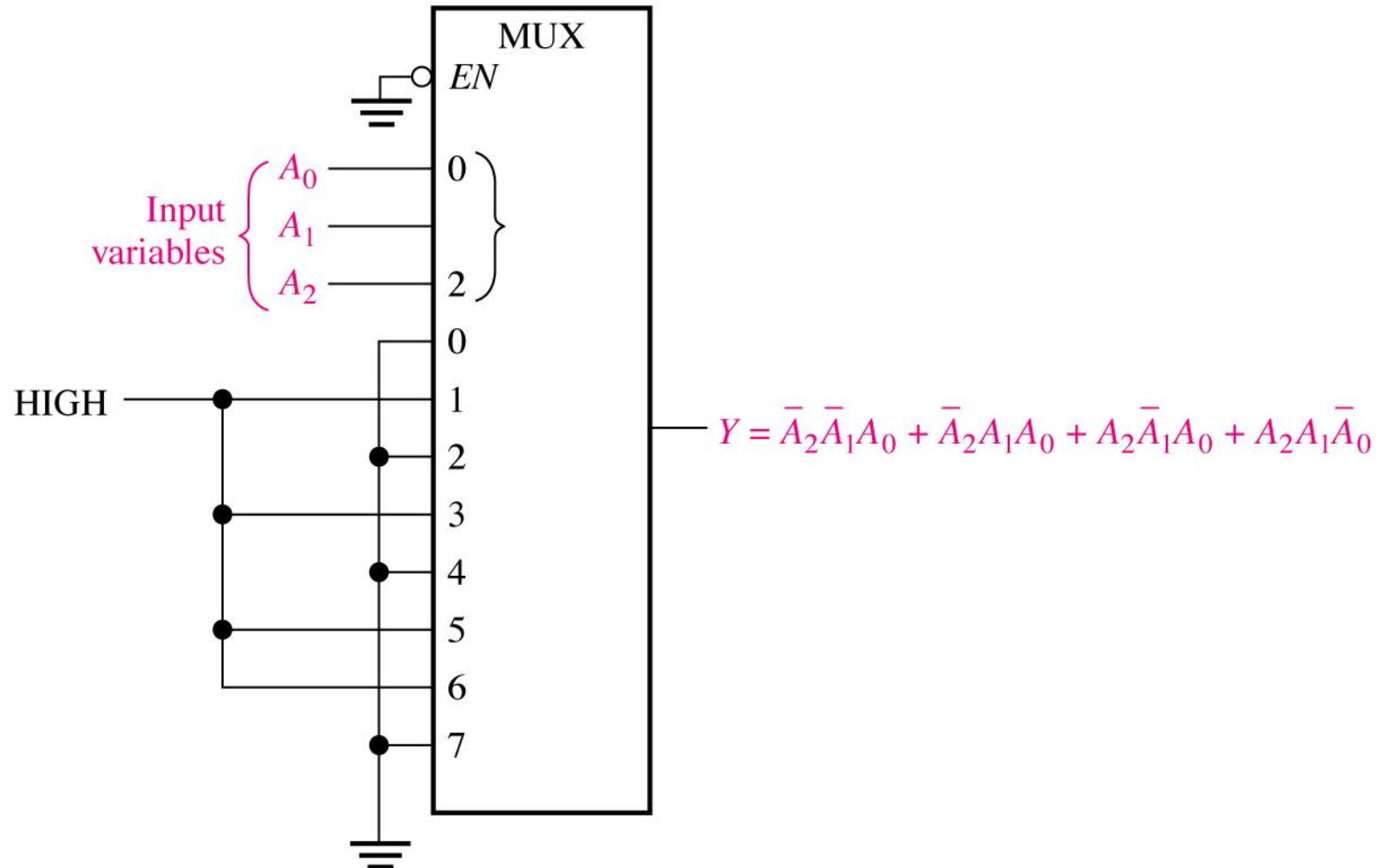
Example: $F(A,B,C,D) = \sum m(1,3,4,11,12-15)$ using 4*1 mux

LOGIC FUNCTION GENERATION USING MUX

Exercise 1:
Implement the logic circuit function specified in the table given below by using 74LS151 8-input data selector/multiplexer.

Input			Output	
A2	A1	A0	Y	
0	0	0	0	0
0	0	1	1	1
0	1	0	0	2
0	1	1	1	3
1	0	0	0	4
1	0	1	1	5
1	1	0	1	6
1	1	1	0	7

Solution :

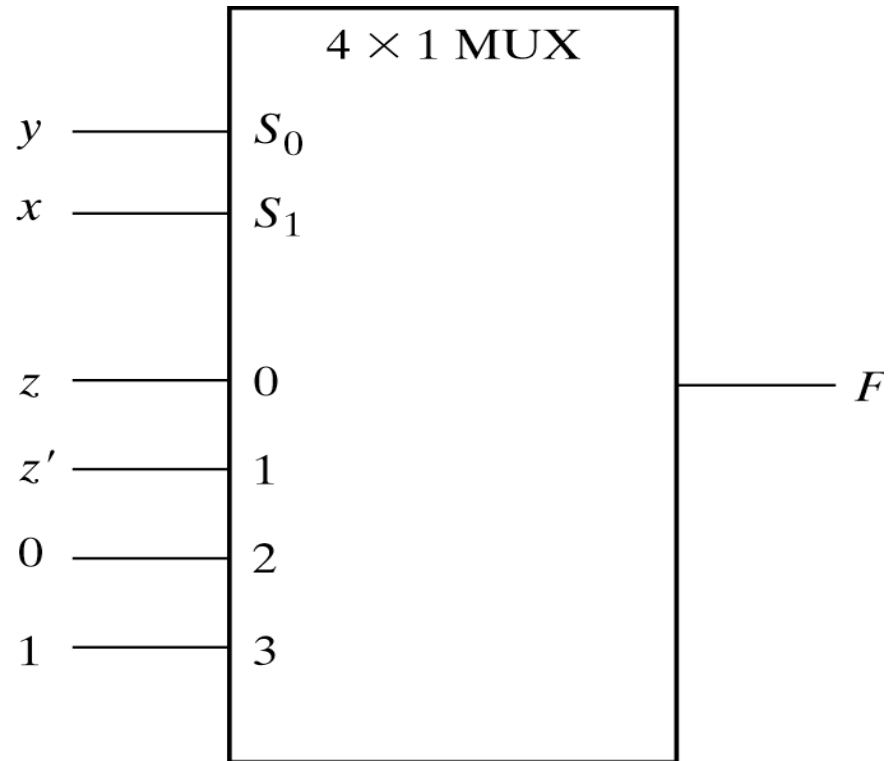


LOGIC FUNCTION GENERATION USING 4 * 1 MUX

Implement the Boolean function $F = x'y'z + x'yz' + xyz' + xyz$ using a suitable MUX

x	y	z	F		
0	0	0	0		
0	0	1	1	$F = z$	0
0	1	0	1		
0	1	1	0	$F = z'$	1
1	0	0	0		
1	0	1	0	$F = 0$	2
1	1	0	1		
1	1	1	1	$F = 1$	3

(a) Truth table



(b) Multiplexer implementation

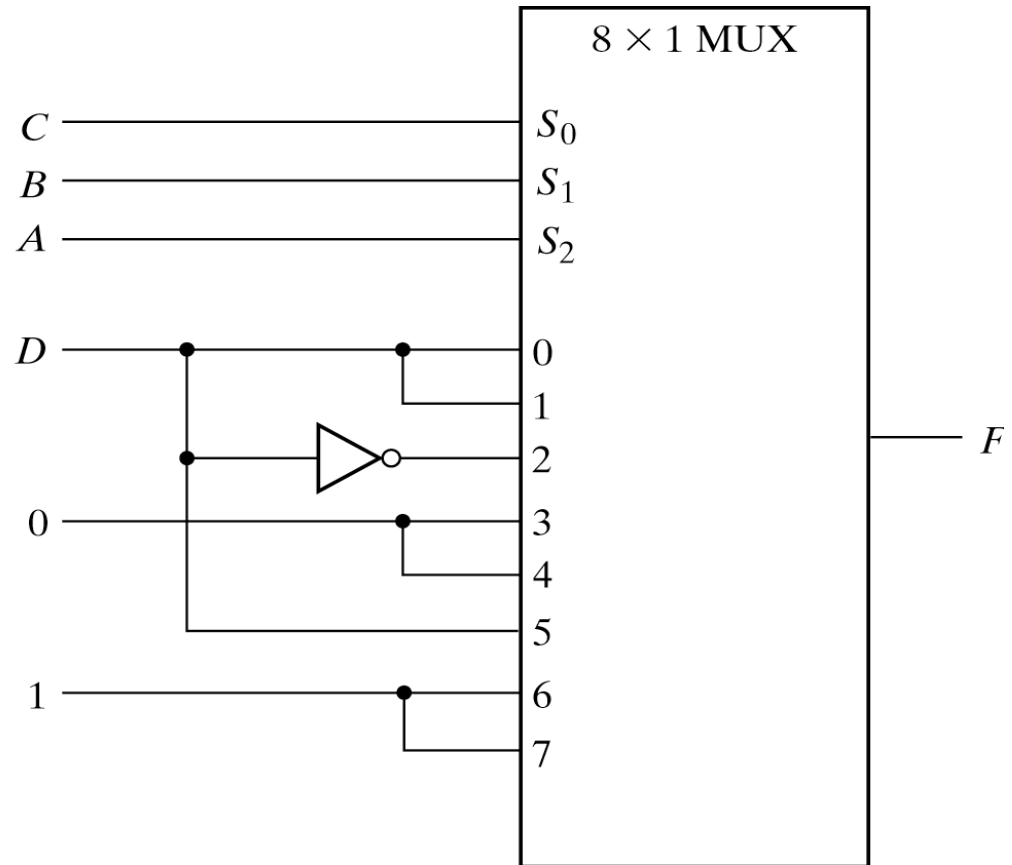
LOGIC FUNCTION GENERATION USING 8*1 MUX

Implement the Boolean function

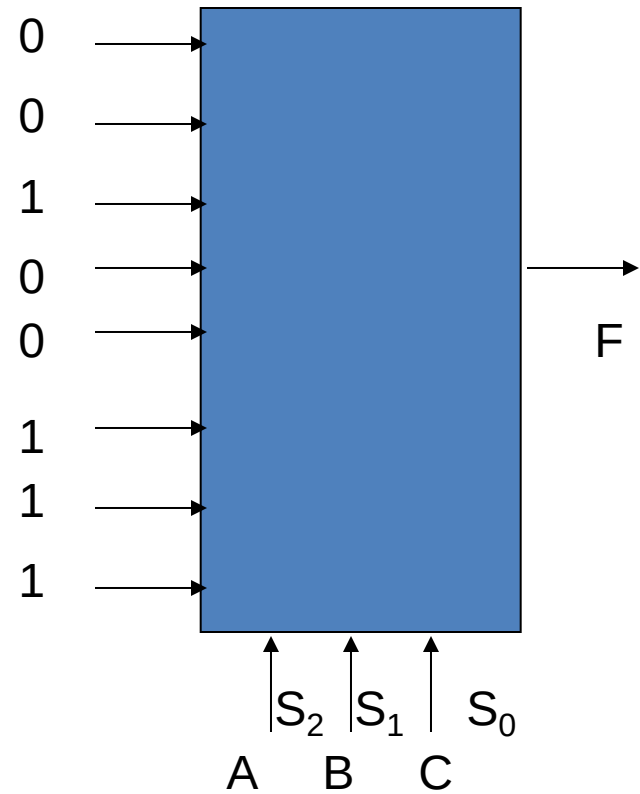
$$F = A'B'C'D + A'B'CD + A'BC'D' + AB'CD + ABC'D' + ABC'D + ABCD' + ABCD$$

using a suitable MUX and 8*1 Mux.

A	B	C	D	F		
0	0	0	0	0		
0	0	0	1	1	$F = D$	0
0	0	1	0	0	$F = D$	1
0	0	1	1	1		
0	1	0	0	1	$F = D'$	2
0	1	0	1	0		
0	1	1	0	0	$F = 0$	3
0	1	1	1	0		
1	0	0	0	0	$F = 0$	4
1	0	0	1	0		
1	0	1	0	0	$F = D$	5
1	0	1	1	1		
1	1	0	0	1	$F = 1$	6
1	1	0	1	1		
1	1	1	0	1	$F = 1$	7
1	1	1	1	1		

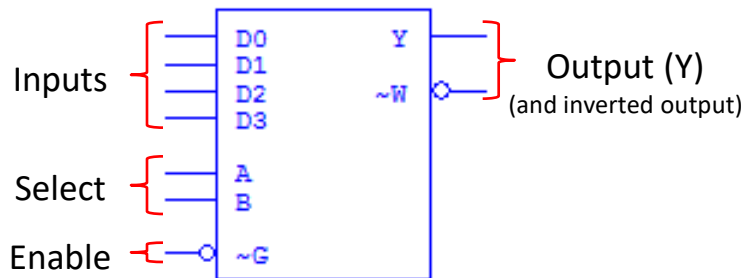


Find out the Boolean expression of given block diagram

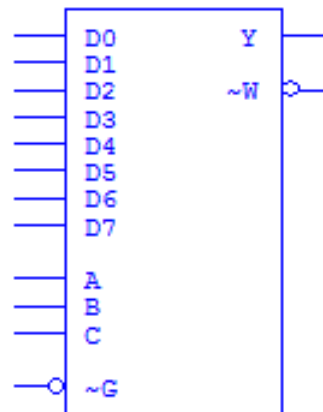


Medium Scale Integration MUX

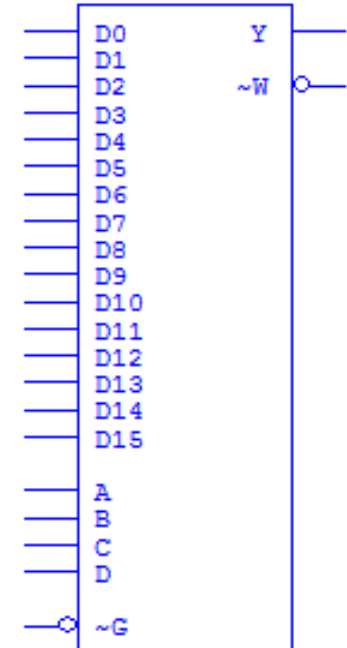
4-to-1 MUX



8-to-1 MUX

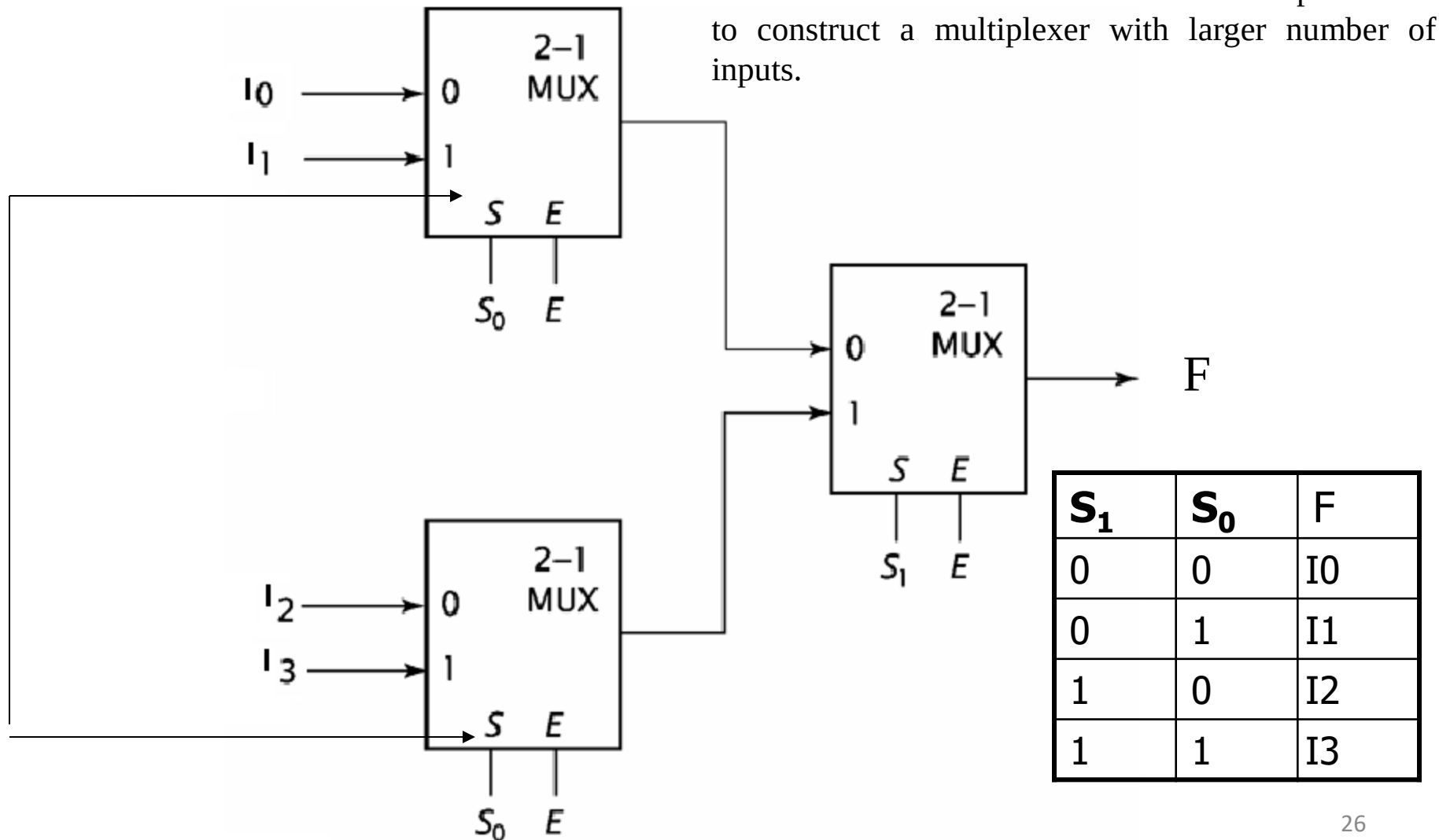


16-to-1 MUX



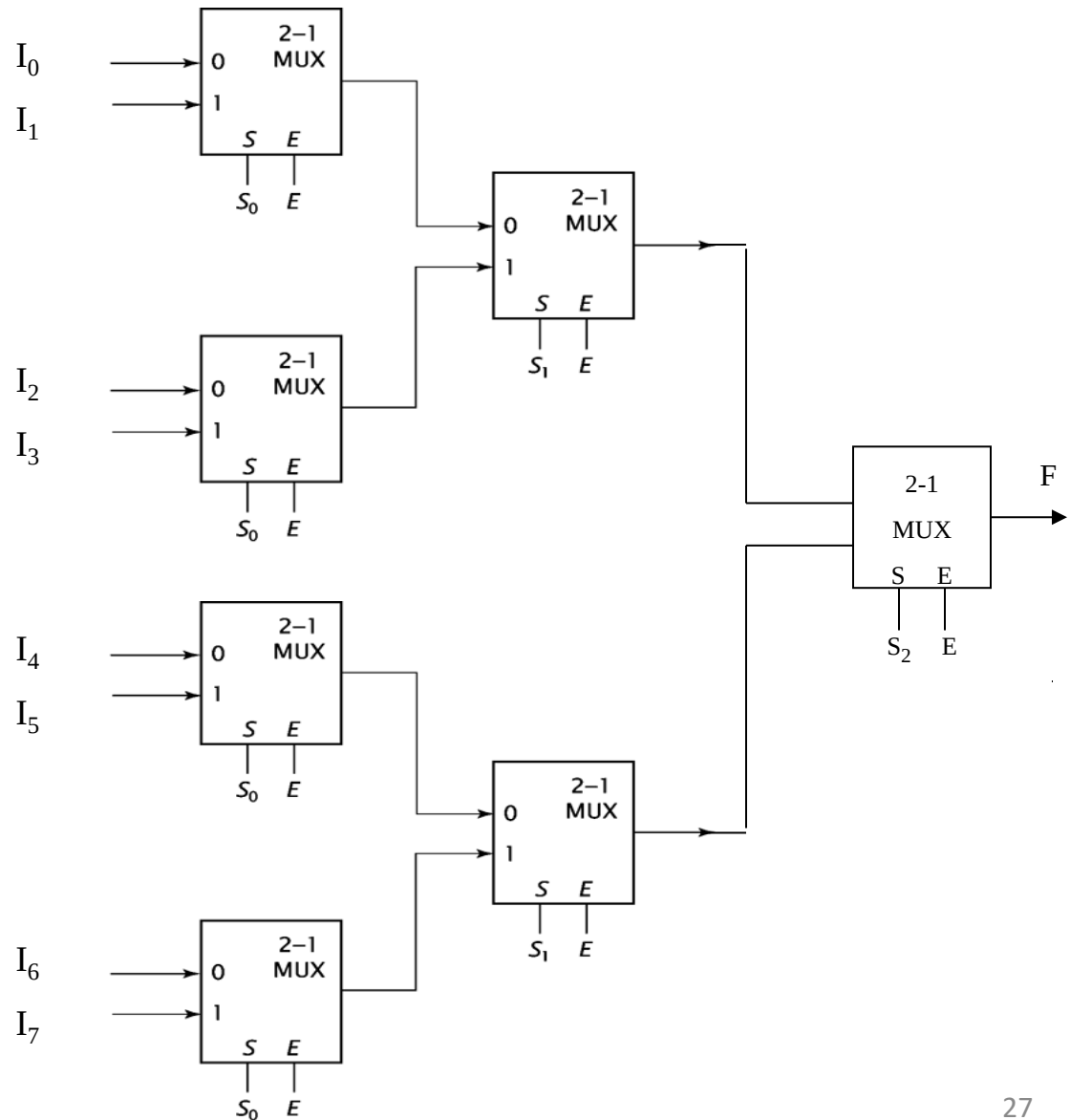
Cascading multiplexers/Mux Tree

Using three 2-1 MUX to make one 4-1 MUX

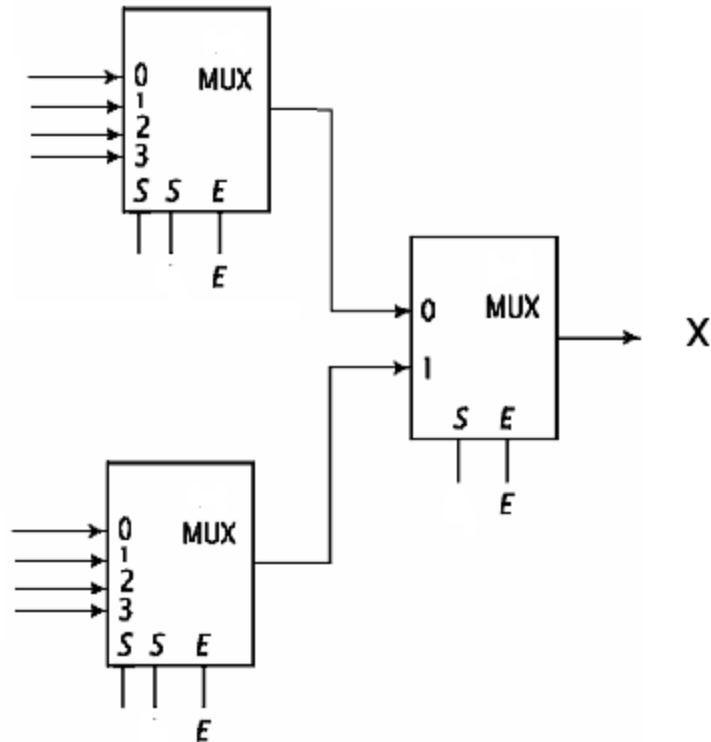


Construct an 8-to-1 multiplexer using 2-to-1 multiplexers.

S_2	S_1	S_0	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7



Example: Construct 8-to-1 multiplexer using one 2-to-1 multiplexer and two 4-to-1 multiplexers



S_2	S_1	S_0	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

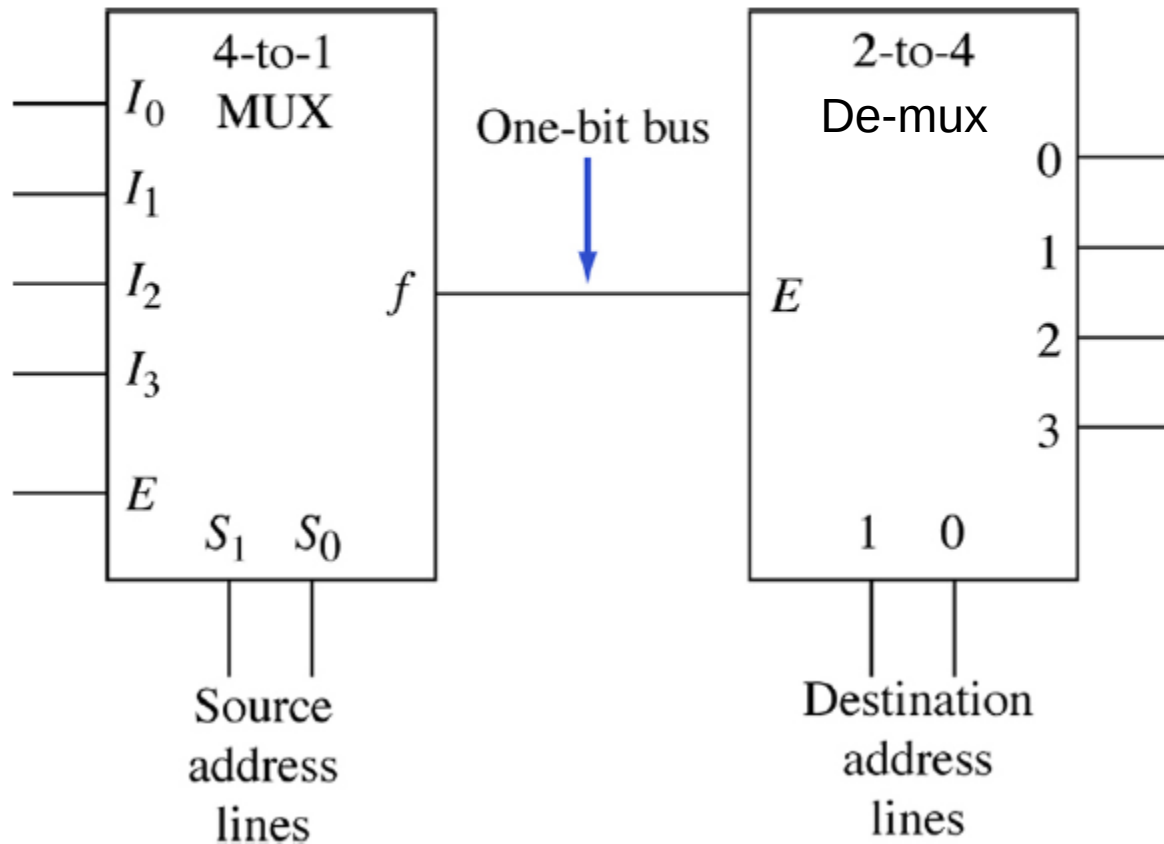
Example: Construct 8-to-1 multiplexer using only two 4-to-1 multiplexers and basic gates.

DEMULTIPLEXERS

A DEMULTIPLEXER (DEMUX) basically reverses the multiplexing function. It takes data from one line and distributes them to a given number of output lines. For this reason, the demultiplexers is also known as a data distributor.

A demultiplexer (DEMUX) performs the reverse operation of MUX. It takes a single input and distributes it over several outputs.

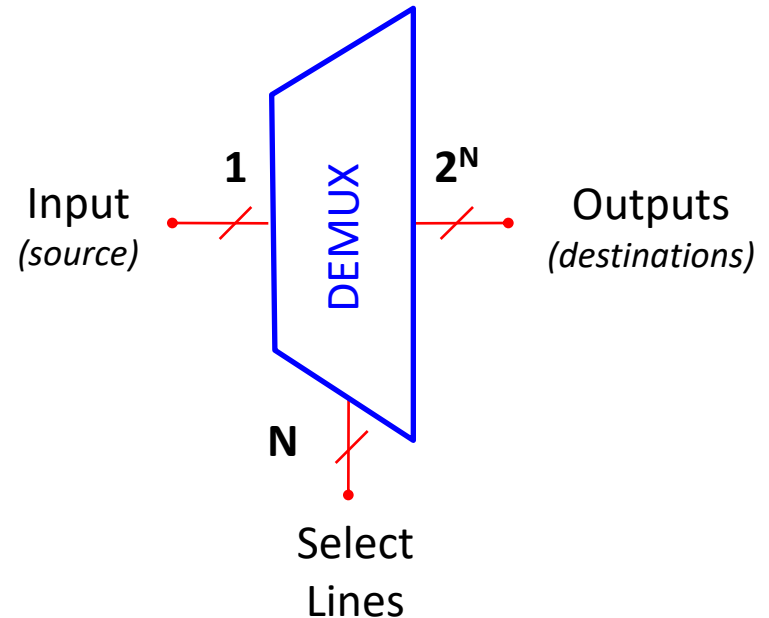
A multiplexer/demultiplexer arrangement for information transmission.



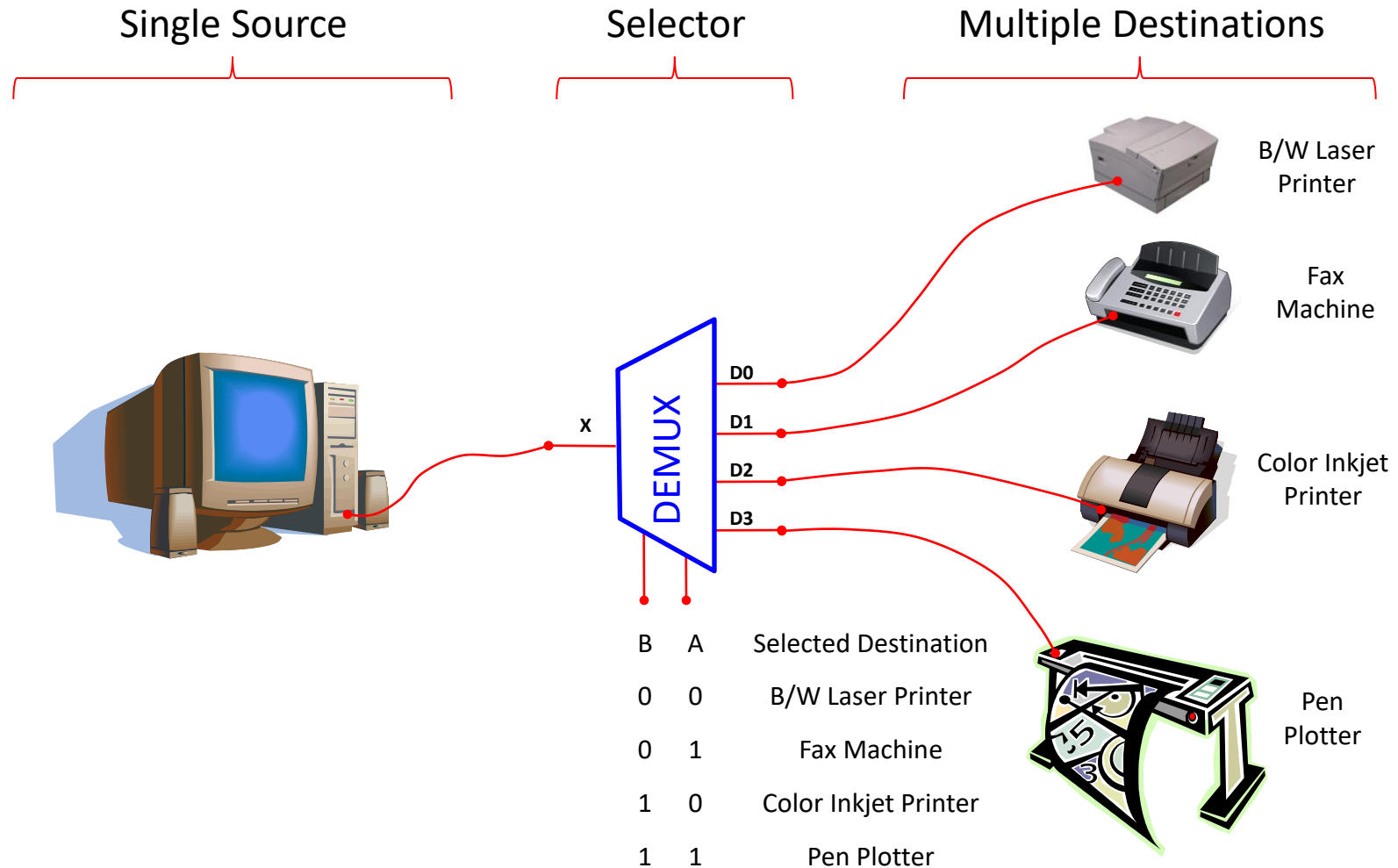
What is a Demultiplexer (DEMUX)?

- A DEMUX is a digital switch with a single input (source) and a multiple outputs (destinations).
- The select lines determine which output the input is connected to.
- DEMUX Types
 - 1-to-2 (1 select line)
 - 1-to-4 (2 select lines)
 - 1-to-8 (3 select lines)
 - 1-to-16 (4 select lines)

Demultiplexer
Block Diagram

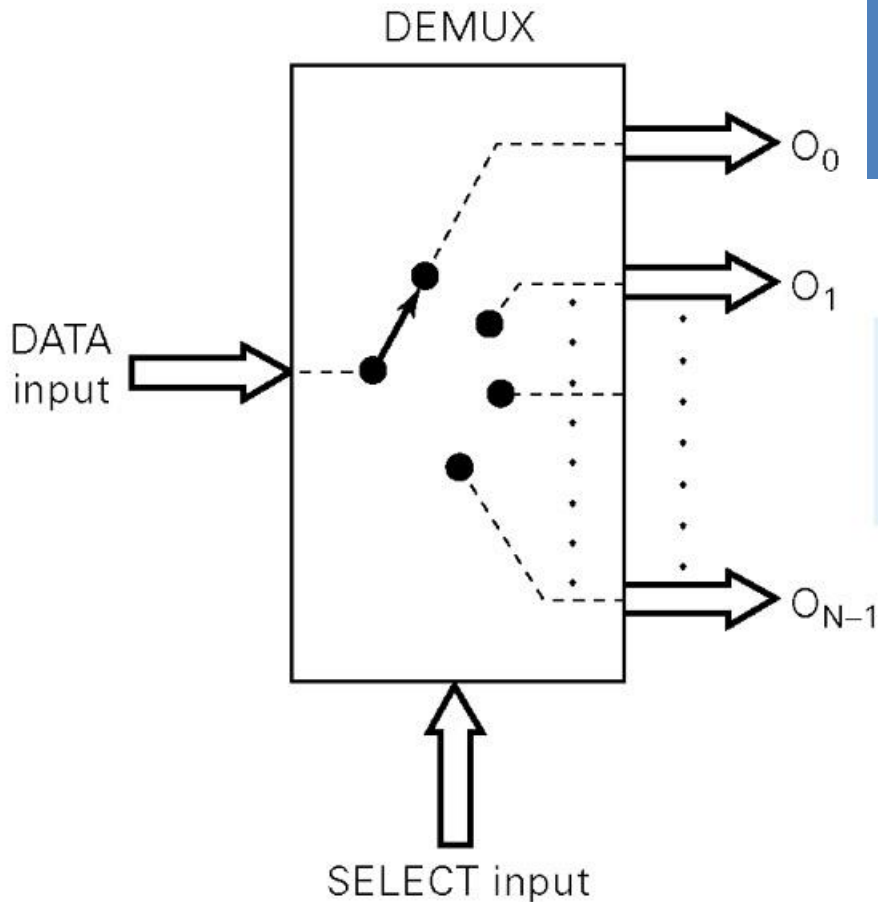


Typical Application of a DEMUX



General demultiplexer

Functional diagram:-

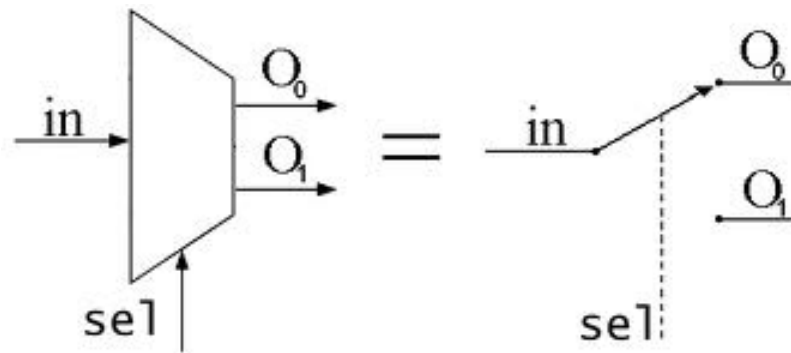


The large arrow indicates one or more lines. The select i/p code determines to which output the DATA input will be transmitted

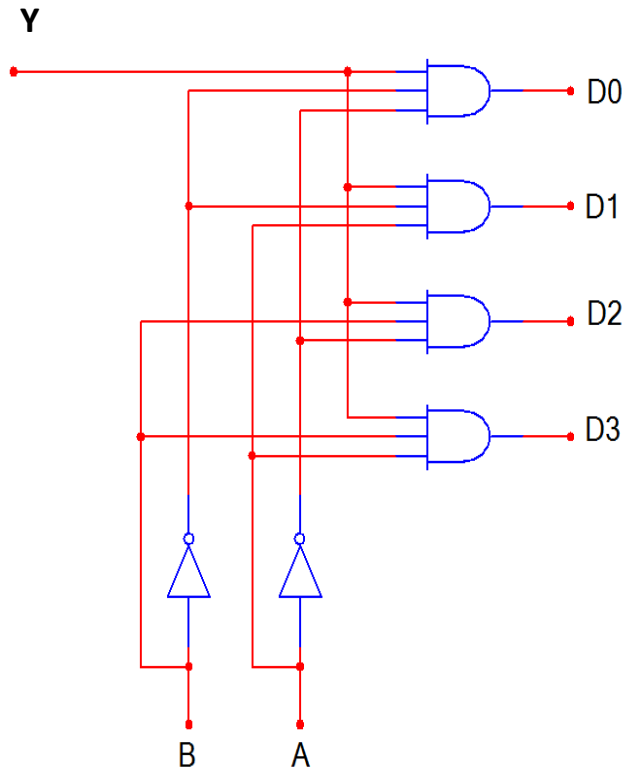
DATA input is transmitted to only one of the outputs as determined by select input code

In other words, the demultiplexer takes one data input source and selectively distributes it to 1 of N output channels just like multiposition switch.

1-to-2 De-Multiplexer



1-to-4 De-Multiplexer (DEMUX)

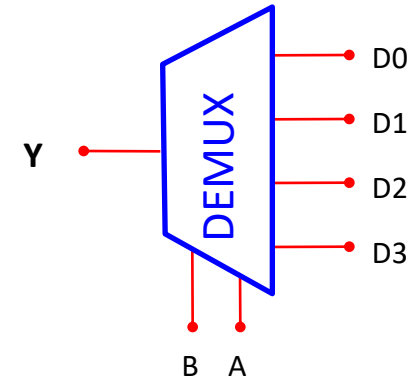


$$D0 = A'.B'.Y$$

$$D1 = A.B'.Y$$

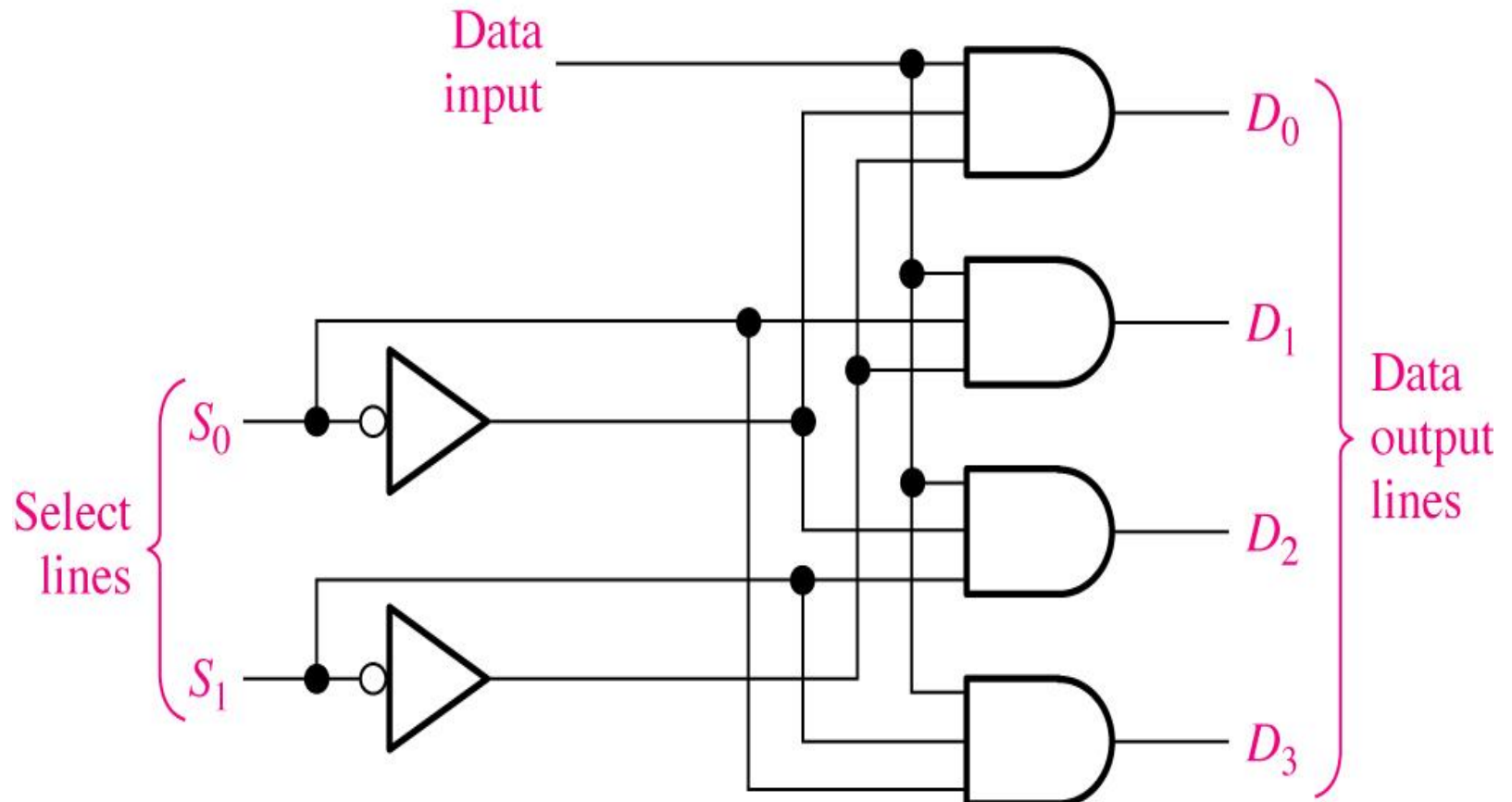
$$D2 = A'.B.Y$$

$$D3 = A.B.Y$$

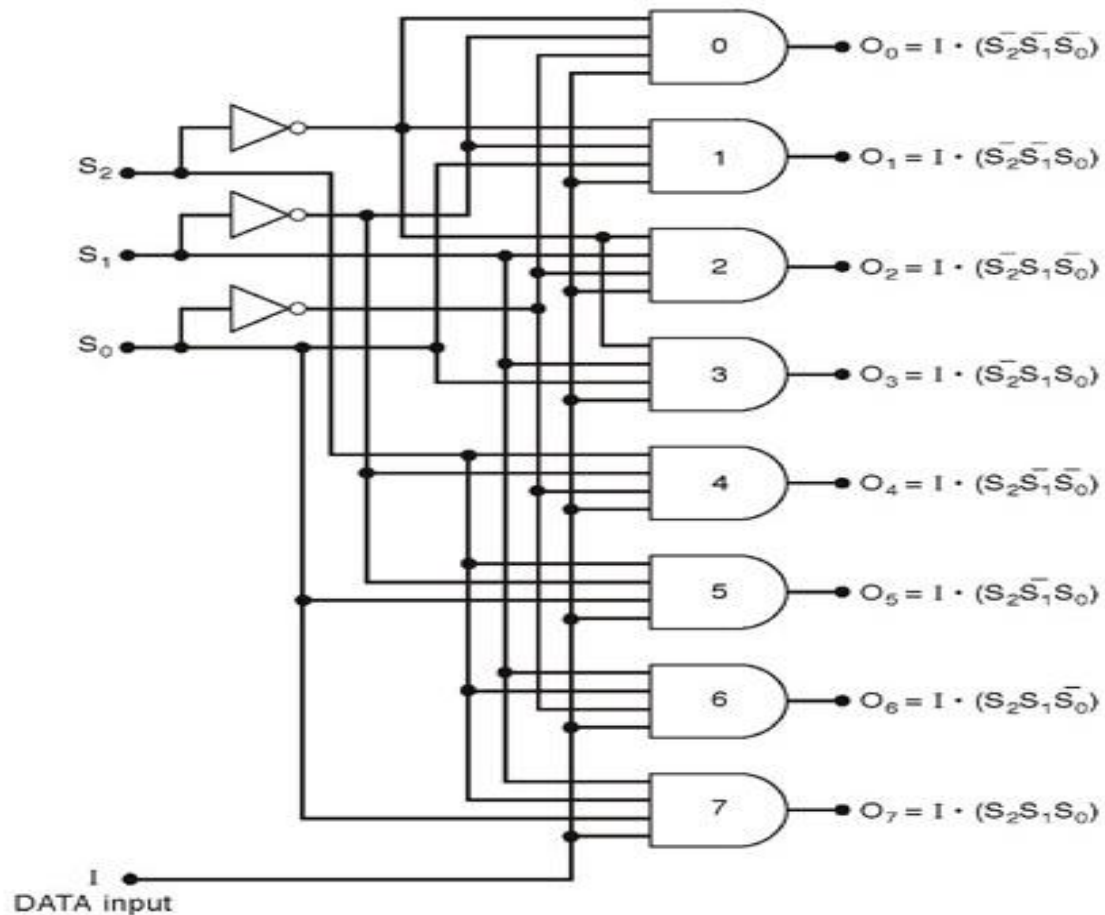


B	A	D0	D1	D2	D3
0	0	Y	0	0	0
0	1	0	Y	0	0
1	0	0	0	Y	0
1	1	0	0	0	Y

A 1-line-to-4-line demultiplexer.



1-line- to-8 line demultiplexer

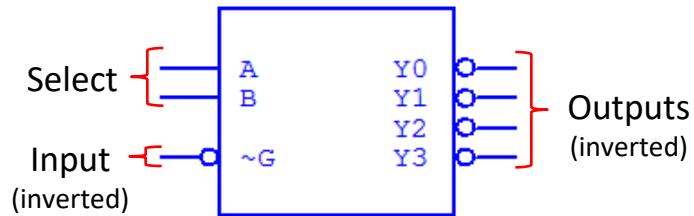


SELECT code			OUTPUTS							
S_2	S_1	S_0	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0	0	0	0	I
0	0	1	0	0	0	0	0	0	I	0
0	1	0	0	0	0	0	0	I	0	0
0	1	1	0	0	0	0	I	0	0	0
1	0	0	0	0	0	I	0	0	0	0
1	0	1	0	0	I	0	0	0	0	0
1	1	0	0	I	0	0	0	0	0	0
1	1	1	I	0	0	0	0	0	0	0

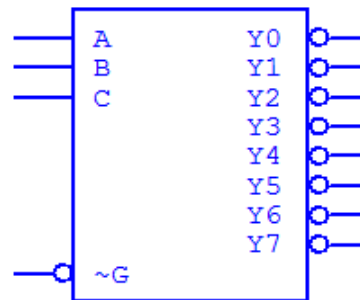
Note: I is the data input

Medium Scale Integration DEMUX

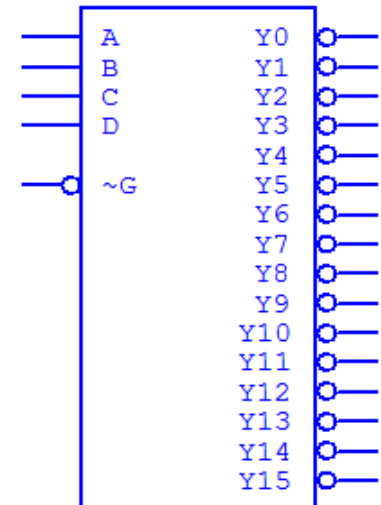
1-to-4 DEMUX



1-to-8 DEMUX



16-to-1 MUX



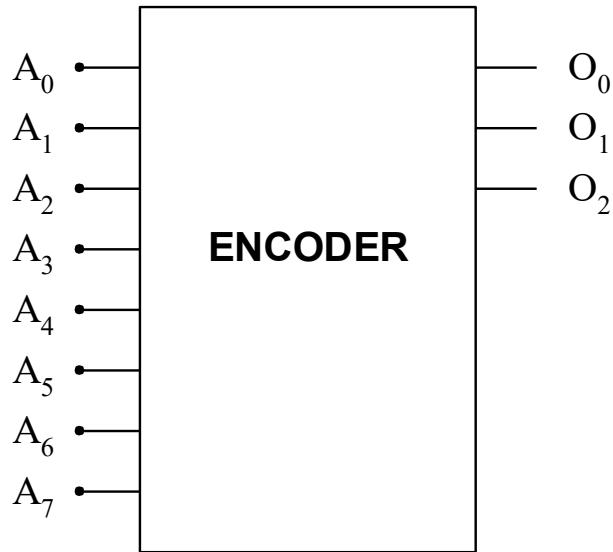
Decoders and Encoders

Encoder/Decoder

ENCODER- a digital circuit that produces a binary output code depending on which of its inputs are activated.

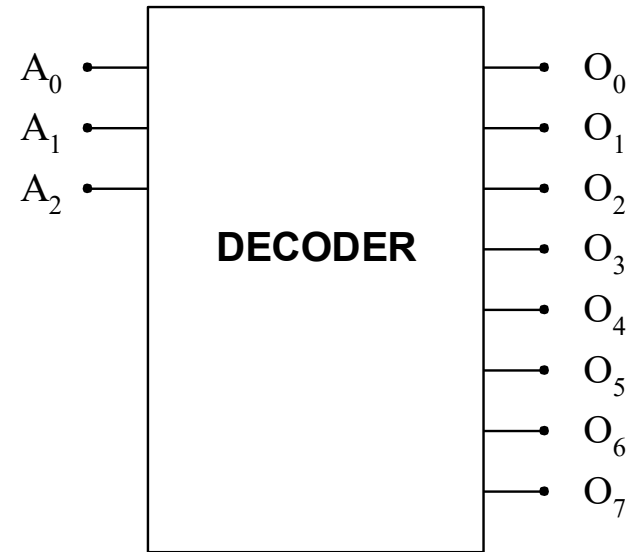
DECODER- a digital circuit that converts an input binary code into a single numeric output.

ENCODERS AND DECODERS



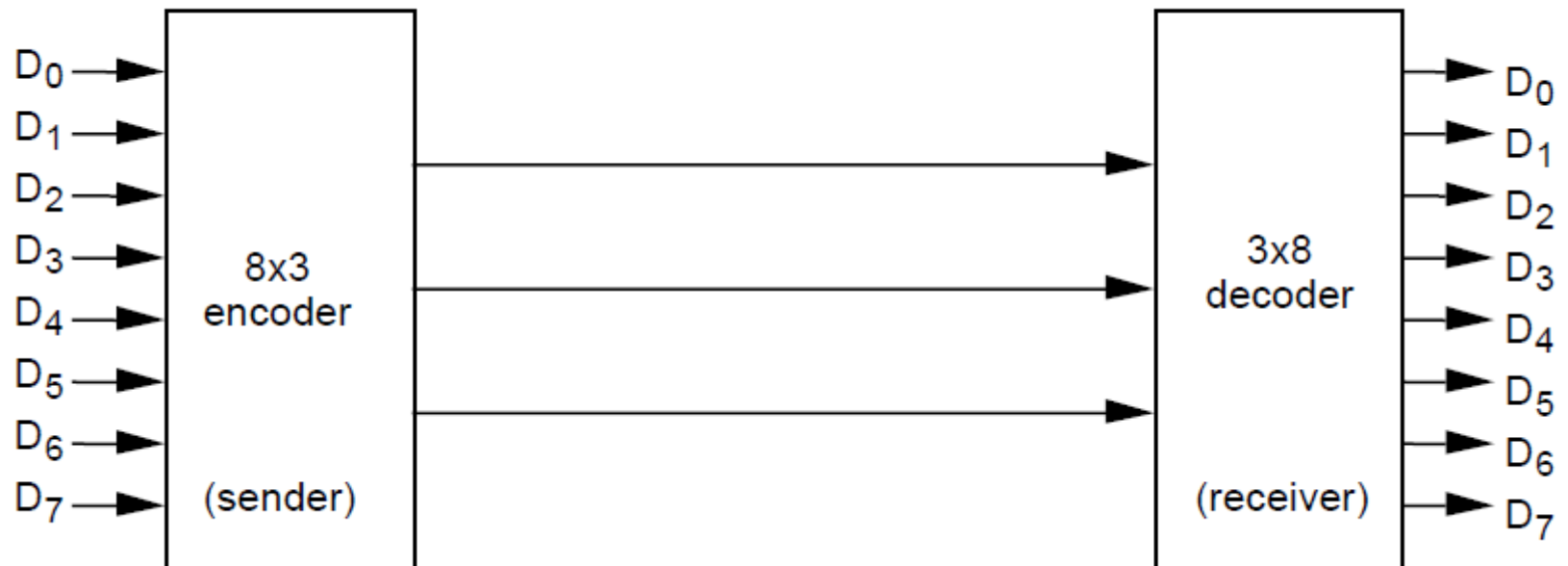
ONLY ONE INPUT
ACTIVATED AT A TIME

BINARY CODE OUTPUT



BINARY CODE INPUT

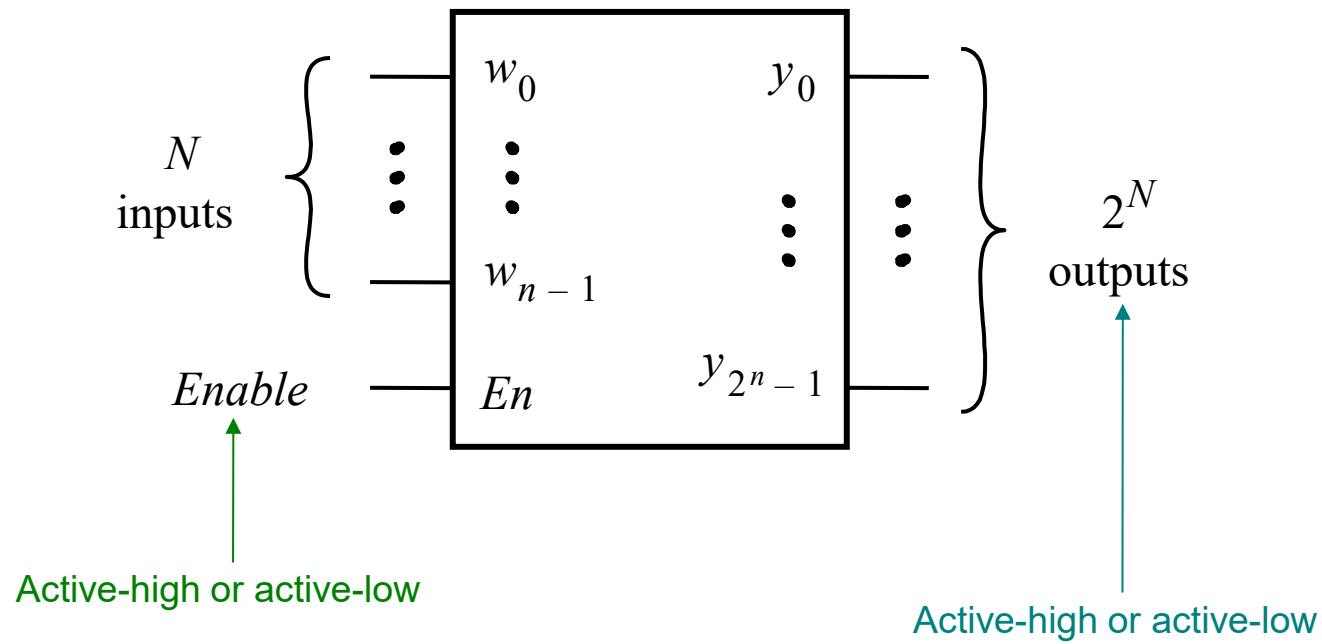
ONLY ONE OUTPUT
ACTIVATED AT A TIME



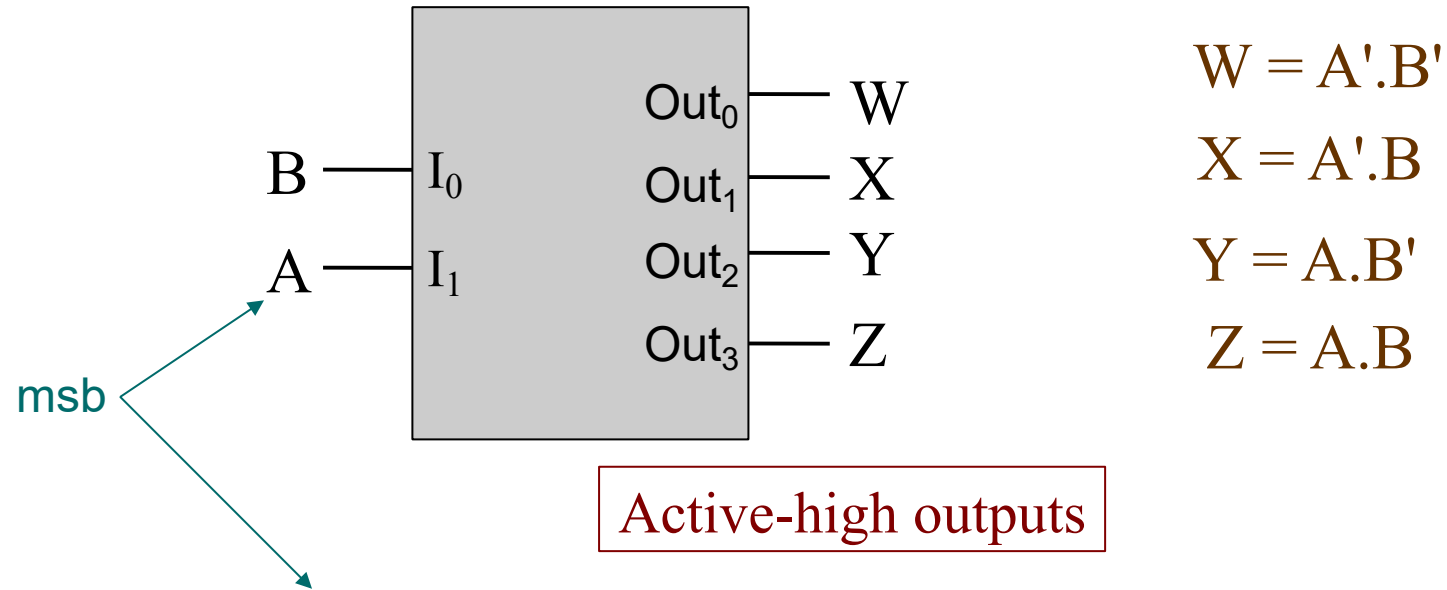
Main function of encoder and decoder

The purpose is to reduce the number of wires required for interconnection.

Decoder: N-to- 2^N



Decoders

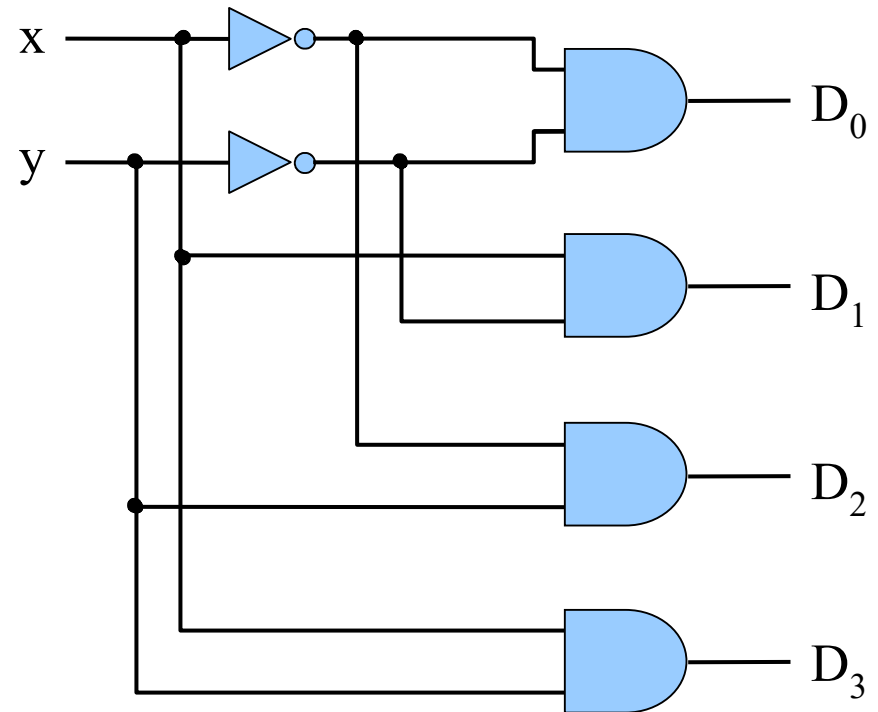


A	B	W	X	Y	Z
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Decoder: 2-to-4

y	x	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

active-high output



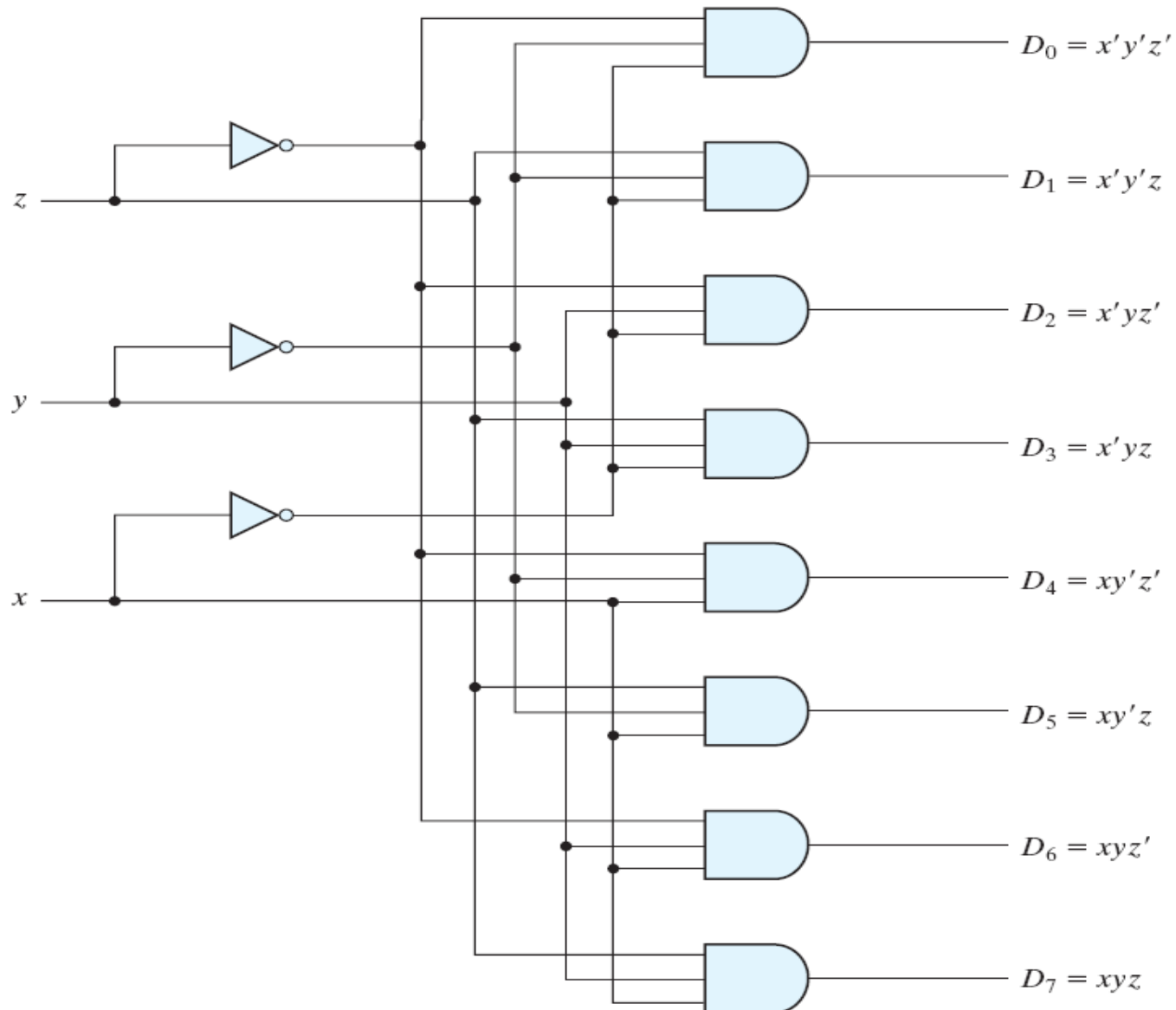
Decoder: 3-to-8

Table 4.6

Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
<i>x</i>	<i>y</i>	<i>z</i>	<i>D</i> ₀	<i>D</i> ₁	<i>D</i> ₂	<i>D</i> ₃	<i>D</i> ₄	<i>D</i> ₅	<i>D</i> ₆	<i>D</i> ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

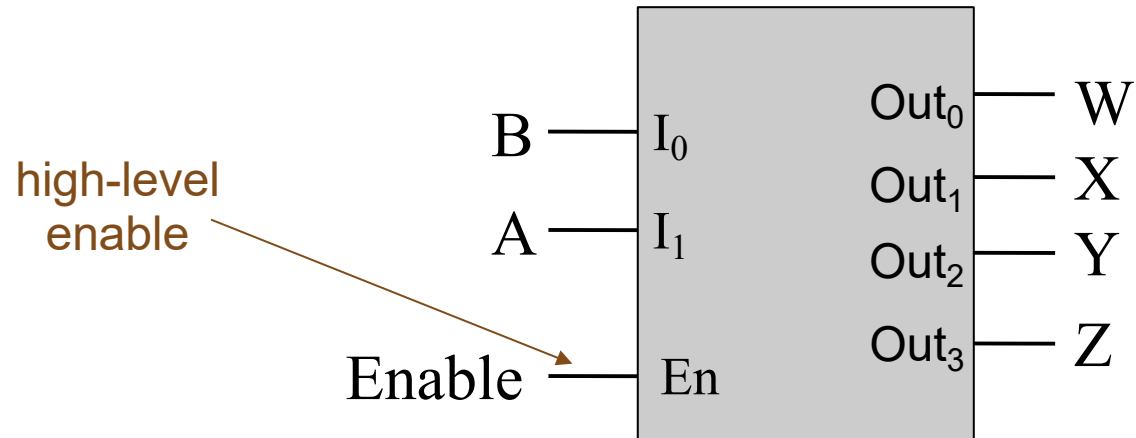
Decoder: 3-to-8



Definitions

- Active – the function is being performed
 - Active low – active when the output is 0.
 - Active high – active when the output is 1.
- Enabled – active circuit can perform its function
 - Low-level enabled
 - circuit performs its function when Enable = 0.
 - High-level enabled
 - circuit performs its function when Enable = 1.

Decoder with Enable

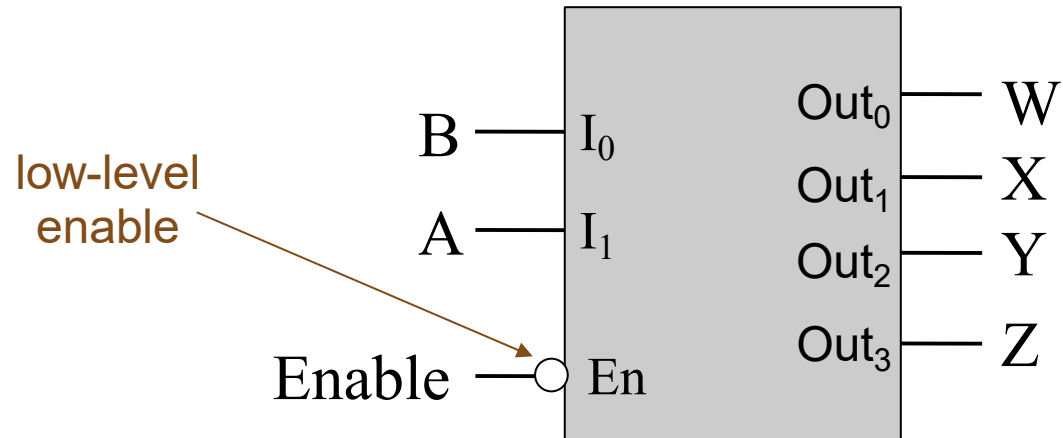


enabled

disabled

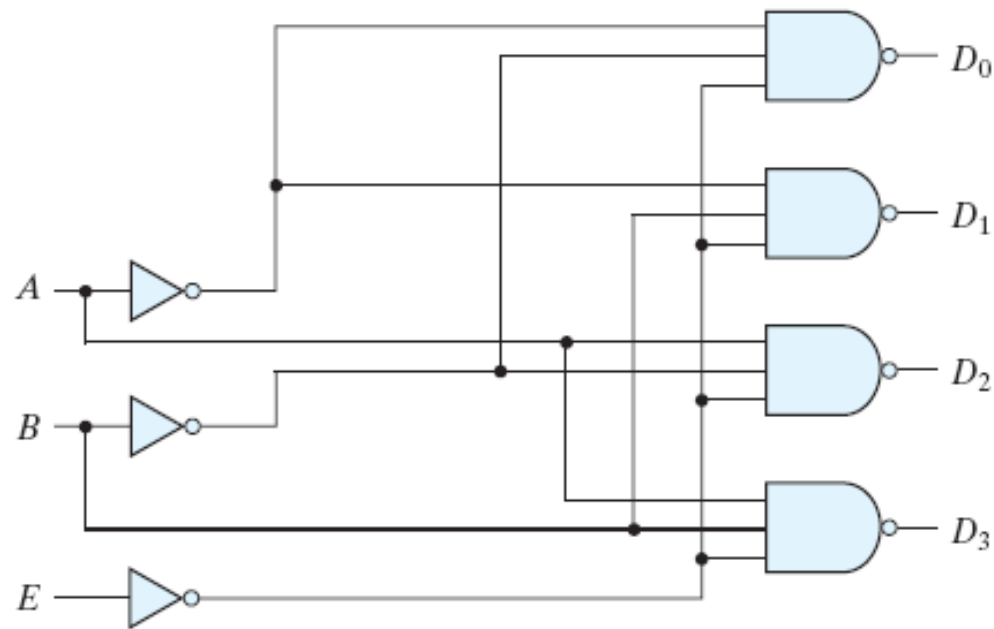
En	A	B	W	X	Y	Z
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

Decoder with Enable



	En	A	B	W	X	Y	Z
enabled	0	0	0	1	0	0	0
	0	0	1	0	1	0	0
	0	1	0	0	0	1	0
	0	1	1	0	0	0	1
disabled	1	x	x	0	0	0	0

Decoder: 2-to-4 with Enable



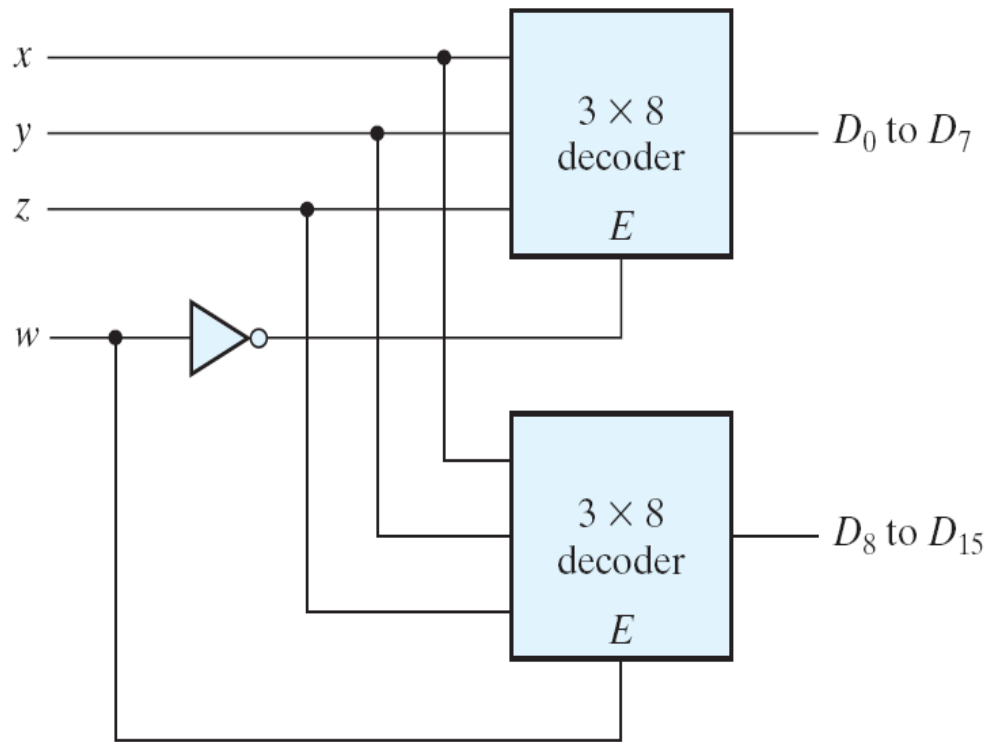
(a) Logic diagram

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

(b) Truth table

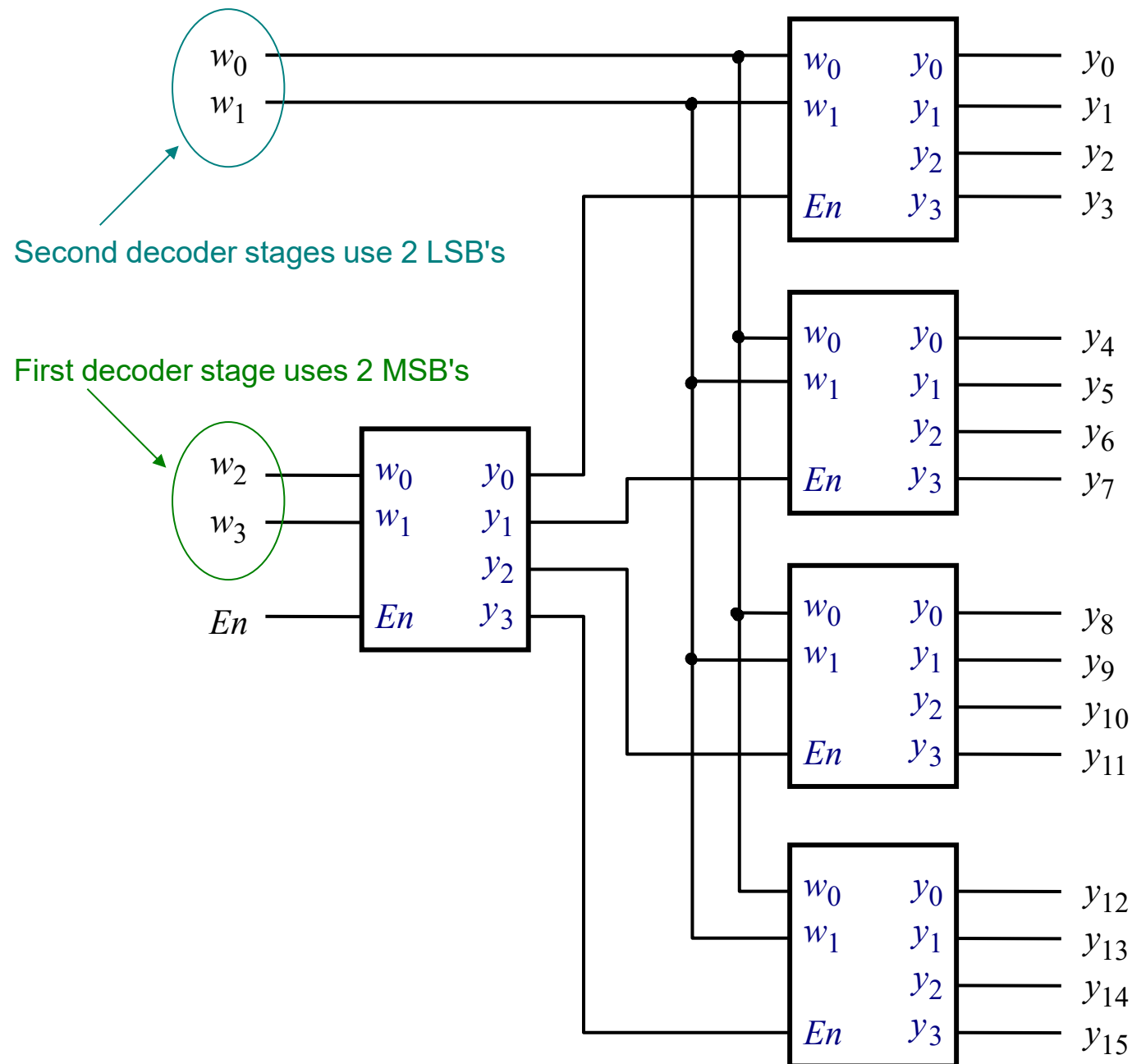
active-low output

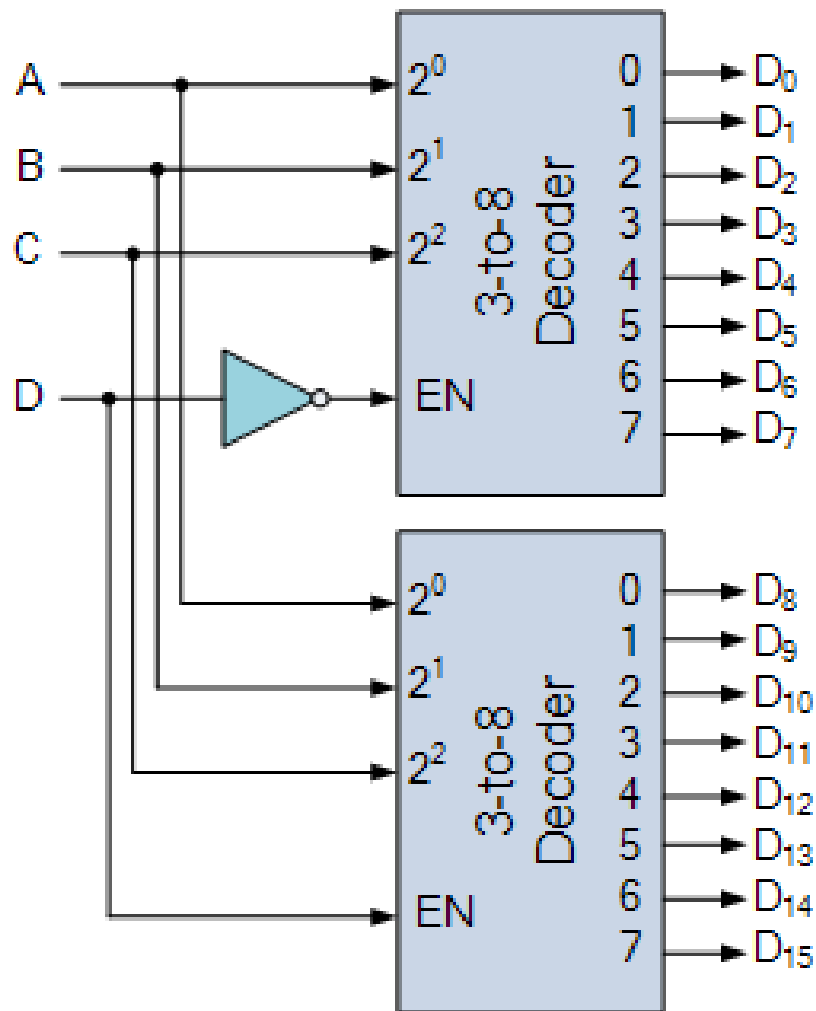
4*16 Decoder using 3* 8 Decoder



W	X	Y	Z	Output
0	0	0	0	D0
0	0	0	1	D1
0	0	1	0	D2
0	0	1	1	D3
0	1	0	0	D4
0	1	0	1	D5
0	1	1	0	D6
0	1	1	1	D7
1	0	0	0	D8
1	0	0	1	D9
1	0	1	0	D10
1	0	1	1	D11
1	1	0	0	D12
1	1	0	1	D13
1	1	1	0	D14
1	1	1	1	D15

4*16 Decoder using 2* 4 Decoder





4-to-16 Line Decoder Implemented
with two 3-to-8 Decoders

D	C	B	A	Output
0	0	0	0	D_0
0	0	0	1	D_1
0	0	1	0	D_2
0	0	1	1	D_3
0	1	0	0	D_4
0	1	0	1	D_5
0	1	1	0	D_6
0	1	1	1	D_7
1	0	0	0	D_8
1	0	0	1	D_9
1	0	1	0	D_{10}
1	0	1	1	D_{11}
1	1	0	0	D_{12}
1	1	0	1	D_{13}
1	1	1	0	D_{14}
1	1	1	1	D_{15}

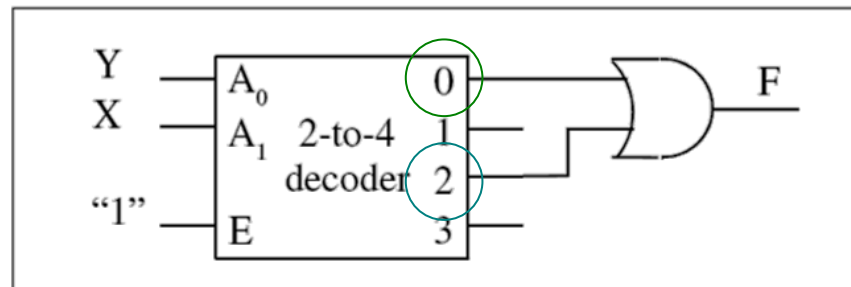
Decoders

Designing Logic Circuits
using Decoders

Decoders: Designing Logic Circuits

$$F = \Sigma m(0,2)$$

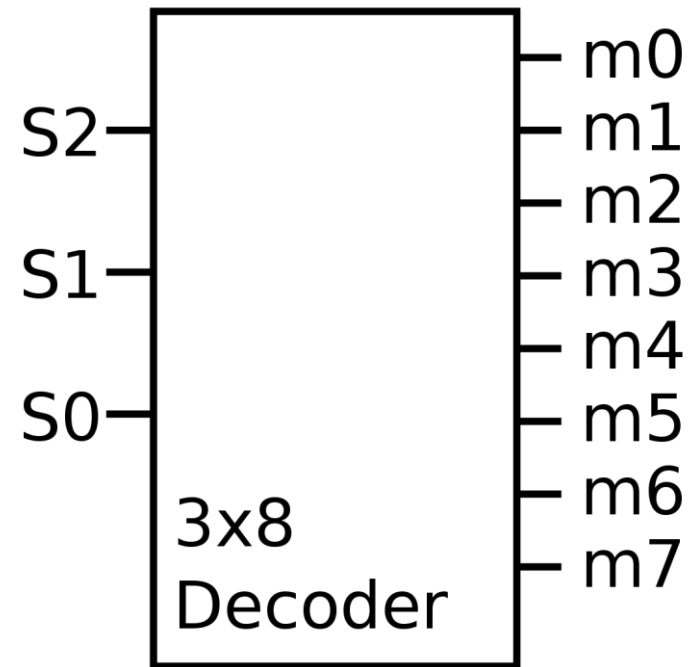
A	B	F	O ₃	O ₂	O ₁	O ₀
0	0	1				1
0	1	0			1	
1	0	1		1		
1	1	0	1			



Design all logic gates using a Decoder.

Design a circuit, using a 3-to-8 Decoder to realize the Boolean expression given below.

$$F_{X,Y,Z} = \sum m(1,2,5,7)$$

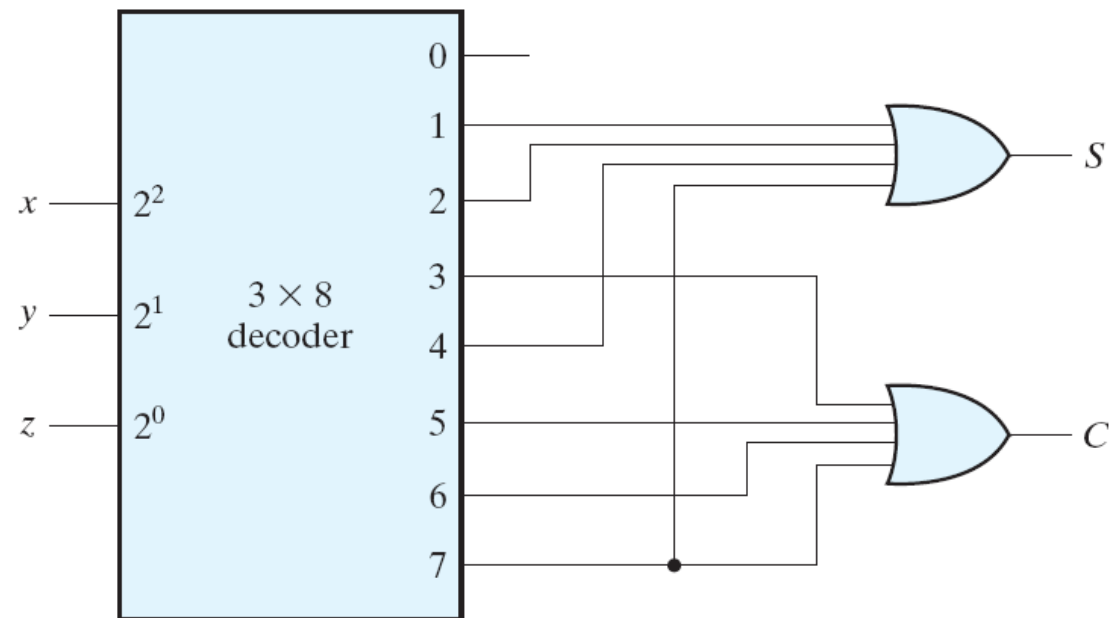


Decoders: Designing Logic Circuits

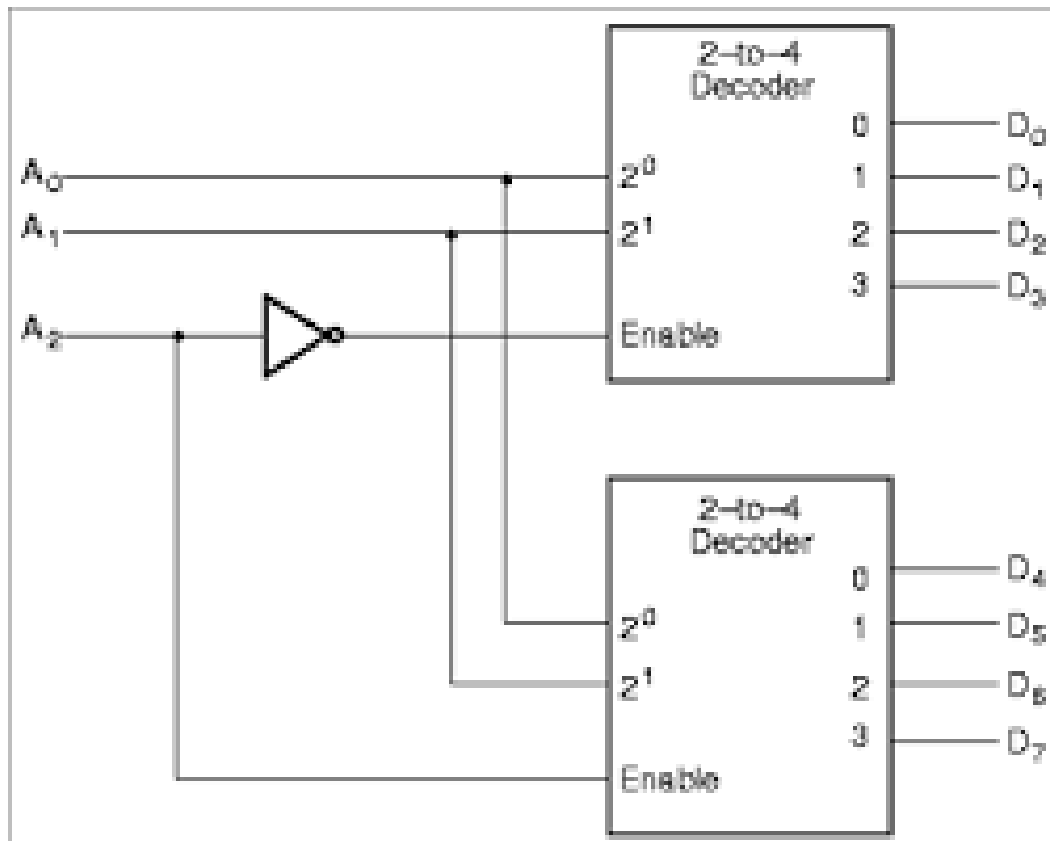
Exercise:

Design a Half Adder using a Decoder

Design a Full Adder using a 3-to-8 Decoder.



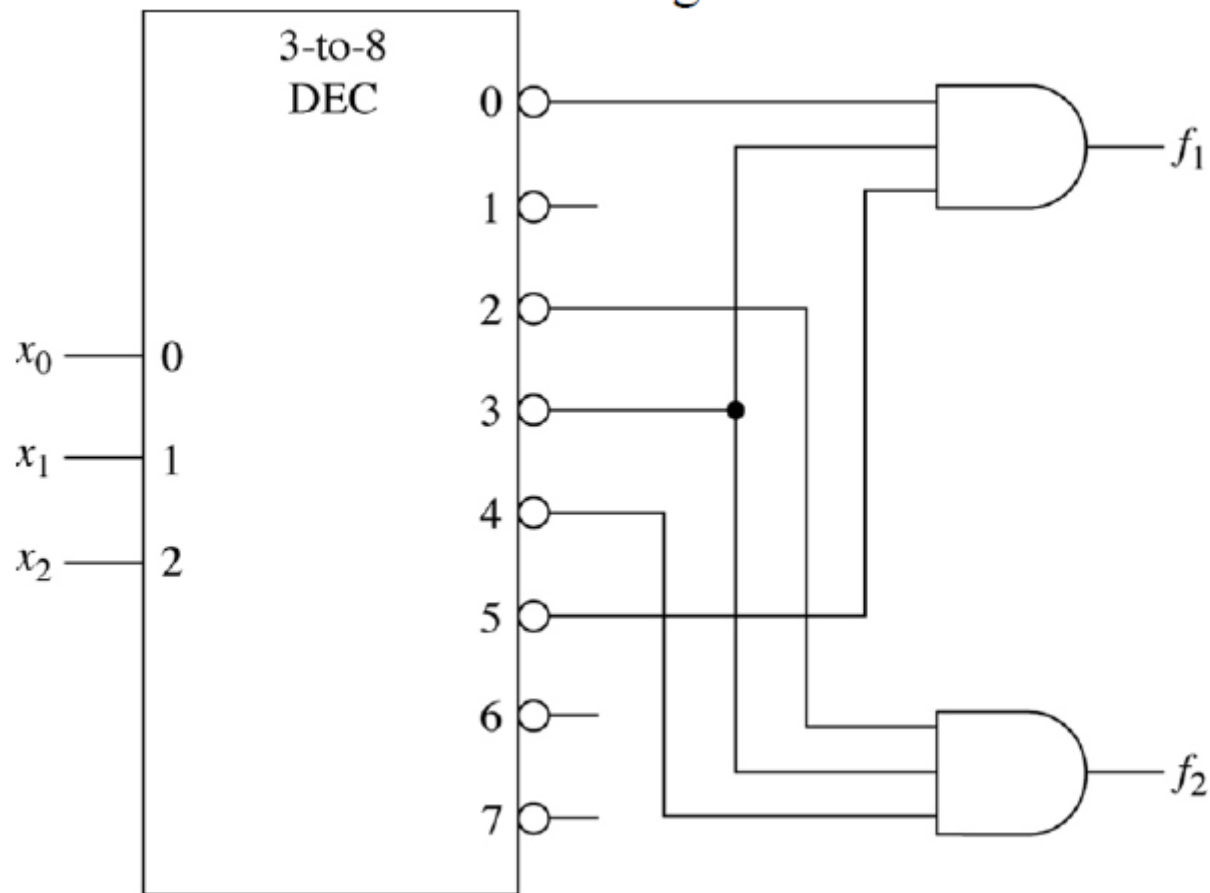
Design a Full Adder using a 2-to-4 Decoder.



Add two OR gate to design sum and carry equation.

Realization of the pair of maxterm canonical expressions
 $f_1(x_2, x_1, x_0) = \Pi M(0, 3, 5)$ and $f_2(x_2, x_1, x_0) = \Pi M(2, 3, 4)$ with a
3-to-8-line decoder and two and-gates.

Realization of the pair of maxterm canonical expressions $f_1(x_2, x_1, x_0) = \prod M(0, 3, 5)$ and $f_2(x_2, x_1, x_0) = \prod M(2, 3, 4)$ with a 3-to-8-line decoder and two and-gates.



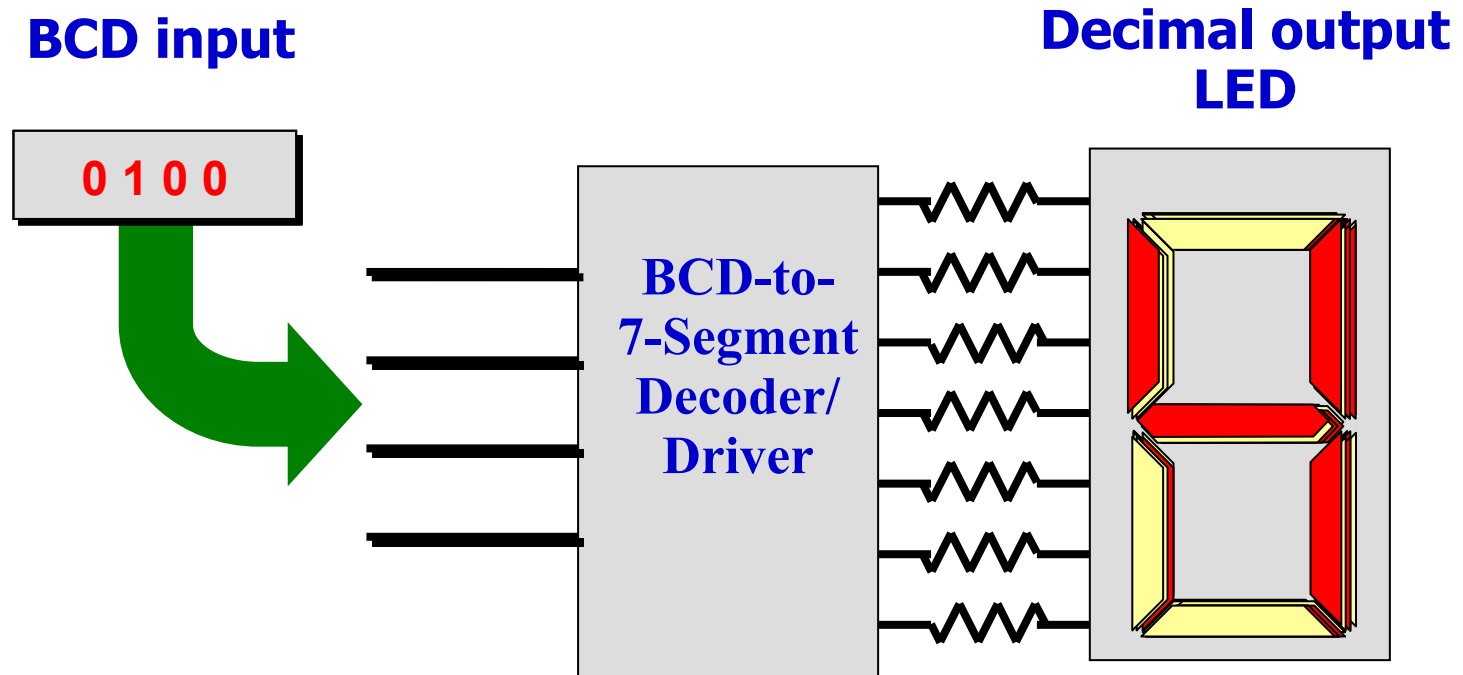
Decoders: Designing Logic Circuits

Design a circuit, using 2-to-4 Decoders to realize the Boolean expression given below.

$$F_{X,Y,Z} = \Sigma m(1,2,5,7)$$

Use two 2*4 decoder with enable Z

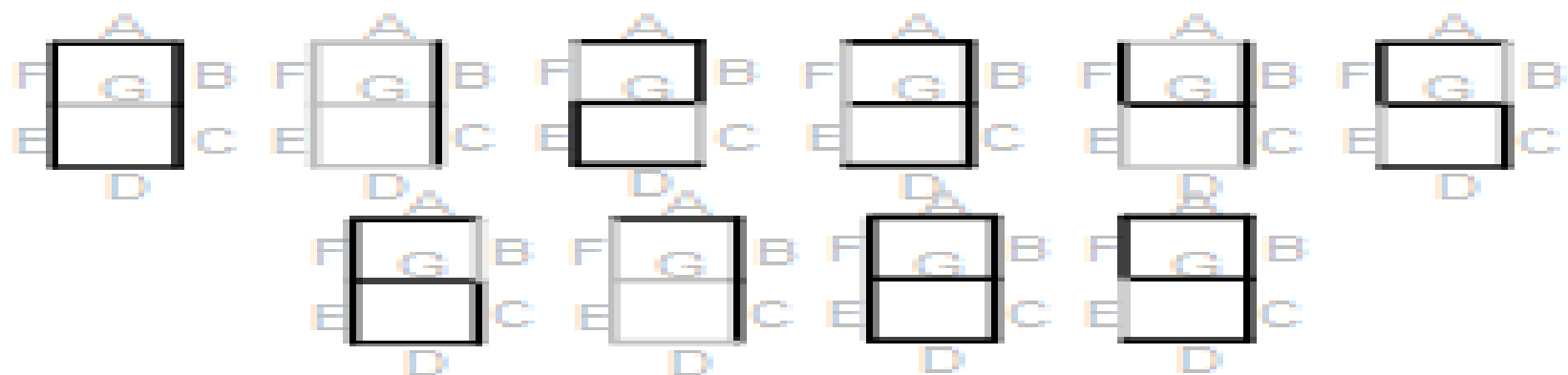
DECODERS: BCD TO 7-SEGMENT DECODER/DRIVER



- **Electronic decoders are available in IC form.**
- **This decoder translates from BCD to decimal.**
- **Decimals are shown on 7-segment LED display.**
- **This IC also drives the 7-segment LED display.**

Binary Inputs				Decoder Outputs							7-Segment Display Outputs
D	C	B	A	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	1	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	1	0	1	1	9

	BCD	A	B	C	D	E	F	G
0	0000	1	1	1	1	1	1	
1	0001		1	1				
2	0010	1	1		1	1		1
3	0011	1	1	1	1			1
4	0100		1	1			1	1
5	0101	1		1	1		1	1
6	0110	1		1	1	1	1	1
7	0111	1	1	1				
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1		1	1
10	1010	x	x	x	x	x	x	x
11	1011	x	x	x	x	x	x	x
12	1100	x	x	x	x	x	x	x
13	1101	x	x	x	x	x	x	x
14	1110	x	x	x	x	x	x	x
15	1111	x	x	x	x	x	x	x

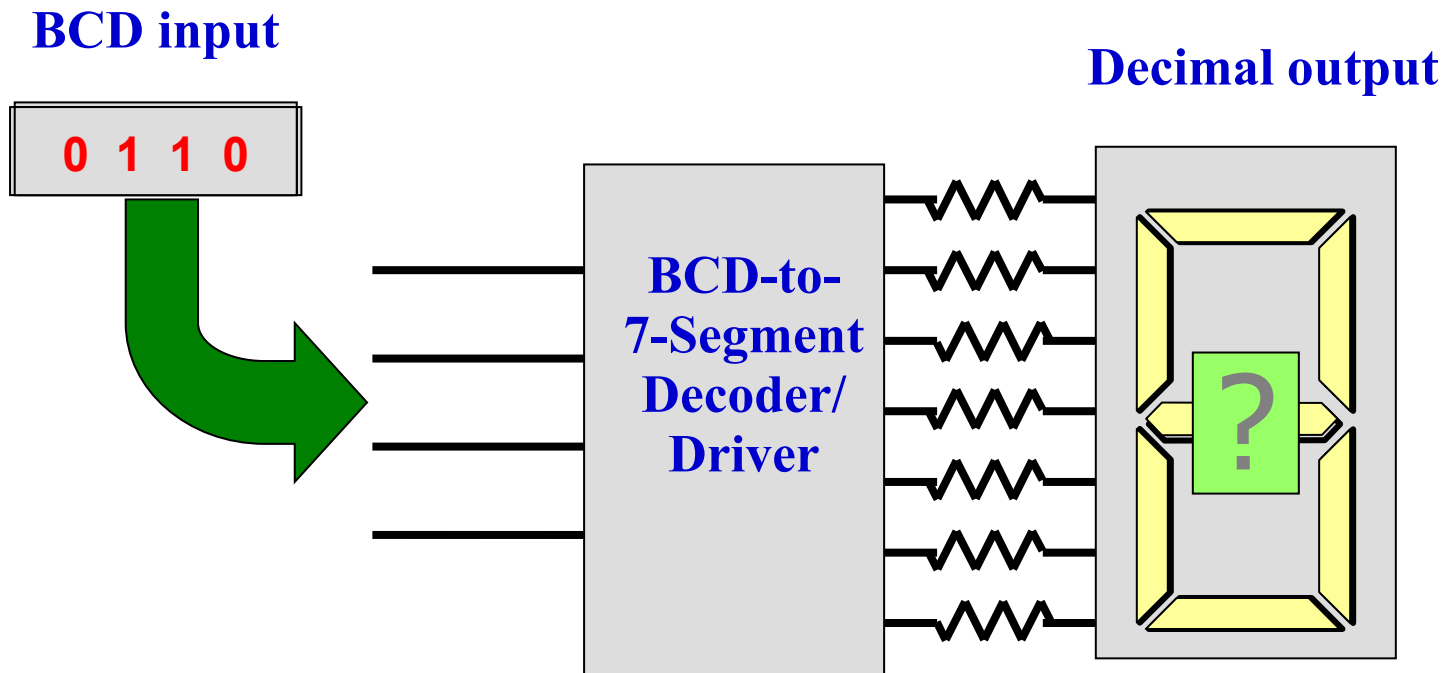




TEST

Q #5- What is the decimal output from the decoder that appears on the 7-segment display?

Answer: 6



Encoders

Encoder

An encoder outputs the binary value of the active input.

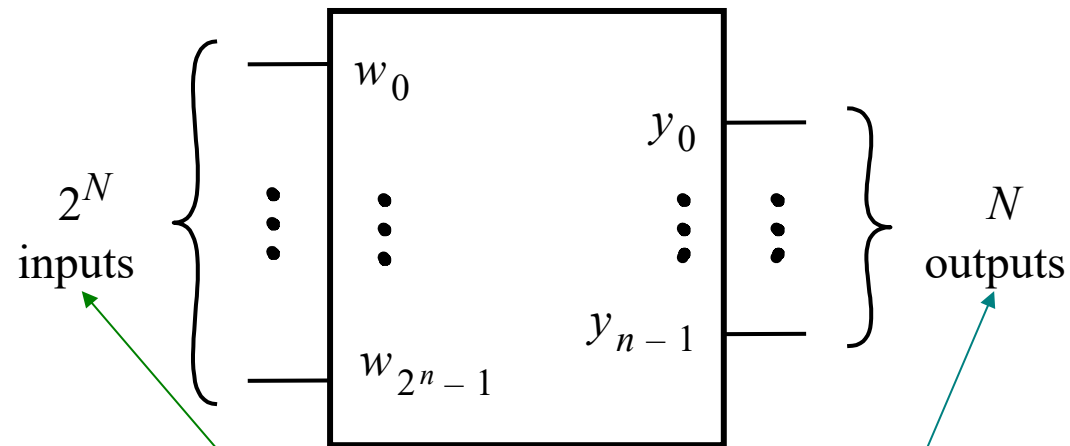
An encoder has 2^N inputs and N outputs.

If we record any music in any recorder, it means that we are giving data to a recorder. So, such process is called encoding.

Encoder

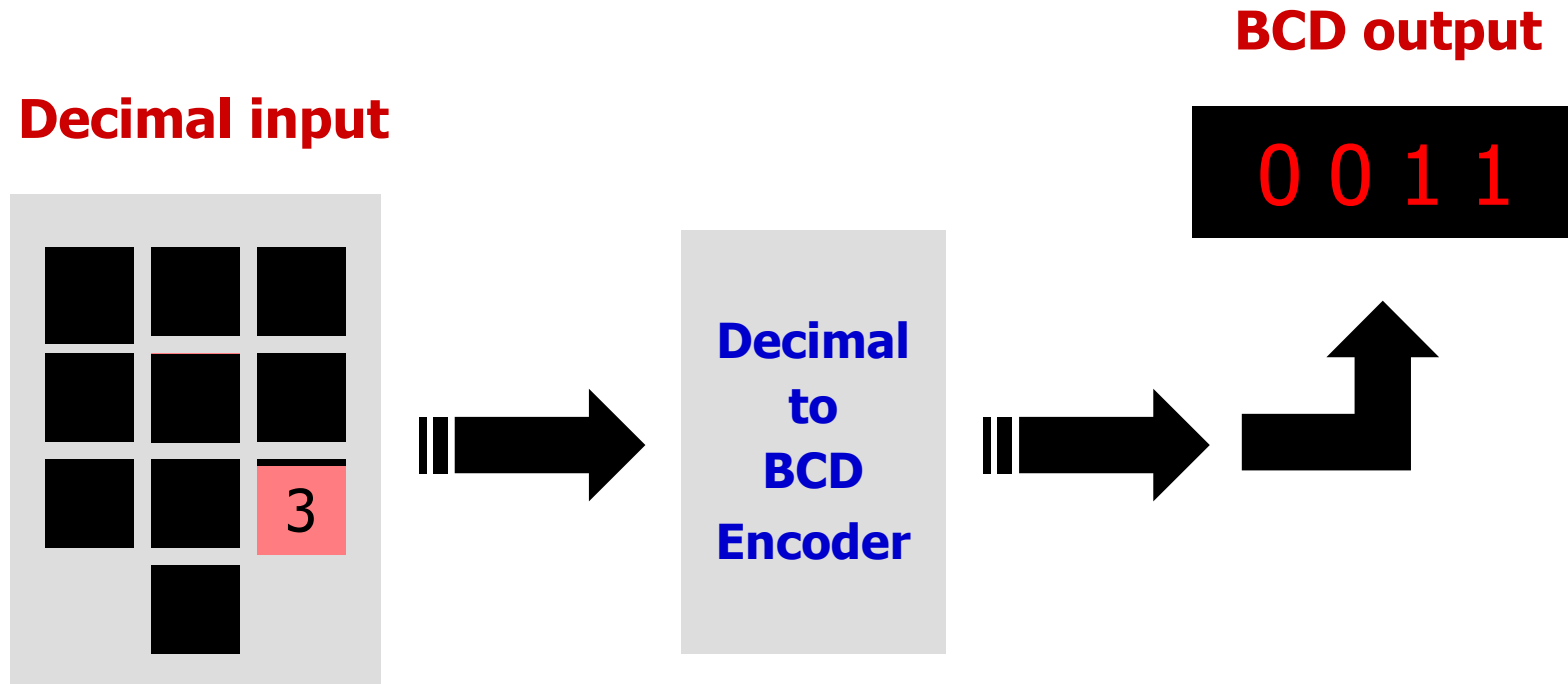
- Inverse operation of the decoder
 - Decoder selects one output based on the input binary number.
 - Encoder outputs a binary number based on the selected input.
- Issues
 - What if more than one input is active?
 - What if no inputs are active?

Encoder: 2^N -to- N



The active (or selected) input is encoded on the n outputs.

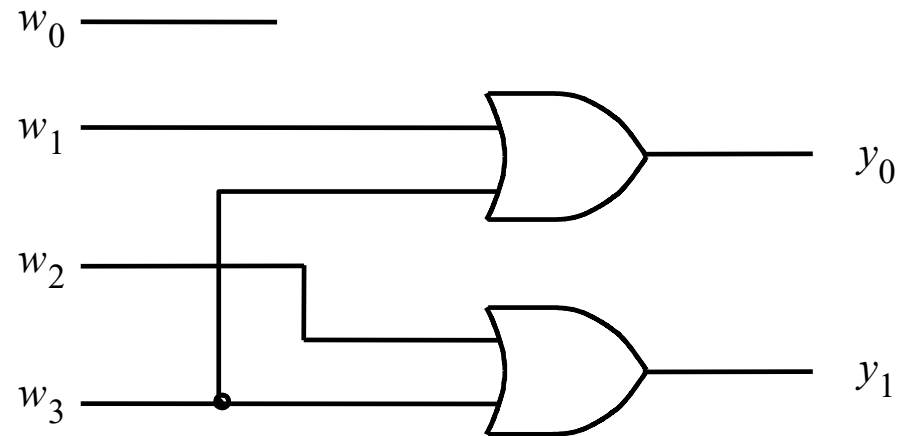
ELECTRONIC ENCODER - DECIMAL TO BCD



- Encoders are available in IC form.
- This encoder translates from decimal input to BCD output.

Encoder: 4-to-2

w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1



Encoder: 8-to-3

Table 4.7

Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Priority Encoder

- Higher-order input has priority of lower-order input
 - Order of the input determined by its binary value
 - I0 has binary value 000 (0)
 - I1 has binary value 001 (1)
 - I2 has binary value 010 (2)
 - I3 has binary value 011 (3) etc.
 - So,
 - I1 selected over I0
 - I2 selected over I1 and I0
 - I3 selected over I2, I1, and I0
 - etc.

Priority Encoders

- If more than one input is active, the higher-order input has priority over the lower-order input.
 - The higher value is encoded on the output
- A valid indicator, d , is included to indicate whether or not the output is valid.
 - Output is invalid when no inputs are active
 - $d = 0$
 - Output is valid when at least one input is active
 - $d = 1$

Why is the valid indicator needed?

Priority Encoder

- Valid indicator
 - Output signal of the (priority) encoder that indicates the validity of the encoded output
 - Encoded output is invalid when no inputs are selected or when the encoder is disabled
 - $V = 0$ (indicates invalid encoded output; active high)
 - Encoded output when one, or more, input(s) is (are) selected, and encoder is enabled
 - $V = 1$ (indicates valid encoded output; active high)

4-to-2 Priority Encoder

Table 4.8

Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Valid indicator

invalid

valid

4-to-2 Priority Encoder

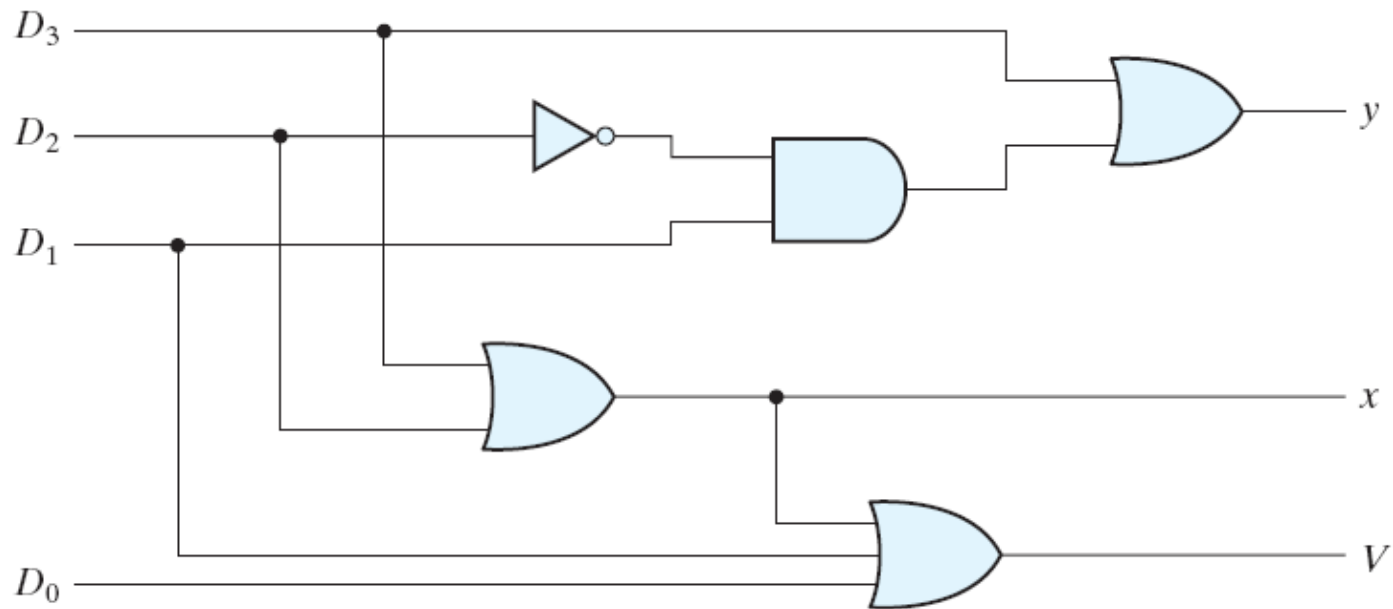
D_0D_1		D_2D_3			
		00	01	11	10
D_0	00	m_0 X	m_1 1	m_3 1	m_2 1
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12}	m_{13} 1	m_{15} 1	m_{14} 1
	10	m_8	m_9 1	m_{11} 1	m_{10} 1
		D_3			

$x = D_2 + D_3$

D_0D_1		D_2D_3			
		00	01	11	10
D_0	00	m_0 X	m_1 1	m_3 1	m_2
	01	m_4 1	m_5 1	m_7 1	m_6
	11	m_{12} 1	m_{13} 1	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11} 1	m_{10}
		D_3			

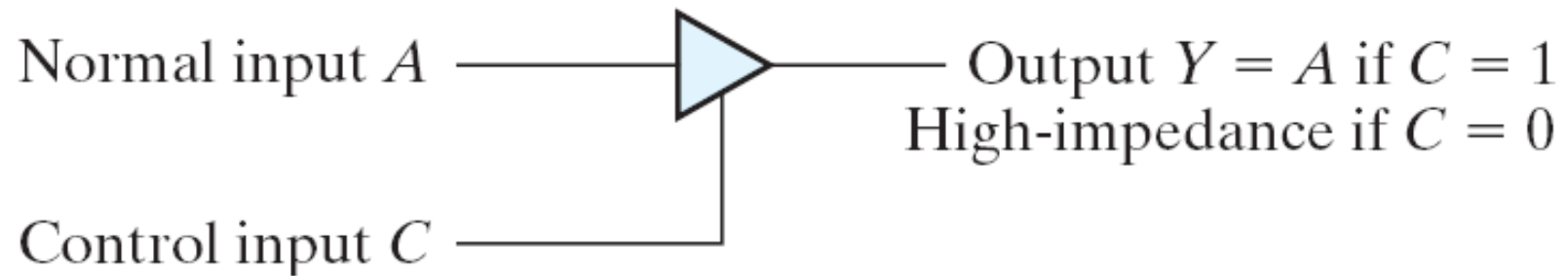
$y = D_3 + D_1D'_2$

4-to-2 Priority Encoder

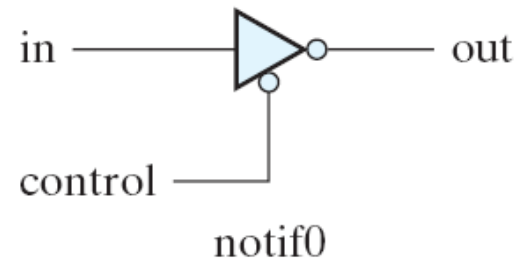
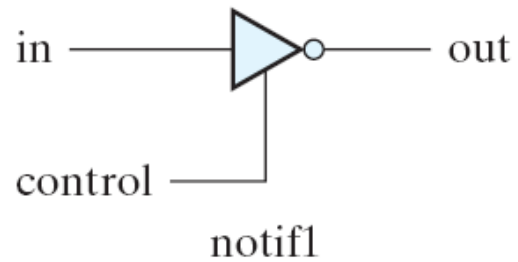
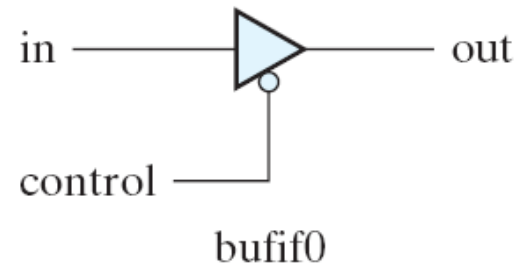
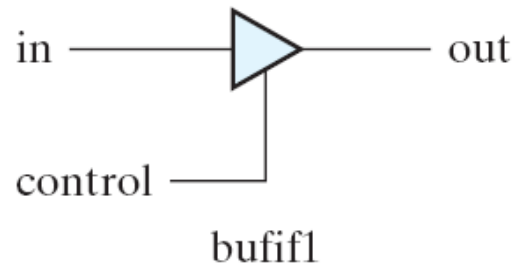


Tri-state Devices

Tri-state Devices



Tri-state Devices

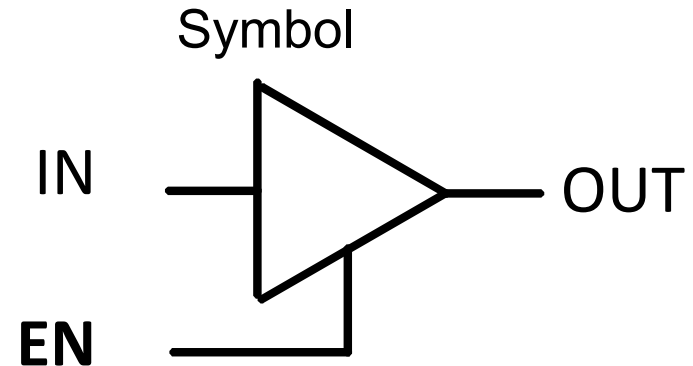


Hi-Impedance Output

- Logic gates introduced thus far ...
 - Have 1 and 0 output values
 - **Cannot** have their outputs connected together
- Three-state logic adds a third logic value:
 - Hi-Impedance output: **Z**
- What is Hi-Impedance output?
 - The output appears to be **disconnected** from the input
 - Behaves as an **open circuit** between gate input & output
- Hi-Z state makes a gate output behave differently:
 - Three output values: 1, 0, and Z
 - Hi-impedance gates **can connect** their outputs together

The 3-State Buffer

- IN = data input
- EN = Enable control input
- OUT = data output
- If EN = 0 then OUT = HI-Z
 - Regardless of the value on IN
 - Output disconnected from input
- If EN = 1, then OUT = IN
 - Output follows the input value
- Variations:
 - EN can be inverted
 - OUT can be inverted
 - By addition of bubbles to signals



Truth Table

EN	IN	OUT
0	X	Z
1	0	0
1	1	1